GOOGLE | 1POINT3ACRES, PRACHUB, HACKTHEROUNDS | TOPIC 1 OF 10 | CORE TOPIC

Heaps, Stacks and Queues

33 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1	24	8
EASY	MEDIUM	HARD

How to use

- Questions run from Easy to Medium to Hard, newest last-asked date first inside each level.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

Heaps, Stacks and Queues | 33 questions | tap a title to jump to it

E	1. Find k pairs with smallest sums	*Heap*	**PracHub**	**2025-11-01**
M	2. Visible People in a Line (Last Person, Then Everyone)	*Array*	**1Point3Acres**	**2026-10-07**
M	3. Meeting Rooms / Interval Scheduling Variants	*Interval*	**1Point3Acres**	**2026-09-29**
M	4. Active Communication Top Users	*Hashmap*	**1Point3Acres**	**2026-09-22**
M	5. Sliding-Window Mean After Removing the Largest K Values	*Heap*	**PracHub**	**2026-09-01**
M	6. Implement Python's itertools.tee	*Iterator*	**1Point3Acres**	**2026-08-20**
M	7. Union of Sorted Interval Lists	*Interval*	**1Point3Acres**	**2026-08-16**
M	8. Expand Recursive Key-Value Placeholders in a Template	*Stack*	**PracHub**	**2026-08-04**
M	9. Streaming Median (LC 295) with Follow-ups	*Heap*	**1Point3Acres**	**2026-08-03**
M	10. Decode String	*String*	**1Point3Acres**	**2026-07-29**
M	11. Batch and Streaming Log Deduplication	*Dedup*	**1Point3Acres**	**2026-07-23**
M	12. Top-K Frequent in Large Logs	*Heap*	**1Point3Acres**	**2026-07-19**
M	13. Find A Threshold-Limited Path With Minimum Required Safety	*Heap*	**PracHub**	**2026-07-06**
M	14. Nested Function Expression Evaluator	*Parsing*	**1Point3Acres**	**2026-06-10**
M	15. Solve Rooms and Top-K Streams	*Heap*	**PracHub**	**2026-06-05**
M	16. Rearrange Tasks With Cooldown	*Heap*	**PracHub**	**2026-06-03**
M	17. Determine if a 14-tile hand is winning	*Heap*	**PracHub**	**2026-03-09**
M	18. Implement frequency + distance top-K queries	*Heap*	**PracHub**	**2026-02-12**
M	19. Compute minimum number of rooms needed	*Heap*	**PracHub**	**2026-02-12**
M	20. Find longest increasing contiguous subarray	*Heaps, Stacks and Queues*	**PracHub**	**2026-01-06**
M	21. Simulate room assignments for scheduled meetings	*Heap*	**PracHub**	**2026-01-06**
M	22. Compute servers needed for daily recurring jobs	*Heap*	**PracHub**	**2025-11-05**
M	23. Compute shortest paths with blocked nodes	*Heap*	**PracHub**	**2025-09-06**
M	24. Meeting Rooms III Variant	*Heap*	**HackTheRounds**	**no date**
M	25. Playlist Shuffle No Adjacent Artists	*Heap*	**HackTheRounds**	**no date**
H	26. Median of Two Sorted Arrays (LC 4) with a Follow-up Variant	*Binary search*	**1Point3Acres**	**2026-10-09**
H	27. Compute Turnstile Crossing Times	*Heaps, Stacks and Queues*	**PracHub**	**2026-05-04**
H	28. Find minimum threshold enabling grid path	*Heap*	**PracHub**	**2026-02-11**
H	29. Compute minimax grid path and network delay	*Heap*	**PracHub**	**2026-01-22**
H	30. Return top-k subset sums in descending order	*Heaps, Stacks and Queues*	**PracHub**	**2026-01-07**
H	31. Minimize travel time with optional meeting point	*Heap*	**PracHub**	**2026-01-06**
H	32. Minimize maximum height along a grid path	*Heap*	**PracHub**	**2026-01-04**
H	33. Compute city skyline outline	*Heap*	**PracHub**	**2025-12-08**

STUDY HERE: topic 1, Heaps, Stacks and Queues

EASY | IN APP LC 373 | PracHub | Technical Screen | 1 of 33

Find k pairs with smallest sums

POSTED 2025-11-01 | 353 DAYS AGO | OLDER

STUDY: TALK ALL TIME 32.6 30 DAYS 0 PRIORITY 119/228

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #373 Find K Pairs with Smallest Sums

LeetCode match: LC 373 Find K Pairs with Smallest Sums

Posted on PracHub: 2025-11-01 (updated 2026-06-27)

Original: https://prachub.com/coding-questions/find-k-pairs-with-smallest-sums

QUESTION

You are given two **sorted** integer arrays **`arr1`** and **`arr2`** (each sorted in non-decreasing order), and an integer **`k`**. Consider all possible pairs formed by taking one element from **`arr1`** and one element from **`arr2`**. For indices **`i`** and **`j`**, the pair is **`(arr1[i], arr2[j])`** with sum **`arr1[i] + arr2[j]`**. Your task is to return the **`k`** **pairs `(arr1[i], arr2[j])`** with the smallest sums. If there are fewer than **`k`** total possible pairs, return all possible pairs. You may return the pairs in any order. ---

Function Signature

(one possible form)

```
kSmallestPairs(arr1: List[int], arr2: List[int], k: in
t) -> List[Tuple[int, int]]
```

Example

```
arr1 = [1, 7, 11] arr2 = [2, 4, 6] k = 3
```

All possible pairs and their sums are:

- (1, 2) -> 3 - (1, 4) -> 5 - (1, 6) -> 7 - (7, 2) -> 9 - (7, 4) -> 11 - (7, 6) -> 13 - (11, 2) -> 13 - (11, 4) -> 15 - (11, 6) -> 17 The 3 pairs with the smallest sums are:

```
[(1, 2), (1, 4), (1, 6)]
```

(The order among these returned pairs does not matter.) Explain an efficient algorithm to solve this problem and analyze its time and space complexity. You may assume the arrays are large, so a solution that explicitly generates and sorts all **`len(arr1) * len(arr2)`** pairs is too slow.

APPROACH

Min-heap seeded with (arr1[i]+arr2[0], i, 0) for the first min(k, len1) indices; pop, then push (i, j+1). O(k log k). Return all pairs if fewer than k exist.

HOW TO THINK ABOUT IT

Because both arrays are sorted, the pair (i, j + 1) is never smaller than (i, j), so each row i of the 'matrix of sums' is sorted. The k smallest pairs are the k smallest items of a collection of sorted rows, which is a heap problem: start with the first element of each row (or the first few) and each time you pop a pair, push its successor in the row.

Basis: LeetCode 373, Find K Pairs with Smallest Sums (two sorted arrays).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(a, b, k):
    pairs = [(x, y) for x in a for y in b]
    pairs.sort(key=lambda p: p[0] + p[1])
    return pairs[:k]
```

TIME	SPACE
O(m * n log(m * n))	O(m * n)

WHY IT IS SLOW

m * n pairs sorted: with 10^5 elements in each array that is 10^10 pairs. Because both arrays are sorted, a min-heap that starts from each x's first pair and expands only the next pair in that row is O(k log k).

OPTIMAL SOLUTION · Min-heap over the sorted rows

TIME	SPACE
O(k log min(k, m))	O(min(k, m))

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The arrays are not sorted.

Sort both first (O(m log m + n log n)); the heap method needs sorted rows. For one huge array you may instead use a heap of size k with a selection of candidates.

FOLLOW-UP 2 Only the k-th smallest sum, with huge arrays.

Binary search the sum value and count how many pairs have sum at most it with two pointers (O(m + n) per check): O((m + n) log(range)) with O(1) space, no heap.

FOLLOW-UP 3 Three arrays, or k pairs with the largest sums.

For largest sums run the same algorithm on the reversed arrays (or negate the values). For three arrays use a heap of triples (i, j, l), pushing the three neighbours with a visited set. **Cost:** O(k log k) with the heap of triples; reversing the arrays gives the largest sums at the same cost.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find k pairs with smallest sums
#
# You are given two **sorted** integer arrays arr1 and
# arr2 (each sorted in non-decreasing order), and an
# integer k. Consider all possible pairs formed by taking
# one element from arr1 and one element from arr2. For
# indices i and j, the pair is (arr1[i], arr2[j]) with sum
# arr1[i] + arr2[j]. Your task is to return the **k
# pairs** (arr1[i], arr2[j]) with the smallest sums. If
# there are fewer than k total possible pairs, return all
# possible pairs. You may return the pairs in any order.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The arrays are not sorted.
# 2. Only the k-th smallest sum, with huge arrays.
# 3. Three arrays, or k pairs with the largest sums.
#
# INTERVIEW NOTES
# Because both arrays are sorted, the pair (i, j + 1) is
# never smaller than (i, j), so each row i of the 'matrix
# of sums' is sorted. The k smallest pairs are the k
# smallest items of a collection of sorted rows, which is
# a heap problem: start with the first element of each row
# (or the first few) and each time you pop a pair, push
# its successor in the row.
#
# BRUTE FORCE (say it first): Generate all pairs and sort
# Build all m * n pairs, sort by sum and take k. Correct
# and the oracle; with 10^5 elements in each array it
# would create 10^10 pairs. Time O(m * n log(m * n)).
# Space O(m * n).


import heapq
from bisect import bisect_right


# BRUTE FORCE (say it first): Generate all pairs and sort
# | time O(m * n log(m * n))
def k_pairs_all(a, b, k):
    return sorted(
        ((x, y) for x in a for y in b),
        key=lambda p: (p[0] + p[1], p),
    )[:k]
```

```
# HELPERS for follow-up 2
def count_at_most(a, b, limit):
    # pairs with sum <= limit: for every x binary search
    # in b
    return sum(bisect_right(b, limit - x) for x in a)


# BASE QUESTION (optimal): Min-heap over the sorted rows
# Idea: Push (a[i] + b[0], i, 0) for the first min(k, m)
# values of a. Pop the smallest, record the pair, and push
# the next pair in the same row, (a[i] + b[j + 1], i, j +
# 1). After k pops you have the k smallest sums.
def k_smallest_pairs(a, b, k):
    # a and b sorted: a heap holds the best unused pair of
    # each row i; popping (i, j) offers (i, j + 1)
    if not a or not b or k <= 0:
        return []
    rows = range(min(k, len(a)))
    heap = [(a[i] + b[0], i, 0) for i in rows]
    heapq.heapify(heap)
    out = []
    while heap and len(out) < k:
        _, i, j = heapq.heappop(heap)
        out.append((a[i], b[j]))
        if j + 1 < len(b):
            nxt = a[i] + b[j + 1]
            heapq.heappush(heap, (nxt, i, j + 1))
    return out


# FOLLOW-UP 1: The arrays are not sorted.
# Answer: Sort both first (O(m log m + n log n)); the heap
# method needs sorted rows. For one huge array you may
# instead use a heap of size k with a selection of
# candidates.
def k_smallest_pairs_unsorted(a, b, k):
    # the heap method needs sorted rows: sort first
    return k_smallest_pairs(sorted(a), sorted(b), k)


# FOLLOW-UP 2: Only the k-th smallest sum, with huge
# arrays.
# Answer: Binary search the sum value and count how many
# pairs have sum at most it with two pointers (O(m + n)
# per check): O((m + n) log(range)) with O(1) space, no
# heap.
def kth_smallest_sum(a, b, k):
    # huge arrays: binary search the SUM, no heap needed
    lo, hi = a[0] + b[0], a[-1] + b[-1]
    while lo < hi:
        mid = (lo + hi) // 2
        if count_at_most(a, b, mid) >= k:
            hi = mid
        else:
            lo = mid + 1
    return lo


# FOLLOW-UP 3: Three arrays, or k pairs with the largest
# sums.
# Answer: For largest sums run the same algorithm on the
# reversed arrays (or negate the values). For three arrays
# use a heap of triples (i, j, l), pushing the three
# neighbours with a visited set.
def k_largest_pairs(a, b, k):
    # largest sums: negate the values and run the same
    # code
    neg_a = [-x for x in sorted(a, reverse=True)]
    neg_b = [-y for y in sorted(b, reverse=True)]
    pairs = k_smallest_pairs(neg_a, neg_b, k)
    return [(-x, -y) for x, y in pairs]


if __name__ == "__main__":
    a, b = [1, 7, 11], [2, 4, 6]
    three = k_smallest_pairs(a, b, 3)
    assert three == [(1, 2), (1, 4), (1, 6)]
    assert k_smallest_pairs(a, b, 4)[3] == (7, 2)
    assert k_smallest_pairs([1, 1, 2], [1, 2, 3], 2) == [
        (1, 1), (1, 1)]
    assert k_smallest_pairs([], [1], 3) == []
    # k bigger than the number of pairs: all of them
    assert len(k_smallest_pairs([1], [1, 2], 10)) == 2
    fixed = k_smallest_pairs_unsorted([7, 1], [4, 2], 2)
    assert fixed == [(1, 2), (1, 4)]
    assert k_largest_pairs(a, b, 2) == [(11, 6), (11, 4)]
    # the 4th smallest sum is 9 (7 + 2); check by sorting
    # all
    sums = sorted(x + y for x in a for y in b)
    for k in range(1, 10):
        assert kth_smallest_sum(a, b, k) == sums[k - 1]
```

STUDY HERE: topic 1, Heaps, Stacks and Queues

Site tags: array, monotonic-stack, stack, medium

MEDIUM | 1Point3Acres | tech-screen | 2 of 33

Visible People in a Line (Last Person, Then Everyone)

ASKED 2026-10-07 | 13 DAYS AGO | LAST 30 DAYS

STUDY: TALK **NO LEETCODE FREQUENCY** **PRIORITY 181/228**

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #1944 Number of Visible People in a Queue

LeetCode match: LC 1944 Number of Visible People in a Queue (related)

Frequency: Single report | Last asked: 2026-10-07 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/visible-people-in-line

QUESTION

Given heights of people in a line, A can see B if nobody between them is taller than both. Part 1: count how many people the last person can see. Part 2: return that count for every person. Written in a plain editor with no compilation.

Input: an array of heights representing a line of people.
Visibility rule: person A can see person B if no one standing between them is taller than both A and B.
Part 1: return how many people the last person in line can see.
Part 2: for the same array, return, for every person in the line, how many people they can see.

EXAMPLES

```
[1, 10, 6, 7, 9, 8, 2, 4, 3, 5] -> 6 (the last person,
    height 5, can see 3, 4, 2, 8, 9, 10).
```

APPROACH

Write a brute-force Part 1 that checks every earlier person against the rule, then look for a single-pass version before attempting Part 2.
Extend the Part 1 solution to Part 2 and state its complexity before coding.
Hand-trace the example above without running code, including a tie case you define yourself.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what blocks the view (someone strictly taller than both), how ties are treated, which directions count in Part 2, and whether the code runs (it does not: trace by hand).

BRUTE FORCE TO SAY OUT LOUD

Check every pair against the people between them. O(n^2) for the last person, O(n^3) for everyone.

INTERVIEW-DAY PLAN

OPTIMAL IDEA

Part 1: walk from the person before the last towards the front, keeping the maximum height between the last person and the current one; the current person is visible when that maximum is at most the larger of the two heights. O(n). Part 2: for each person, everyone up to the first strictly taller person on the right is visible, then only people at least as tall as everyone from there on; a monotonic stack gives the next at-least-as-tall and next strictly-taller indexes and the chain length in O(n). Run it on the reversed line for the left side and add.

DRY-RUN THESE CASES

A single person, all equal heights, strictly increasing and decreasing lines, the example from the statement (the last person sees 6), a tie case you invent.

LIKELY FOLLOW-UPS

Only looking in one direction, blocking on equal heights, or returning who they see instead of how many.

PITFALLS

Using a strict comparison where the rule says taller than both. Forgetting that the first taller person is itself visible. Rereading the example without tracing it by hand.

HOW TO THINK ABOUT IT

Person A sees person B when nobody strictly between them is taller than both, which means the tallest person between them is at most as tall as the taller of A and B. Walking away from the last person and keeping the running maximum of everyone passed lets you test each person in O(1). For everyone at once the visible people to the right form a chain: everything up to the first taller person, then only the people who are at least as tall as everyone before them. A monotonic stack builds those chains for all positions in O(n).

Basis: LeetCode 1944, Number of Visible People in a Queue, with the rule that a person is hidden only by someone taller than both.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def sees(a, i, j):
    lo, hi = min(i, j), max(i, j)
    for k in range(lo + 1, hi):
        if a[k] > max(a[i], a[j]):
            return False
    return True
def count_for(a, i):
    return sum(1 for j in range(len(a))
               if j != i and sees(a, i, j))
```

TIME	SPACE
O(n^3) for everyone, O(n^2) for the last person	**O(1)**

WHY IT IS SLOW

BRUTE FORCE

Each pair rescans the people between it, O(n^3) for everyone and O(n^2) for the last person. One pass with a running maximum answers the last person in O(n), and a monotonic stack answers everyone in O(n).

OPTIMAL SOLUTION · One pass with a running maximum (last person)

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Part 2: return the number of people each person can see.

Run the right-looking count for every position with a monotonic stack, run it again on the reversed line for the looking-left count, and add the two. The stack makes both passes O(n), instead of the O(n^2) you would get by repeating the last-person scan for every position.

FOLLOW-UP 2 How do ties and the direction of sight change the answer?

Under the stated rule only someone strictly taller than both blocks the view, so equal heights never block: in [5, 5, 5] the first person sees both others. If the interviewer wants 'taller or equal blocks', replace the non-strict comparisons with strict ones. If a person only looks towards the front, drop the reversed pass and use see_right on the reversed line alone.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Visible People in a Line (Last Person,
# Then Everyone)
#
# Given heights of people in a line, A can see B if nobody
# between them is taller than both. Part 1: count how many
# people the last person can see. Part 2: return that
# count for every person. Written in a plain editor with
# no compilation. Input: an array of heights representing
# a line of people. Visibility rule: person A can see
# person B if no one standing between them is taller than
# both A and B. Part 1: return how many people the last
# person in line can see. Part 2: for the same array,
# return, for every person in the line, how many people
# they can see. Examples: [1, 10, 6, 7, 9, 8, 2, 4, 3, 5]
# -> 6 (the last person, height 5, can see 3, 4, 2, 8, 9,
# 10).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Part 2: return the number of people each person can
# see.
# 2. How do ties and the direction of sight change the
# answer?
#
# INTERVIEW NOTES
# Person A sees person B when nobody strictly between them
# is taller than both, which means the tallest person
# between them is at most as tall as the taller of A and
# B. Walking away from the last person and keeping the
# running maximum of everyone passed lets you test each
# person in O(1). For everyone at once the visible people
# to the right form a chain: everything up to the first
# taller person, then only the people who are at least as
# tall as everyone before them. A monotonic stack builds
# those chains for all positions in O(n).
#
# BRUTE FORCE (say it first): Test every pair
# For a pair (i, j) look at everyone between them and
# check that none is taller than both. Counting over all
# pairs is O(n^3). Fine as the oracle and for hand-tracing
# the example. Time O(n^3) for everyone, O(n^2) for the
# last person. Space O(1).


# BRUTE FORCE (say it first, it also checks the answers)
def sees(a, i, j):
    # the oracle: nobody strictly between is taller than
    # both
    lo, hi = min(i, j), max(i, j)
    return all(a[k] <= max(a[i], a[j])
               for k in range(lo + 1, hi))


# BRUTE FORCE (say it first, it also checks the answers)
def counts_brute(a):
    return [sum(1 for j in range(len(a))
                if j != i and sees(a, i, j))
            for i in range(len(a))]


# HELPERS for the base question
INF = float("inf")


# BASE QUESTION (optimal): One pass with a running maximum
# (last person)
# Idea: Walk from the person next to the last one towards
# the front, keeping the largest height seen strictly
# between the last person and the current one.
def last_sees(a):
    """Part 1: walk away from the last person keeping the
    tallest height seen strictly between us and them."""
    if not a:
        return 0
    last, between, count = a[-1], -INF, 0
    for k in range(len(a) - 2, -1, -1):
        # nobody between is taller than both of us
        if between <= max(last, a[k]):
            count += 1
        between = max(between, a[k])
    return count


# FOLLOW-UP 1: Part 2: return the number of people each
# person can see.
# Answer: Run the right-looking count for every position
# with a monotonic stack, run it again on the reversed
# line for the looking-left count, and add the two.
def see_right(a):
    """Part 2, looking right: everyone up to the first
    strictly taller person is visible, then only people at
    least as tall as everyone from that person on."""
    n = len(a)
```

```
    next_ge, next_gt = [None] * n, [None] * n
    stack = []
    for k in range(n - 1, -1, -1):
        # next person at least as tall as person k
        while stack and a[stack[-1]] < a[k]:
            stack.pop()
        next_ge[k] = stack[-1] if stack else None
        stack.append(k)
    stack = []
    for k in range(n - 1, -1, -1):
        # next person strictly taller than person k
        while stack and a[stack[-1]] <= a[k]:
            stack.pop()
        next_gt[k] = stack[-1] if stack else None
        stack.append(k)
    chain = [0] * n
    for k in range(n - 1, -1, -1):
        # how many "at least as tall" steps follow k
        if next_ge[k] is not None:
            chain[k] = 1 + chain[next_ge[k]]
    out = []
    for i in range(n):
        t = next_gt[i]
        if t is None:
            out.append(n - 1 - i)
        else:
            out.append(t - i + chain[t])
    return out


# FOLLOW-UP 1: Part 2: return the number of people each
# person can see.
# Answer: Run the right-looking count for every position
# with a monotonic stack, run it again on the reversed
# line for the looking-left count, and add the two.
def see_all(a):
    # looking left is looking right on the reversed line
    left = see_right(a[::-1])[::-1]
    return [x + y for x, y in zip(left, see_right(a))]


if __name__ == "__main__":
    line = [1, 10, 6, 7, 9, 8, 2, 4, 3, 5]
```

```
    assert last_sees(line) == 6
    assert see_all(line)[-1] == 6
    assert last_sees([]) == 0 and last_sees([7]) == 0
    # equal heights never block each other
    assert last_sees([5, 5, 5]) == 2
    assert see_all([5, 5, 5]) == [2, 2, 2]
    import random
    for _ in range(300):
        a = [random.randint(1, 6)
             for _ in range(random.randint(0, 9))]
        assert see_all(a) == counts_brute(a)
        if a:
            assert last_sees(a) == counts_brute(a)[-1]
```

NOTES FROM CANDIDATES

Pitfalls: The code is written in a plain text editor that does not compile or run; trace the example by hand before declaring the solution done.
Confirm the direction of "see" for Part 2 (toward the front only, or both directions) and how equal heights are treated before coding; the example does not exercise ties.

Follow-ups: Generalize from the last person to every person in the line (Part 2).

PREPARATION

Write a brute-force Part 1 that checks every earlier person against the rule, then look for a single-pass version before attempting Part 2.
Extend the Part 1 solution to Part 2 and state its complexity before coding.
Hand-trace the example above without running code, including a tie case you define yourself.

STUDY HERE: topic 1, Heaps, Stacks and Queues

Site tags: interval, heap, greedy, medium

MEDIUM | IN APP LC 253 | 1Point3Acres | phone-screen, tech-screen, onsite-coding | 3 of 33

Meeting Rooms / Interval Scheduling Variants

ASKED 2026-09-29 | 21 DAYS AGO | LAST 30 DAYS

STUDY: TYPE **ALL TIME 58** **30 DAYS 0** **PRIORITY 44/228**

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues

Optimal answer from: your app's interview approach (LeetCode #253)

LeetCode match: LC 253 Meeting Rooms II

Frequency: Very high | Last asked: 2026-09-29 | Roles: SWE, MLE

Original: https://www.1point3acres.com/interview/problems/company/google/meeting-rooms-variants

QUESTION

LC 253 (Meeting Rooms II) appears in many forms across phone screen and onsite coding rounds — minimum number of rooms, room assignment, cyclic time, daily server jobs, gate entry/exit. Common follow-ups push toward sweep line, min-heap, or distributed processing.

The base task in every variant: given a list of intervals (start, end) — or pickup/return pairs, meeting time pairs, server job runs — answer one or more of:

Min rooms / cars / servers needed to honor all intervals concurrently (the canonical min-meeting-rooms problem).
Assignment: which interval goes to which room/car, so the caller can use the rental record.
Single-interval overlap test between two intervals.

Variant: parking-lot capacity

Arrival and departure intervals represent cars occupying a parking lot. Compute the peak simultaneous occupancy using the same interval-overlap model.

Variant: cyclic time (24h wrap)

Intervals wrap around midnight; if an interval ends after the day boundary, it carries into the next day.
Follow-up: "if N is huge, how do you optimize?" — expect a sweep-line discussion.

Variant: daily-recurring server jobs

Same set of jobs runs every day. If a job can't finish in one day, truncate at midnight and resume next day.
A server can run multiple identical jobs that overlap on the same machine; different jobs need separate servers.
Follow-up: a single job that runs many days needs its own dedicated server.

Variant: on-call rotation consolidation

QUESTION

Input: on-call rotations {name, start, end} using half-open intervals.
Output maximal contiguous segments during which the active on-call set does not change; omit gaps with nobody on call.
Segments are sorted by start, and names in each segment are the active set for that time range.
Clean approach: sort starts, keep a min-heap by end time, and emit a segment whenever the next start or next end changes the active set.

Variant: gate entry / exit with priority rules

Array of (timestamp, action) where action in {enter, exit}; each index is a different person.
Return for each person what time they actually enter/exit, applying tie-break rules:

If the prior tick had an enter, enter wins on a tie.
If the prior tick had an exit, exit wins on a tie.
If the prior tick had no activity, exit wins.
Within the winning class, lower index wins.

Optimization: instead of stepping time tick-by-tick, jump through the sorted input timestamps to reach O(n).

Variant: Google Calendar free-time intersection

Input: requested attendees plus each person's busy calendar intervals.
Return every time range where all requested attendees are free.
Normalize each person's busy blocks into free gaps, then intersect the candidate ranges across attendees.

Variant: car-rental assignment

Input: rental orders containing an order ID, pickup time, and return time.
Assign every order to a car so that no assigned intervals overlap, minimize the number of cars, and return each car's complete order list. Model each car explicitly with its own rental record.
A current output variant instead returns a dictionary from each reservation ID to its assigned car ID; the follow-up asks for the final fleet size.
A return at time t and another pickup at time t may reuse the same car.
Follow-up: if the fleet is capped at K cars and all requests cannot be served, define how the allocator detects and handles the failure.

Variant: exact-concurrency intervals

Compute the maximum number of simultaneously active meetings.
Given a target concurrency X, also return every maximal time range during which exactly X meetings are active.
Define the event ordering when one meeting starts at the same timestamp another ends; this choice controls whether a zero-width overlap is counted.

Variant: maximum-profit job scheduling

QUESTION

Each job has startTime, endTime, and profit. Choose a set of non-overlapping jobs maximizing total profit. The candidate identified it as LC 1235 (Maximum Profit in Job Scheduling). Asked as the first question of an L4 onsite round, followed by an unrelated range-query problem in the same slot.

On-call employee metadata variant

Each duty record includes employee ID, employee name, start time, and end time. Return the employee count and identifying information for each time segment.

EXAMPLES

```
Canonical min-meeting-rooms example:
    [[0,30],[5,10],[15,20]] -> 2.
On-call consolidation: A[10,50], B[20,60], C[30,40],
    D[30,40] -> [10,20]: A, [20,30]: A,B, [30,40]:
    A,B,C,D, [40,50]: A,B, [50,60]: B.
Single-interval overlap:
    intervalsOverlap([1,5],[4,7]) -> true.
```

Long-running job: a 36-hour daily job is impossible under the truncate rule; reserve a separate server.

APPROACH

Drill three solutions cold: sweep-line / prefix delta, min-heap on end time, sort + greedy room assignment.
Be ready to convert between the three when interviewers add follow-ups ("give me the assignment", "now handle cyclic days").
Prepare a 1-minute pitch on map-reduce style sharding (per-shard intervals -> merged timeline -> global max).

HOW TO THINK ABOUT IT

The number of rooms needed is the largest number of intervals overlapping at any moment. Turn each interval into +1 at its start and -1 at its end, sort the events (ends before starts at the same time) and keep a running count. To say which room each meeting gets, keep a min-heap of (end time, room) and reuse the room that frees up first.

Basis: LeetCode 253, Meeting Rooms II, with the common variants: room assignment and cyclic (24-hour) time.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(intervals):
    best = 0
    for start, _ in intervals:
        busy = 0
        for s, e in intervals:
            if s <= start < e:
                busy += 1
        best = max(best, busy)
    return best
```

TIME	SPACE
O(n^2)	O(1)

BRUTE FORCE

WHY IT IS SLOW

Every interval is compared with every other, O(n^2). Sorting start and end events and keeping a running count is O(n log n).

OPTIMAL SOLUTION · Sweep line over start and end events

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Meetings wrap around midnight (cyclic time).

Split every wrapped meeting [s, e) with s >= e into [s, 24h) and [0, e) and run the same sweep: the overlap at any clock time is unchanged. A meeting that is exactly 24 hours long covers the whole day.

FOLLOW-UP 2 The input is huge and spread over many machines.

Each shard sorts its events and produces a compressed timeline of (time, delta) changes; merge the sorted timelines with a k-way merge and take the running maximum, or add the per-shard counts at fixed time buckets. The answer is a max over time of a sum over shards.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Meeting Rooms / Interval Scheduling
# Variants
#
# LC 253 (Meeting Rooms II) appears in many forms across
# phone screen and onsite coding rounds — minimum number
# of rooms, room assignment, cyclic time, daily server
# jobs, gate entry/exit. Common follow-ups push toward
# sweep line, min-heap, or distributed processing. The
# base task in every variant: given a list of intervals
# (start, end) — or pickup/return pairs, meeting time
# pairs, server job runs — answer one or more of: Min
# rooms / cars / servers needed to honor all intervals
# concurrently (the canonical min-meeting-rooms problem).
# Assignment: which interval goes to which room/car, so
# the caller can use the rental record. Single-interval
# overlap test between two intervals. Variant: parking-lot
# capacity Arrival and departure intervals represent cars
# occupying a parking lot. Compute the peak simultaneous
# occupancy using the same interval-overlap model.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Meetings wrap around midnight (cyclic time).
# 2. The input is huge and spread over many machines.
#
# INTERVIEW NOTES
# The number of rooms needed is the largest number of
```

```python
# intervals overlapping at any moment. Turn each interval
# into +1 at its start and -1 at its end, sort the events
# (ends before starts at the same time) and keep a running
# count. To say which room each meeting gets, keep a min-
# heap of (end time, room) and reuse the room that frees
# up first.
#
# BRUTE FORCE (say it first): Count the overlap at every
# start time
# For every interval count how many intervals contain its
# start point and take the maximum. Correct and simple;
# quadratic. Time O(n^2). Space O(1).


import heapq
from collections import Counter


# BRUTE FORCE (say it first): Count the overlap at every
```

```python
# start time | time O(n^2)
def min_rooms_brute(intervals):
    return max(
        (
            sum(
                1
                for s2, e2 in intervals
                if s2 <= s < e2
            )
            for s, e in intervals
        ),
        default=0,
    )


# BASE QUESTION (optimal): Sweep line over start and end
# events
# Idea: Create (time, +1) for each start and (time, -1)
# for each end, sort so that an end at time t comes before
# a start at t (rooms free up the moment a meeting ends)
# and track the running maximum.
def min_rooms(intervals):
    # +1 at every start, -1 at every end
    events = []
    for s, e in intervals:
        events.append((s, 1))
        events.append((e, -1))
    # at equal times ends (-1) sort before starts (+1)
    events.sort()
    cur = best = 0
    for _, delta in events:
        cur += delta
        best = max(best, cur)
    return best


# VARIANT: which room for each meeting
def assign_rooms(intervals):
    # which room does each meeting get?
    order = sorted(range(len(intervals)),
                   key=lambda i: intervals[i])
    heap, rooms, used = [], [0] * len(intervals), 0
    for i in order:
        start, end = intervals[i]
        # reuse the room that frees first, if it is free
```

```python
        if heap and heap[0][0] <= start:
            _, room = heapq.heappop(heap)
        else:
            room, used = used, used + 1
        rooms[i] = room
        heapq.heappush(heap, (end, room))
    return rooms
```

```python
# VARIANT: on-call consolidation
def oncall_segments(rotations):
    # rotations: (name, start, end), half-open intervals
    delta = {}
    for name, s, e in rotations:
        delta.setdefault(s, []).append((name, 1))
        delta.setdefault(e, []).append((name, -1))
    active, out, times = Counter(), [], sorted(delta)
    for t, nxt in zip(times, times[1:]):
        # apply every change that happens at time t
        for name, d in delta[t]:
            active[name] += d
        who = frozenset(n for n in active if active[n])
        # skip gaps; merge with the previous equal segment
        if not who:
            continue
        if out and out[-1][1] == t and out[-1][2] == who:
            out[-1] = (out[-1][0], nxt, who)
        else:
            out.append((t, nxt, who))
    return out


# FOLLOW-UP 1: Meetings wrap around midnight (cyclic
# time).
# Answer: Split every wrapped meeting [s, e) with s >= e
# into [s, 24h) and [0, e) and run the same sweep: the
# overlap at any clock time is unchanged. A meeting that
# is exactly 24 hours long covers the whole day.
def min_rooms_cyclic(intervals, day=24 * 60):
    parts = []
    for s, e in intervals:
        if s < e:
            parts.append((s, e))
        else:
            # wraps past midnight: split at the boundary
```

```python
            parts += [(s, day), (0, e)]
    return min_rooms(parts)


if __name__ == "__main__":
    base = [(0, 30), (5, 10), (15, 20)]
    assert min_rooms(base) == 2
    # touching meetings share a room
    assert min_rooms([(5, 8), (8, 9)]) == 1
    rooms = assign_rooms(base)
    assert rooms[0] != rooms[1] and rooms[1] == rooms[2]
    wrap = [(22 * 60, 2 * 60), (23 * 60, 60), (0, 3 * 60)]
    assert min_rooms_cyclic(wrap) == 3
    rot = [("A", 1, 5), ("B", 3, 8)]
    assert oncall_segments(rot) == [
        (1, 3, frozenset("A")),
        (3, 5, frozenset("AB")),
        (5, 8, frozenset("B"))]
    # a gap with nobody on call is left out
    gap = [("A", 1, 2), ("B", 4, 5)]
    assert len(oncall_segments(gap)) == 2
```

NOTES FROM CANDIDATES

Asked in Round 1 coding, Round 2 onsite coding, phone screen, and as a warm-up in the FDE loop. Almost every other coding loop contains some flavor of this family.
Interviewer commonly chains two follow-ups: (1) reconstruct assignment, (2) scale out to many shards / regions.
For the assignment follow-up, min-heap keyed on end time is the clean answer: pop the earliest-ending room, push the new interval.
For "distributed across shards", expect open-ended discussion: per-shard sweep, cross-shard merge, reconciliation; the interviewer often acknowledges the question is intentionally vague.

The on-call metadata variant includes detailed complexity follow-ups: account for heap operations, stored employee information, and output size separately, and explain each symbol in the complexity expression.

PREPARATION

Drill three solutions cold: sweep-line / prefix delta, min-heap on end time, sort + greedy room assignment.
Be ready to convert between the three when interviewers add follow-ups ("give me the assignment", "now handle cyclic days").
Prepare a 1-minute pitch on map-reduce style sharding (per-shard intervals -> merged timeline -> global max).

STUDY HERE: topic 1, Heaps, Stacks and Queues

Site tags: hashmap, set, sorting, top-k, streaming, medium

MEDIUM | DESIGN-STYLE (skippable) | 1Point3Acres | phone-screen | 4 of 33

Active Communication Top Users

ASKED 2026-09-22 | 28 DAYS AGO | LAST 30 DAYS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 228/228**

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-09-22 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/active-communication-top-users

QUESTION

Process a continuous stream of user pairs such as **`A-B`** and **`B-C`**; each pair creates an active communication edge, and the system should output users ordered by current communication count. Follow-ups add self-communication handling, duplicate communication dedupe, and top-K users by count. An additional API variant exposes register and mostActiveUser(), then asks for constant-time most-active lookup and a changing-K ranking extension.SWEhashmapsetsortingtop-kstreamingmediumFrequencyLowLast asked2026-09-22Stagephone-screen

APPROACH

Implement the pair-normalization layer separately from count maintenance.
Practice maintaining user -> count, edge set, and count -> ordered users views without letting updates drift.
Dry-run duplicate edges and self-edges before discussing top-K optimization.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether a pair A-B counts for both users, whether repeated pairs count once, how self communication is treated, how ties order, and whether k changes over time.

BRUTE FORCE TO SAY OUT LOUD

Recount from all edges and sort users on every query. O(E log U).

OPTIMAL IDEA

Normalize pairs (sorted tuple). Keep an edge set for deduplication, a dict of user to count, and either a sorted structure of (count, user) or a bucket map count to ordered users so an update moves a user by one bucket. Top-K reads the highest buckets. Note the edge set can use quadratic space in the number of users; say so.

DRY-RUN THESE CASES

INTERVIEW-DAY PLAN

A-B then B-A (duplicate), a self pair, ties on count, a new user, k larger than the number of users.

LIKELY FOLLOW-UPS

Constant-time most active user (track the maximum count with buckets), a changing k, or expiry of old edges.

PITFALLS

Claiming linear space while storing all pairs. Counting a duplicate edge twice.

HOW TO THINK ABOUT IT

Normalize the pair first (sorted tuple) so A-B and B-A are one edge, then keep an edge set for dedup and a count per user. The ranking part is a separate question: top-K comes from a heap, and the single most active user can be maintained in O(1) because counts only ever go up. Mention that the edge set uses quadratic space in the worst case.

Basis: Written. Assumed rules (confirm them): a pair 'A-B' is an unordered communication edge; repeated pairs count once; a user's activity is the number of distinct partners; order by activity descending, then name; self pairs (A-A) are ignored unless a flag says otherwise.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
pairs = set()
def add(a, b):
    pairs.add((min(a, b), max(a, b)))
def top_users():
    partners = defaultdict(set)
    for a, b in pairs:
        partners[a].add(b)
        partners[b].add(a)
    def rank(u):
        return (-len(partners[u]), u)
    return sorted(partners, key=rank)
```

TIME	SPACE
O(E + U log U) per query	**O(E)**

WHY IT IS SLOW

Each query rebuilds all counts and sorts, O(E + U log U). Maintaining counts incrementally with count buckets gives O(1) updates and an O(1) most-active lookup.

OPTIMAL SOLUTION · Edge set, counts per user, count buckets for the maximum

TIME	SPACE
O(1) per pair, O(U log k) for top-K, O(1) for the most active user	**O(E)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Self communication (A-A) should now count.

Pass a flag to the class: a self pair is one edge that adds one to that user's count (and only once). The dedup set already handles repeated self pairs.

FOLLOW-UP 2 Top-K with a changing K, or a fixed-size priority queue.

A heap of size K can be wrong after counts change, because an element outside it can overtake one inside. Re-run heapq.nsmallest on the count dict per query (O(U log K)) or maintain a sorted structure keyed by (count, name) updated on every increment.

FOLLOW-UP 3 The space claim.

Distinct pairs can number U^2 / 2 in the worst case, so the edge set is quadratic in the users; the counts alone are linear. If duplicates are not a concern, drop the set and keep O(U).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Active Communication Top Users
#
# Process a continuous stream of user pairs such as A-B
# and B-C; each pair creates an active communication edge,
# and the system should output users ordered by current
# communication count. Follow-ups add self-communication
# handling, duplicate communication dedupe, and top-K
# users by count. An additional API variant exposes
# register and mostActiveUser(), then asks for constant-
# time most-active lookup and a changing-K ranking
# extension.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Self communication (A-A) should now count.
# 2. Top-K with a changing K, or a fixed-size priority
# queue.
# 3. The space claim.
#
# INTERVIEW NOTES
# Normalize the pair first (sorted tuple) so A-B and B-A
# are one edge, then keep an edge set for dedup and a
# count per user. The ranking part is a separate question:
# top-K comes from a heap, and the single most active user
# can be maintained in O(1) because counts only ever go
# up. Mention that the edge set uses quadratic space in
# the worst case.
#
# BRUTE FORCE (say it first): Recompute counts from all
# edges on every query
# Keep every edge and, for each query, rebuild the
# distinct-partner counts and sort the users. Fine as an
# oracle; the work is repeated on each call. Time O(E + U
# log U) per query. Space O(E).


import heapq
from collections import Counter, defaultdict
```

```
# BRUTE FORCE (say it first): Recompute counts from all
# edges on every query | time O(E + U log U) per query
def _norm(pair):
    a, b = pair.split("-")
    return (a, b) if a <= b else (b, a)
```

```
class ActiveBrute:
    def __init__(self, count_self=False):
        self.edges, self.count_self = set(), count_self

    def add(self, pair):
        a, b = _norm(pair)
        if a == b and not self.count_self:
            return
        self.edges.add((a, b))

    def counts(self):
        c = Counter()
        for a, b in self.edges:
            c[a] += 1
            if a != b:
                c[b] += 1
        return c

    def top_k(self, k):
        c = self.counts()
        return sorted(c, key=lambda u: (-c[u], u))[:k]

    def most_active(self):
        c = self.counts()
        if not c:
            return set()
        m = max(c.values())
        return {u for u, v in c.items() if v == m}


# BASE QUESTION (optimal): Edge set, counts per user,
# count buckets for the maximum
# Idea: A new edge increments the count of both users.
# Buckets map each count to the set of users with it, so a
# user moves from bucket c to bucket c + 1.
# FOLLOW-UP 1: Self communication (A-A) should now count.
# Answer: Pass a flag to the class: a self pair is one
# edge that adds one to that user's count (and only once).
# The dedup set already handles repeated self pairs.
class Active:
    def __init__(self, count_self=False):
        # distinct edges seen so far (for the dedupe rule)
        self.edges = set()
        # user -> how many different partners
```

```
        self.count = Counter()
        # count -> users having exactly that count
        self.bucket = defaultdict(set)
        # the highest count right now (it never drops)
        self.top = 0
        self.count_self = count_self

    def bump(self, user):
        c = self.count[user]
        # move the user from bucket c to bucket c + 1
        if c:
            self.bucket[c].discard(user)
        self.count[user] = c + 1
        self.bucket[c + 1].add(user)
        self.top = max(self.top, c + 1)

    def add(self, a, b):
        # A-B and B-A are the same edge
        pair = (min(a, b), max(a, b))
        # follow-ups: skip self pairs (unless counted)
        # and skip a repeated pair
```

```
        skip_self = a == b and not self.count_self
        if skip_self or pair in self.edges:
            return
        self.edges.add(pair)
        self.bump(a)
        # a self pair counts once, not twice
        if a != b:
            self.bump(b)

    def most_active(self):
        # O(1): read the top bucket
        if not self.top:
            return set()
        return set(self.bucket[self.top])

    def top_k(self, k):
        # k users by count, ties broken by name
        def rank(u):
            return (-self.count[u], u)
        return heapq.nsmallest(k, self.count, key=rank)


if __name__ == "__main__":
    a = Active()
```

```
    for x, y in [("A", "B"), ("B", "C"), ("B", "A")]:
        a.add(x, y)
    # B talks to A and C; the repeat B-A is ignored
    assert a.most_active() == {"B"}
    assert a.count["B"] == 2 and a.count["A"] == 1
    assert a.top_k(2) == ["B", "A"]
    # self pairs are ignored by default
    a.add("C", "C")
    assert a.count["C"] == 1
    # ... but can be switched on, counting once
    s = Active(count_self=True)
    s.add("C", "C")
    s.add("C", "C")
    assert s.count["C"] == 1
    assert Active().most_active() == set()
```

NOTES FROM CANDIDATES

A Java solution used an ordered map / tree structure to keep counts sortable.
The interviewer asked many clarification questions and expected dry runs.
The first version did not require self-communication handling; it was added as a coded follow-up.
Account for the per-user contact sets when explaining space use: the distinct relationship state can take quadratic space in the number of users. The interviewer explicitly challenged an initial linear-space claim.
A fixed-size priority-queue proposal did not finish the changing-K discussion; the interviewer asked for further optimization.
Maintaining the API variant
With registrations only and any tied maximum acceptable, normalize undirected pairs and store the distinct edges. For a new edge, increment each distinct endpoint once; an accepted self-edge changes one count, while an ignored self-edge changes none. Clarify that choice and the empty-state response.
Cache a user with the largest count. Counts never decrease, so only the endpoints of a new edge can displace the cached winner; duplicate edges do not change it. Reading this cache makes mostActiveUser() O(1), with expected O(1) registration work using

hash-based edge membership and counters under unit-cost user identifiers. A stricter tie rule requires separate handling.
For changing K, one option is a balanced ordered index of all users keyed by count and an agreed tie-breaker. Remove and reinsert the affected users when counts change. Updates cost O(log U); starting at the highest-ranked user and walking entries in descending-count order costs O(log U + K) for at most U returned users. This supports different K values without rebuilding a fixed-capacity heap and is a preparation option, not a specified interviewer solution.
Counters, the maximum cache, and the ranking index use O(U) space. Distinct relationship storage adds O(E), giving O(U + E) overall and O(U^2) in a dense graph.

PREPARATION

Implement the pair-normalization layer separately from count maintenance.
Practice maintaining user -> count, edge set, and count -> ordered users views without letting updates drift.
Dry-run duplicate edges and self-edges before discussing top-K optimization.

STUDY HERE: topic 1, Heaps, Stacks and Queues

MEDIUM | PracHub | Technical Screen | 5 of 33

Sliding-Window Mean After Removing the Largest K Values

POSTED 2026-09-01 | 49 DAYS AGO | LAST 90 DAYS

STUDY: TALK ALL TIME 29.8 30 DAYS 0 PRIORITY 126/228

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #480 Sliding Window Median

LeetCode match: LC 480 Sliding Window Median (order statistics in a window)

Posted on PracHub: 2026-09-01 (updated 2026-09-23)

Original: https://prachub.com/coding-questions/sliding-window-mean-after-removing-the-largest-k-values

QUESTION

For every contiguous window of a fixed size, remove the largest **k** values by multiplicity and compute the mean of the remaining values. Process sliding updates incrementally rather than sorting each complete window again. Implement **`trimmed_window_means(nums: int[], window: int, k: int) -> string[]`**. Return means in window order as reduced fractions **`numerator/denominator`**, using a positive denominator. This exact fraction encoding is an explicit practice representation that avoids cross-language floating-point rounding differences.

Constraints & Assumptions

- **`1 <= len(nums) <= 200000`**, **`1 <= window <= len(nums)`**, and **`0 <= k < window`**.
- Values range from -1,000,000 through 1,000,000. Use a sufficiently wide integer for window sums.
- Remove exactly **k** occurrences, not **k** distinct values. Equal values on the partition boundary may be represented in either group as long as the multiset counts are correct.
- For **`k == 0`**, retain the whole window. A window that would retain zero values is excluded by the input contract.
- Reduce each fraction by the greatest common divisor of the absolute numerator and denominator. Zero is **`0/1`**; an integer mean such as 5 is **`5/1`**.
- Aim for **`O(n log window)`** processing with **`O(window)`** maintained state, apart from the returned results.

Examples

```
nums = [1,9,2,8], window = 3, k = 1 result = ["3/2","5/1"]
```

```
nums = [-1,1,3], window = 2, k = 0 result = ["0/1","2/1"]
```

QUESTION

Explain how insertions, removals, duplicate values, and rebalancing preserve the retained sum. For the reported large-data/stream follow-up, describe which state must remain in memory and why a new window does not require a full sort.

```
Maintain a size and an ordering invariant Partition the window into the removed largest values and the retained values. Keep their sizes correct and ensure no retained value is larger than a removed value.
```

APPROACH

Keep the window in two structures (top-k multiset and the rest) with running sums, using two heaps with lazy deletion or a balanced BST; mean = rest_sum / (w - k). O(n log w).

HOW TO THINK ABOUT IT

The mean needs two sums: the whole window and the sum of its K largest values. Keep the window in a sorted container; the K largest are its last K elements. When one value enters or leaves, only one element crosses the boundary between 'kept' and 'removed', so the sum of the K largest can be updated in O(1) after an O(log) search.

Basis: LeetCode 480 (Sliding Window Median) style order statistics; the mean of the window after dropping exactly K occurrences of the largest values, returned as reduced fractions.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(nums, w, k):
    means = []
    for i in range(len(nums) - w + 1):
        window = sorted(nums[i:i + w])
        kept = window[:w - k]  # drop k largest
        total = sum(kept)
        means.append(Fraction(total, len(kept)))
    return means
```

TIME	SPACE
O(n * w log w)	O(w)

WHY IT IS SLOW

Each window is sorted from scratch, O(n * w log w). A sorted list updated by one insertion and one removal, with the sum of the top k adjusted at the boundary, avoids it.

OPTIMAL SOLUTION · Sorted window with an incrementally maintained top-K sum

TIME	SPACE
O(n * (log w + w) in memmove, O(n log w) with a balanced tree)	O(w)

OPTIMAL SOLUTION · Sorted window with an incrementally maintained top-K sum

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Make the update O(log w) in the worst case.

A Python list insert shifts elements (fast in practice, O(w) in theory). A balanced tree or a Fenwick tree over compressed values gives O(log w) insert, delete and 'sum of the top K' via a descent.

FOLLOW-UP 2 Remove the smallest K instead, or both ends (trimmed mean).

Maintain the sum of the K smallest as well (the first K elements of the same sorted list); the mean of the middle values is (total - top - bottom) / (w - 2K).

FOLLOW-UP 3 Duplicates on the boundary.

Removing exactly K occurrences, not K distinct values, is what the positional approach does: elements equal to the boundary value are interchangeable, so which copy sits in the top K does not matter for the sum. **Cost:** unchanged: O(w) per update for list insertion (O(log w) with a balanced tree).

FOLLOW-UP 4 Trim the smallest K as well (a trimmed mean).

Maintain the sum of the K smallest alongside the K largest with the mirror-image boundary updates; the mean of the middle values is (total - top - bottom) / (w - 2K). Checked against sorting every window.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Sliding-Window Mean After Removing the
# Largest K Values
#
# For every contiguous window of a fixed size, remove the
# largest k values by multiplicity and compute the mean of
# the remaining values. Process sliding updates
# incrementally rather than sorting each complete window
# again. Implement trimmed_window_means(nums: int[],
# window: int, k: int) -> string[]. Return means in window
# order as reduced fractions numerator/denominator, using
# a positive denominator. This exact fraction encoding is
# an explicit practice representation that avoids cross-
# language floating-point rounding differences. ###
# Constraints & Assumptions - 1 <= len(nums) <= 200000, 1
# <= window <= len(nums), and 0 <= k < window. - Values
# range from -1,000,000 through 1,000,000. Use a
# sufficiently wide integer for window sums. - Remove
# exactly k occurrences, not k distinct values.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Make the update O(log w) in the worst case.
# 2. Remove the smallest K instead, or both ends (trimmed
# mean).
# 3. Duplicates on the boundary.
# 4. Trim the smallest K as well (a trimmed mean).
#
# INTERVIEW NOTES
# The mean needs two sums: the whole window and the sum of
# its K largest values. Keep the window in a sorted
# container; the K largest are its last K elements. When
# one value enters or leaves, only one element crosses the
# boundary between 'kept' and 'removed', so the sum of the
# K largest can be updated in O(1) after an O(log) search.
#
# BRUTE FORCE (say it first): Sort every window and sum
# the kept values
# For each window sort a copy, drop the largest k values,
# and compute the exact fraction. Correct and the oracle;
# every window is sorted from scratch. Time O(n * w log
# w). Space O(w).


from bisect import bisect_left, bisect_right
from fractions import Fraction


# HELPERS for the brute force
def as_text(fr):
    return f"{fr.numerator}/{fr.denominator}"


def brute(nums, window, k, both=False):
    # sort every window: the check used by the demo
    out = []
    for i in range(len(nums) - window + 1):
        part = sorted(nums[i:i + window])
        part = part[k:-k] if both and k else part
        part = part[:len(part) - k] if not both else part
        mean = Fraction(sum(part), len(part))
        out.append(as_text(mean))
    return out


# HELPERS for the base question
# FOLLOW-UP 4: Trim the smallest K as well (a trimmed
# mean).
# Answer: Maintain the sum of the K smallest alongside the
# K largest with the mirror-image boundary updates; the
# mean of the middle values is (total - top - bottom) / (w
# - 2K). Checked against sorting every window.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
class Window:
    """A sorted window that keeps the sum of its k largest
    and k smallest values while values come and go."""

    def __init__(self, k):
        self.k, self.s = k, []
        self.total = self.top = self.bottom = 0

    def insert(self, x):
        k, s = self.k, self.s
        old_len = len(s)
        p = bisect_right(s, x)
        s.insert(p, x)
        self.total += x
        # top-k region is the last k places (new list)
        if old_len < k:
            self.top += x
        elif p >= old_len + 1 - k:
            # x entered it: the value at old_len - k left
            # it
            self.top += x - s[old_len - k]
```

```
        # bottom-k region is the first k places
        if old_len < k:
            self.bottom += x
        elif p < k:
            self.bottom += x - s[k]

    def remove(self, x):
        k, s = self.k, self.s
        old_len = len(s)
        p = bisect_left(s, x)
        self.total -= x
        if old_len <= k:
            self.top -= x
            self.bottom -= x
        else:
            if p >= old_len - k:
                # the value just below the region joins it
                self.top += s[old_len - k - 1] - x
            if p < k:
                self.bottom += s[k] - x
        del s[p]


# BASE QUESTION (optimal): Sorted window with an
# incrementally maintained top-K sum
# Idea: Keep the window values in a sorted list, the total
# window sum, and top = the sum of its K largest elements.
def trimmed_window_means(nums, window, k, both=False):
    # mean of each window after dropping the k largest
    # (and, with both=True, the k smallest as well)
    w, out = Window(k), []
    for i, x in enumerate(nums):
        w.insert(x)
        if i >= window:
            w.remove(nums[i - window])
        if i >= window - 1:
            kept = window - (2 * k if both else k)
            drop = w.top + (w.bottom if both else 0)
            mean = Fraction(w.total - drop, kept)
            out.append(as_text(mean))
    return out
```

```
if __name__ == "__main__":
    nums = [5, 1, 4, 2, 8, 3, 3, 9]
    fast = trimmed_window_means
    assert fast(nums, 3, 1) == brute(nums, 3, 1)
    assert fast(nums, 4, 2) == brute(nums, 4, 2)
    # a reduced fraction: (1 + 4) / 2 = 5/2
    assert fast([5, 1, 4], 3, 1) == ["5/2"]
    # duplicates at the boundary: remove exactly k copies
    dup = [3, 3, 3, 3]
    assert fast(dup, 3, 1) == ["3/1", "3/1"]
    # trimmed mean: drop k smallest and k largest
    both = fast(nums, 5, 1, True)
    assert both == brute(nums, 5, 1, True)
```

FUNCTION AND TEST CASES

Signature

```
def trimmed_window_means(nums: list[int], window: int, k:
int) -> list[str]:
```

Sample call

```
trimmed_window_means([1, 9, 2, 8], 3, 1)
```

Test cases

Input	Expected
`([1, 9, 2, 8], 3, 1)`	`['3/2', '5/1']`
`([-1, 1, 3], 2, 0)`	`['0/1', '2/1']`

ALSO REPORTED · 2 more

Sliding Window Average After Dropping the Largest Values | PracHub | Technical Screen

https://prachub.com/coding-questions/sliding-window-average-after-dropping-the-largest-values

Compute sliding-window average excluding top k | PracHub | Technical Screen

https://prachub.com/coding-questions/compute-sliding-window-average-excluding-top-k

STUDY HERE: topic 1, Heaps, Stacks and Queues

Site tags: iterator, queue, data-structure, object-design, medium

MEDIUM | DESIGN-STYLE (skippable) | 1Point3Acres | onsite-coding | 6 of 33

Implement Python's itertools.tee

ASKED 2026-08-20 | 61 DAYS AGO | LAST 90 DAYS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 225/228**

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-08-20 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/python-tee-independent-iterators

QUESTION

Implement the behavior of Python's `tee`: accept one iterator and an integer **n**, then return **n** independent iterators over the same source sequence. Define the required function and classes yourself and improve a naive design toward an efficient shared-state implementation.SWEiteratorqueuedata-structureobject-designmediumFrequencySingle reportLast asked2026-08-20Stageonsite-coding

APPROACH

Implement a custom Python iterator class from memory, including __iter__ and __next__.
Trace two and three consumers advancing at different speeds and identify what state must remain available.
Compare a fully materialized baseline with a lazy shared-state design and explain the memory behavior.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the source iterator can be consumed by one consumer first, what happens if one consumer is far ahead, and whether thread safety is needed.

BRUTE FORCE TO SAY OUT LOUD

Store the whole source in a list and give each consumer an index. Simple, but defeats the purpose for large or infinite iterators.

OPTIMAL IDEA

Keep a deque per consumer. When a consumer needs an item and its deque is empty, pull the next item from the source and append it to every consumer's deque. Each item is held only until all consumers have read it. Build with a shared state object and a small iterator class with __iter__ and __next__. The linked-list variant uses one chain of nodes with each consumer holding a pointer, which frees nodes as the slowest consumer moves.

DRY-RUN THESE CASES

Two consumers at different speeds, n = 1, n = 0, an empty source, an exhausted source.

LIKELY FOLLOW-UPS

Thread safety (a lock around the pull), and memory growth if one consumer stalls.

PITFALLS

Pulling from the source once per consumer instead of once per item. Not appending to the other deques.

HOW TO THINK ABOUT IT

The source can only be consumed once, so someone must remember items until every consumer has read them. The state to keep is exactly 'items the slowest consumer has not read yet', so either give each consumer a queue that the leader fills, or keep one shared linked list with a pointer per consumer so memory is freed automatically.

Basis: The itertools.tee documentation's recipe; the linked-list variant is how CPython shares buffers.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def tee(source, n):
    items = list(source)  # read it all now
    def reader():
        for x in items:
            yield x
    return [reader() for _ in range(n)]
```

TIME	SPACE
O(n) up front	**O(n)**

WHY IT IS SLOW

It reads the source completely before anything is consumed, never works on an infinite or lazy source, and keeps every item in memory. One buffer per consumer (or a shared linked list) holds only what the slowest consumer has not read.

OPTIMAL SOLUTION · One deque per consumer, filled lazily

TIME	SPACE
O(n) total, O(1) amortized per item per consumer	**O(lag between fastest and slowest consumer)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Memory grows when one consumer stalls. Is there a way to free items automatically?

Use one shared singly linked list with a pointer per consumer. The only references to old nodes are the consumers' pointers, so Python's garbage collector frees nodes behind the slowest consumer without any bookkeeping. This is the design CPython uses.

FOLLOW-UP 2 Make it safe for several threads.

Pull from the source under one lock and append to the queues inside it. Without the lock two consumers can pull two items and append them in different orders to different queues. **Cost:** one lock acquisition per item pulled, O(1) extra per item; contention grows with the number of consumers.

FOLLOW-UP 3 Why not just copy the iterator?

Most iterators (files, generators, sockets) cannot be copied. tee gives independent views by buffering, which is the only thing you can do when the source cannot rewind.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Implement Python's itertools.tee
#
# Implement the behavior of Python's tee: accept one
# iterator and an integer n, then return n independent
# iterators over the same source sequence. Define the
# required function and classes yourself and improve a
# naive design toward an efficient shared-state
# implementation.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Memory grows when one consumer stalls. Is there a way
# to free items automatically?
# 2. Make it safe for several threads.
# 3. Why not just copy the iterator?
#
# INTERVIEW NOTES
# The source can only be consumed once, so someone must
# remember items until every consumer has read them. The
# state to keep is exactly 'items the slowest consumer has
# not read yet', so either give each consumer a queue that
# the leader fills, or keep one shared linked list with a
# pointer per consumer so memory is freed automatically.
#
# BRUTE FORCE (say it first): Materialize the whole source
# into a list
# Read everything into a list and give each consumer an
# index. Correct for finite data, but it consumes the
# source eagerly, never works on an infinite iterator and
# holds all items in memory. Time O(n) up front. Space
# O(n).


import threading
from collections import deque


# FIRST IDEA (simplest): copy into a list
def tee_naive(iterable, n=2):
    # simplest idea: copy everything into a list
    items = list(iterable)
    # but it never works for endless iterators
    return tuple(iter(items) for _ in range(n))


# BASE QUESTION (optimal): One deque per consumer, filled
# lazily
# Idea: When a consumer's deque is empty it pulls the next
# item from the source and appends it to every consumer's
# deque, then pops its own.
def tee_queues(iterable, n=2):
    source = iter(iterable)
    # one waiting line of values per consumer
    queues = [deque() for _ in range(n)]

    def consumer(mine):
        while True:
            if not mine:
                try:
                    value = next(source)
                except StopIteration:
                    return
                # give the new value to everybody
                for q in queues:
                    q.append(value)
            yield mine.popleft()

    return tuple(consumer(q) for q in queues)


# FOLLOW-UP 1: Memory grows when one consumer stalls. Is
# there a way to free items automatically?
# Answer: Use one shared singly linked list with a pointer
# per consumer. The only references to old nodes are the
# consumers' pointers, so Python's garbage collector frees
# nodes behind the slowest consumer without any
# bookkeeping.
class Cell:
    # one link of the shared list
    def __init__(self):
        self.filled = False
        self.value = None
        self.next = None


# FOLLOW-UP 2: Make it safe for several threads.
# Answer: Pull from the source under one lock and append
# to the queues inside it. Without the lock two consumers
# can pull two items and append them in different orders
# to different queues.
def tee(iterable, n=2):
    source = iter(iterable)
    # one lock keeps the pulls in a single order
    lock = threading.Lock()

    def consumer(cell):
        while True:
            with lock:
                if not cell.filled:
                    # the first reader pulls the value
                    try:
                        cell.value = next(source)
                    except StopIteration:
                        return
                    cell.filled = True
                    cell.next = Cell()
                value, cell = cell.value, cell.next
            yield value
```

```
    # every reader starts at the same first cell
    head = Cell()
    return tuple(consumer(head) for _ in range(n))


if __name__ == "__main__":
    a, b = tee(range(5))
    # readers are independent, at any speed
    assert next(a) == 0 and next(a) == 1
    assert list(b) == [0, 1, 2, 3, 4]
    assert list(a) == [2, 3, 4]
    x, y, z = tee_queues("abc", 3)
    assert list(x) == list(y) == list(z) == list("abc")
    # works on an endless source, which tee_naive cannot
    def count():
        i = 0
        while True:
            yield i
            i += 1
    e1, e2 = tee(count())
    assert [next(e1) for _ in range(3)] == [0, 1, 2]
    assert next(e2) == 0
    assert [list(i) for i in tee_naive([1, 2], 2)] == [
        [1, 2], [1, 2]]
```

NOTES FROM CANDIDATES

The round depends on Python iterator-protocol fluency as much as general data-structure reasoning. Clarify laziness, source exhaustion, invalid n, and whether concurrent consumption is in scope before implementing.
A correct baseline consumes the source lazily once and gives each returned iterator its own queue: when one queue is empty, pull one source value, append it to every queue, then return it from the requesting queue. If n iterators trail across a gap of g values, however, those queues can retain O(n·g) buffered copies. For the optimal shared-state design, keep one linked buffer (or indexed deque) of source values plus one cursor per iterator, and discard a buffered value only after every cursor has passed it. That preserves independent consumption with O(g+n) auxiliary state, where g is the distance between the fastest and slowest consumers.

PREPARATION

Implement a custom Python iterator class from memory, including __iter__ and __next__.
Trace two and three consumers advancing at different speeds and identify what state must remain available.
Compare a fully materialized baseline with a lazy shared-state design and explain the memory behavior.

STUDY HERE: topic 1, Heaps, Stacks and Queues

Site tags: interval, two-pointer, heap, streaming, medium

MEDIUM | 1Point3Acres | onsite-coding | 7 of 33

Union of Sorted Interval Lists

ASKED 2026-08-16 | 65 DAYS AGO | LAST 90 DAYS

STUDY: TALK **ALL TIME 25.5** **30 DAYS 0** **PRIORITY 139/228**

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #986 Interval List Intersections

LeetCode match: LC 986 Interval List Intersections (related)

Frequency: Single report | Last asked: 2026-08-16 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/sorted-interval-list-union

QUESTION

Reverse of LC 986: given two individually sorted, internally non-overlapping interval lists, return their union. Follow-ups extend to K lists with a heap, inputs too large for memory, and lists that may overlap internally.SWEintervaltwo-pointerheapstreamingmediumFrequencySingle reportLast asked2026-08-16Stageonsite-coding

APPROACH

Code the two-list merge without special-casing more than an empty result and a non-overlapping next interval.
Generalize the implementation to K iterators using a heap, then explain the memory bound.
Rehearse how the invariant changes when individual inputs are not internally disjoint.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether intervals are closed or half-open, whether touching intervals merge, whether each list is sorted and non-overlapping internally (later follow-up removes this), and how large the inputs are.

BRUTE FORCE TO SAY OUT LOUD

Concatenate both lists, sort, and merge. O((m + n) log (m + n)).

OPTIMAL IDEA

Two-pointer merge: repeatedly take the interval with the smaller start from either list and fold it into the last output interval if it overlaps (start at most the previous end), else append it. O(m + n). For K lists use heapq.merge (a heap of K heads), and since it is a generator the inputs never need to fit in memory.

DRY-RUN THESE CASES

One list empty, no overlaps, one interval swallowing many, touching endpoints, identical lists.

INTERVIEW-DAY PLAN

LIKELY FOLLOW-UPS

K lists, lists too big for memory, lists that overlap internally (sort each first or heap everything).

PITFALLS

Using end instead of start to pick the next interval. Not taking the maximum of ends when merging.

HOW TO THINK ABOUT IT

Each list is already sorted, so you can walk both with two pointers like the merge step of merge sort: always take the interval with the smaller start next. Once the intervals come out in start order, merging is the usual one-pass fold: extend the last output interval if the next one overlaps or touches it, otherwise start a new one.

Basis: The reverse of LeetCode 986 (Interval List Intersections): union instead of intersection.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(list1, list2):
    merged = []
    for s, e in sorted(list1 + list2):
        if merged and s <= merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], e)
        else:
            merged.append([s, e])
    return merged
```

TIME	SPACE
O((m + n) log (m + n))	O(m + n)

WHY IT IS SLOW

Sorting ignores that both lists are already sorted: O((m + n) log (m + n)) instead of O(m + n) for a two-pointer merge.

OPTIMAL SOLUTION · Two-pointer merge

TIME	SPACE
O(m + n)	O(m + n) for the output

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What if the lists are too big for memory?

Read each list through an iterator (a file or a paged API). union_k only ever holds one interval per list plus the current merged interval, so memory is O(K) and it can emit results as it goes. Sorted order is what makes streaming possible.

FOLLOW-UP 2 What if one list may overlap itself?

The two-pointer merge already works, because it folds every incoming interval into the last output interval. What breaks is only the assumption that each list is sorted by start; if that also fails, sort each list first (O(n log n)) or push everything through a heap.

FOLLOW-UP 3 Do touching intervals merge?

Decide the endpoint rule before coding. For closed intervals [1, 3] and [3, 5] share a point, so they merge (start <= last end). For half-open intervals [1, 3) and [3, 5) they do not overlap but are adjacent; whether to merge adjacent ones is a product choice, so ask.

FOLLOW-UP 4 Each list may overlap itself or be unsorted.

Normalise each list first (sort and merge it by itself, O(n log n)), after which the two-pointer union applies. Or sort everything together if memory allows.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Union of Sorted Interval Lists
#
# Reverse of LC 986: given two individually sorted,
# internally non-overlapping interval lists, return their
# union. Follow-ups extend to K lists with a heap, inputs
# too large for memory, and lists that may overlap
# internally.
#
# FOLLOW-UPS (answered below, in this order):
# 1. What if the lists are too big for memory?
# 2. What if one list may overlap itself?
# 3. Do touching intervals merge?
# 4. Each list may overlap itself or be unsorted.
#
# INTERVIEW NOTES
# Each list is already sorted, so you can walk both with
# two pointers like the merge step of merge sort: always
# take the interval with the smaller start next. Once the
# intervals come out in start order, merging is the usual
# one-pass fold: extend the last output interval if the
# next one overlaps or touches it, otherwise start a new
# one.
#
# BRUTE FORCE (say it first): Concatenate, sort and merge
# Put all intervals in one list, sort by start, and fold
# overlapping ones. Correct and short; the sort ignores
# that both inputs were already sorted. Time O((m + n) log
# (m + n)). Space O(m + n).


import heapq


# BRUTE FORCE (say it first): Concatenate, sort and merge
# | time O((m + n) log (m + n))
def union_sort(a, b):
    out = []
    for s, e in sorted(a + b):
        if out and s <= out[-1][1]:
            out[-1][1] = max(out[-1][1], e)
        else:
            out.append([s, e])
    return out
```

```
# HELPERS for the base question
# FOLLOW-UP 1: What if the lists are too big for memory?
# Answer: Read each list through an iterator (a file or a
# paged API). union_k only ever holds one interval per
# list plus the current merged interval, so memory is O(K)
# and it can emit results as it goes.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def union_stream(lists, touching=True):
    # lists: sorted iterables of (start, end)
    merged = heapq.merge(*lists)
    cur = None
    for start, end in merged:
        if cur is None:
            cur = [start, end]
            continue
        # touching: [1, 3] and [3, 5] share the point 3
        joined = start < cur[1]
        if touching:
            joined = start <= cur[1]
        if joined:
            cur[1] = max(cur[1], end)
        else:
            # finished one block: emit it as we go
            yield tuple(cur)
            cur = [start, end]
    if cur is not None:
        yield tuple(cur)


# BASE QUESTION (optimal): Two-pointer merge
# Idea: Always take the interval with the smaller start
# next. If it touches or overlaps the last output interval
# (start <= last end, closed intervals) extend that one,
# otherwise append it.
def union_two(a, b, touching=True):
    return list(union_stream([a, b], touching))


# VARIANT: K lists with a heap
def union_k(lists, touching=True):
    # same code, any number of lists, O(K) memory
    return list(union_stream(lists, touching))
```

```
# FOLLOW-UP 2: What if one list may overlap itself?
# Answer: The two-pointer merge already works, because it
# folds every incoming interval into the last output
# interval.
def normalise(intervals):
    # a list that is unsorted or overlaps itself
    ordered = sorted(intervals)
    return list(union_stream([ordered]))


# FOLLOW-UP 4: Each list may overlap itself or be
# unsorted.
# Answer: Normalise each list first (sort and merge it by
# itself, O(n log n)), after which the two-pointer union
# applies. Or sort everything together if memory allows.
def union_unsorted(a, b):
    # fix each list first, then use the same union
    return union_two(normalise(a), normalise(b))


if __name__ == "__main__":
```

```
a = [(1, 3), (6, 8)]
b = [(2, 4), (9, 10)]
assert union_two(a, b) == [(1, 4), (6, 8), (9, 10)]
# closed intervals that touch merge
assert union_two([(1, 3)], [(3, 5)]) == [(1, 5)]
# half-open intervals that only touch stay apart
assert union_two([(1, 3)], [(3, 5)], False) == [
    (1, 3), (3, 5)]
# three lists, one of them empty
three = union_k([a, b, []])
assert three == [(1, 4), (6, 8), (9, 10)]
# a list that overlaps itself, and an unsorted one
merged = union_unsorted([(5, 9), (1, 6)], [(8, 12)])
assert merged == [(1, 12)]
assert union_unsorted([(9, 10), (1, 2)], []) == [
    (1, 2), (9, 10)]
```

NOTES FROM CANDIDATES

The round intentionally layers one constraint at a time. Keep the base invariant explicit—every interval before the current pointer has already been absorbed into a canonical output prefix—so the streaming and K-way versions remain easy to justify.
Agree on endpoint semantics before coding: touching closed intervals merge, while touching half-open intervals do not overlap. For K streaming inputs, the heap plus one current interval per input uses O(K) working memory, excluding the output.

PREPARATION

Code the two-list merge without special-casing more than an empty result and a non-overlapping next interval.
Generalize the implementation to K iterators using a heap, then explain the memory bound.
Rehearse how the invariant changes when individual inputs are not internally disjoint.

STUDY HERE: topic 1, Heaps, Stacks and Queues

MEDIUM | PracHub | Technical Screen | 8 of 33

★ TOP 75: rank 71 in the Medium and Hard priority list

★ Expand Recursive Key-Value Placeholders in a Template

POSTED 2026-08-04 | 77 DAYS AGO | LAST 90 DAYS

STUDY: TYPE | ALL TIME 57.7 | 30 DAYS 0 | PRIORITY 46/228

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #394 Decode String

LeetCode match: No direct match; related LC 394 Decode String (inferred from overview)

Posted on PracHub: 2026-08-04 (updated 2026-08-16)

Original: https://prachub.com/coding-questions/expand-recursive-key-value-placeholders-in-a-template

QUESTION

Overview: Expand %KEY% placeholders in a template, recursively resolving placeholders inside mapped values. Use memoized dependency traversal to preserve literal text, avoid repeated work, and explain cycle, unknown-key, and escaping policies.

APPROACH

DFS expansion with memoization per key and a 'visiting' set to detect cycles; leave unknown keys per the spec; keep literal text unchanged.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask for the placeholder syntax (%KEY%), what to do with an unknown key (leave it as is or raise), how to write a literal percent sign (escaping such as %%), and how to handle cycles (raise an error).

BRUTE FORCE TO SAY OUT LOUD

Repeatedly run a regex replace on the whole string until nothing changes. It loops forever on a cycle.

OPTIMAL IDEA

Resolve each key with memoized DFS: resolve(key) expands the value of that key by scanning for placeholders and recursively resolving them, with a visiting set to detect cycles and a cache so each key is expanded once. Expand the template by scanning once and replacing each placeholder with resolve(name), leaving literal text untouched. Linear in the sizes of text and values.

DRY-RUN THESE CASES

A value that refers to another key, a cycle (A refers to B, B to A), an unknown key, a template with no placeholders, two placeholders adjacent, an escaped percent.

LIKELY FOLLOW-UPS

Escaping, recursion depth limit (iterative expansion), or limiting the output size to prevent blow-up.

PITFALLS

Re-expanding a key every time. Treating an unknown key and a cycle the same way.

INTERVIEW-DAY PLAN

HOW TO THINK ABOUT IT

Expanding one key can require expanding others, so define expand(key) recursively and remember results (memoization) so a key used many times is expanded once. A 'currently expanding' set detects cycles: meeting a key that is already on the stack means a cycle. Decide the policy for unknown keys and for literal percent signs up front.

Basis: Written. Assumed rules (confirm them): placeholders look like %KEY%; the value of a key may itself contain placeholders; an unknown key is left as is; a cycle raises an error.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def expand(text):
    for _ in range(LIMIT):
        new = text
        for key, value in values.items():
            tag = "%" + key + "%"
            new = new.replace(tag, value)
        if new == text:
            return text
        text = new
    raise Error("cycle")
```

TIME	SPACE
O(depth * length)	O(length)

WHY IT IS SLOW

It re-expands the same key every time it appears and needs an arbitrary cap to stop on cycles. A memoised recursive expansion with a visiting set expands each key once and detects cycles exactly.

OPTIMAL SOLUTION · Memoised recursive expansion with a visiting set

TIME	SPACE
O(total size of all values + template)	O(number of keys)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 A literal percent sign must be possible.

Choose an escape such as %% and handle it in the same scan: when the text has %%, output one % and skip both characters; otherwise try to match a placeholder. Test percent signs next to placeholders.

FOLLOW-UP 2 An unknown key should be an error, or replaced by an empty string.

Make the policy a parameter: raise KeyError, return the placeholder unchanged (as coded), or return ''. State your default.

FOLLOW-UP 3 Values are huge and expansion explodes.

The output can be exponentially larger than the input (a key referring to the previous key twice, repeatedly). Check the expanded length as you go and fail past a limit, or return a lazy expansion.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Expand Recursive Key-Value Placeholders
# in a Template
#
# Expand %KEY% placeholders in a template, recursively
# resolving placeholders inside mapped values. Use
# memoized dependency traversal to preserve literal text,
# avoid repeated work, and explain cycle, unknown-key, and
# escaping policies.
#
# FOLLOW-UPS (answered below, in this order):
# 1. A literal percent sign must be possible.
# 2. An unknown key should be an error, or replaced by an
# empty string.
# 3. Values are huge and expansion explodes.
#
# INTERVIEW NOTES
# Expanding one key can require expanding others, so
# define expand(key) recursively and remember results
# (memoization) so a key used many times is expanded once.
# A 'currently expanding' set detects cycles: meeting a
# key that is already on the stack means a cycle. Decide
# the policy for unknown keys and for literal percent
# signs up front.
#
# BRUTE FORCE (say it first): Keep substituting until the
# text stops changing
# Replace every %KEY% found in the text by its value and
# repeat until nothing changes. Simple, but it never stops
# on a cycle (so it needs an iteration cap) and re-expands
# the same key every time it appears. Time O(depth *
# length). Space O(length).


import re


# BRUTE FORCE (say it first): Keep substituting until the
# text stops changing | time O(depth * length)
_PH = re.compile(r"%([A-Za-z_][A-Za-z0-9_]*)%")
```

```
# HELPERS for the base question
PATTERN = re.compile(r"%%|%(\w+)%")
```

```
def expand_by_rewriting(
    template, mapping, max_rounds=50
):
    text = template
    for _ in range(max_rounds):
        new = _PH.sub(
            lambda m: mapping.get(
                m.group(1), m.group(0)
            ),
            text,
        )
        if new == text:
            return text
        text = new
    raise ValueError("cycle")


# BASE QUESTION (optimal): Memoised recursive expansion
# with a visiting set
# Idea: expand(key) substitutes every placeholder in the
# key's value by expand(inner), caches the result, and
# uses a visiting set to raise on a cycle. Unknown keys
# stay as written.
# FOLLOW-UP 1: A literal percent sign must be possible.
# Answer: Choose an escape such as %% and handle it in the
# same scan: when the text has %%, output one % and skip
# both characters; otherwise try to match a placeholder.
# Test percent signs next to placeholders.
def expand(template, mapping, unknown="keep",
           limit=10 ** 6):
    # unknown: "keep" the text, "empty" it, or "error"
    cache, visiting = {}, set()

    def expand_text(text):
        def one(m):
            key = m.group(1)
            # FOLLOW-UP 1: %% is a literal percent sign
            if key is None:
                # %% is a literal percent sign
                return "%"
            if key not in mapping:
                # FOLLOW-UP 2: unknown key: keep, empty or
                # error
                if unknown == "error":
                    raise KeyError(key)
```

```
                if unknown == "empty":
                    return ""
                return m.group(0)
            return expand_key(key)
        out = PATTERN.sub(one, text)
        # FOLLOW-UP 3: huge values: cap the expansion
        # stop explosive expansions early
        if len(out) > limit:
            raise ValueError("expansion too large")
        return out

    def expand_key(key):
        # each key is expanded once and then cached
        if key in cache:
            return cache[key]
        if key in visiting:
            raise ValueError(f"cycle at {key}")
        visiting.add(key)
        cache[key] = expand_text(mapping[key])
        visiting.discard(key)
        return cache[key]
```

```
    return expand_text(template)


if __name__ == "__main__":
    m = {"USER": "admin", "HOME": "/%USER%/home"}
    text = "I am %USER% My home is %HOME%"
    want = "I am admin My home is /admin/home"
    assert expand(text, m) == want
    # unknown keys: kept, emptied, or an error
    assert expand("a %X% b", m) == "a %X% b"
    assert expand("a %X% b", m, unknown="empty") == "a  b"
    try:
        expand("%X%", m, unknown="error")
        raise AssertionError("missing key not reported")
    except KeyError:
        pass
    # a literal percent, even next to a placeholder
    assert expand("100%% of %USER%", m) == "100% of admin"
    # a cycle is reported
    try:
        expand("%A%", {"A": "%B%", "B": "%A%"})
        raise AssertionError("cycle not caught")
    except ValueError:
        pass
    # a key that doubles the previous one explodes
    boom = {"K0": "ab"}
    for i in range(1, 30):
        boom[f"K{i}"] = f"%K{i - 1}%%K{i - 1}%"
    try:
        expand("%K29%", boom, limit=1000)
        raise AssertionError("explosion not stopped")
    except ValueError:
        pass
```

FUNCTION AND TEST CASES

Signature

```
def expand_template(values: dict[str, str], template: str)
 -> str:
```

Sample call

```
expand_template({}, 'plain text')
```

Test cases

Input	Expected
`({}, 'plain text')`	`'plain text'`
`({'USER': 'admin', 'HOME': '/%USER%/home'}, 'I am %USER% My home is %HOME%')`	`'I am admin My home is /admin/home'`

ALSO REPORTED · 1 more

Template Variable Expansion with Follow-Ups | 1Point3Acres | onsite-coding | asked 2026-08-14 | Single report

https://www.1point3acres.com/interview/problems/company/google/template-variable-expansion-dependencies

ALSO REPORTED · 1 more

The interviewer spent most of the round probing the delimiter and escaping contract before allowing implementation. State a compact grammar and get explicit agreement; otherwise a nominally easy parser can lose the coding budget to unresolved edge cases.

Preparation: Implement one-pass placeholder scanning under a written delimiter/escape contract.
Resolve a dependency chain in the map before processing the template, then add cycle detection.
Prepare three malformed-input cases and explain whether each is rejected, preserved literally, or interpreted as an escape.

STUDY HERE: topic 1, Heaps, Stacks and Queues

Site tags: heap, streaming, medium

MEDIUM | IN APP LC 295 | DESIGN-STYLE (skippable) | 1Point3Acres | tech-screen | 9 of 33

Streaming Median (LC 295) with Follow-ups

ASKED 2026-08-03 | 78 DAYS AGO | LAST 90 DAYS

STUDY: TYPE | **ALL TIME 50.4** | **30 DAYS 0** | **PRIORITY 69/228**

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues

Optimal answer from: your app's interview approach (LeetCode #295)

LeetCode match: LC 295 Find Median from Data Stream

Frequency: Single report | Last asked: 2026-08-03 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/streaming-median-with-follow-ups

QUESTION

A near-canonical LC 295 streaming-median implementation using two heaps, followed by two design extensions: exploit a very small value domain, then support an arbitrary k-th-smallest query instead of only the median.SWEheapstreamingmediumFrequencySingle reportLast asked2026-08-03Stagetech-screen

APPROACH

Implement the two-heap median structure and dry-run odd and even stream lengths.
Sketch a bounded-domain alternative and state its update and query costs.
Compare what must change when the requested order statistic is an arbitrary k rather than a fixed median.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether values are integers, the range of values, how often median queries happen compared with inserts, and what to return for an even count (average of two middles).

BRUTE FORCE TO SAY OUT LOUD

Keep a sorted list and insert with bisect; median is the middle. O(n) per insert due to shifting.

OPTIMAL IDEA

Two heaps: a max-heap for the lower half and a min-heap for the upper half, sizes within one. Insert by pushing through one heap into the other and rebalancing. Median is the top of the larger heap or the average of both tops. O(log n) insert, O(1) median.

DRY-RUN THESE CASES

One element, two elements, duplicates, all equal, alternating very large and very small values.

INTERVIEW-DAY PLAN

LIKELY FOLLOW-UPS

Values in a small range (counting array), arbitrary k-th smallest (Fenwick tree or order-statistic tree), or too much data (quantile sketch).

PITFALLS

Rebalancing after the wrong heap push. Integer division when the median should be fractional.

HOW TO THINK ABOUT IT

Keep the data split at the median: a max-heap for the smaller half and a min-heap for the larger half, with sizes equal or differing by one. Then the median is read from the tops. Insert by pushing through one heap into the other to keep every smaller value on the left, and rebalance the sizes.

Basis: LeetCode 295, Find Median from Data Stream.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
nums = []  # always kept sorted
def add(x):
    insort(nums, x)
def median():
    n = len(nums)
    if n % 2:
        return nums[n // 2]
    return (nums[n // 2 - 1] + nums[n // 2]) / 2
```

TIME	SPACE
O(n) per insert (list shift), O(1) median	**O(n)**

WHY IT IS SLOW

Each insert shifts up to n elements, O(n) per add (fast in practice, but linear). Two heaps keep the median available with O(log n) per add and O(1) per query.

OPTIMAL SOLUTION · Two heaps

TIME	SPACE
O(log n) add, O(1) median	**O(n)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Almost every value is in a small range (say 0 to 100). Can you do better than two heaps?

Yes: a counting array gives O(1) inserts, and the median needs one pass over at most V counts, so O(V) per query with V fixed. Keep a moving pointer to the median bucket to make queries amortized O(1).

FOLLOW-UP 2 Now support an arbitrary k-th smallest, not only the median.

Two heaps only work for one fixed rank. Use an order-statistics structure: a Fenwick tree over (compressed) values, or a balanced BST augmented with subtree sizes. Both give O(log n) add and O(log n) k-th, and the Fenwick tree is far shorter to write in an interview.

FOLLOW-UP 3 What if the data stream is too large to store?

Exact medians need memory proportional to the data, so use a quantile sketch (t-digest, GK, KLL). They answer any quantile within a chosen error bound in O(1/epsilon) memory.

FOLLOW-UP 4 The median of only the last K values.

Keep the window in a deque (to know what expires) and a sorted list (to read the middle); insert and remove with bisect. O(K) per update because of list shifting; two heaps with lazy deletion give O(log K).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```python
# MAIN QUESTION: Streaming Median (LC 295) with Follow-ups
#
# A near-canonical LC 295 streaming-median implementation
# using two heaps, followed by two design extensions:
# exploit a very small value domain, then support an
# arbitrary k-th-smallest query instead of only the
# median.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Almost every value is in a small range (say 0 to
# 100). Can you do better than two heaps?
# 2. Now support an arbitrary k-th smallest, not only the
# median.
# 3. What if the data stream is too large to store?
# 4. The median of only the last K values.
#
# INTERVIEW NOTES
# Keep the data split at the median: a max-heap for the
# smaller half and a min-heap for the larger half, with
# sizes equal or differing by one. Then the median is read
# from the tops. Insert by pushing through one heap into
# the other to keep every smaller value on the left, and
# rebalance the sizes.
#
# BRUTE FORCE (say it first): Keep a sorted list and
# insert with bisect
# Insert each value into a sorted list with bisect.insort
# and read the middle. Very simple and the oracle; every
# insert shifts up to n elements, though in practice that
# is a fast memmove. Time O(n) per insert (list shift),
# O(1) median. Space O(n).


import bisect
import heapq
from bisect import bisect_left, insort
```

```python
# BRUTE FORCE (say it first): Keep a sorted list and
# insert with bisect | time O(n) per insert (list shift),
# O(1) median
class MedianSorted:
    def __init__(self):
        self.a = []
```

```python
    def addNum(self, x):
        bisect.insort(self.a, x)

    def findMedian(self):
        n = len(self.a)
        return (
            float(self.a[n // 2])
            if n % 2
            else (self.a[n // 2 - 1] + self.a[n // 2])
            / 2
        )


# BASE QUESTION (optimal): Two heaps
# Idea: Lower half in a max-heap, upper half in a min-
# heap, sizes differing by at most one and every lower
# value <= every upper value. The median is the larger
# heap's root, or the average of both roots.
class MedianFinder:
    def __init__(self):
        # low: max-heap (stored negated); high: min-heap
        self.low, self.high = [], []

    def add(self, x):
        # push through low so the halves stay ordered
        heapq.heappush(self.low, -x)
        top = -heapq.heappop(self.low)
        heapq.heappush(self.high, top)
        # keep low the same size as high, or one bigger
        if len(self.high) > len(self.low):
            top = heapq.heappop(self.high)
            heapq.heappush(self.low, -top)

    def median(self):
        if len(self.low) > len(self.high):
            return float(-self.low[0])
        return (-self.low[0] + self.high[0]) / 2


# FOLLOW-UP 1: Almost every value is in a small range (say
# 0 to 100). Can you do better than two heaps?
# Answer: Yes: a counting array gives O(1) inserts, and
# the median needs one pass over at most V counts, so O(V)
# per query with V fixed. Keep a moving pointer to the
# median bucket to make queries amortized O(1).
```

```python
class CountingMedian:
    """Follow-up: values are whole numbers in 0..maxv."""

    def __init__(self, maxv):
        self.counts = [0] * (maxv + 1)
        self.n = 0

    def add(self, x):
        self.counts[x] += 1
        self.n += 1

    def kth(self, k):
        # k-th smallest (1-based): one pass over the
        # counts
        seen = 0
        for value, c in enumerate(self.counts):
            seen += c
            if seen >= k:
                return value
```

```
    def median(self):
        lo = self.kth((self.n + 1) // 2)
        hi = self.kth(self.n // 2 + 1)
        return (lo + hi) / 2


# FOLLOW-UP 2: Now support an arbitrary k-th smallest, not
# only the median.
# Answer: Two heaps only work for one fixed rank. Use an
# order-statistics structure: a Fenwick tree over
# (compressed) values, or a balanced BST augmented with
# subtree sizes.
class Fenwick:
    """Follow-up: any k-th smallest in O(log V)."""

    def __init__(self, size):
        self.size, self.tree = size, [0] * (size + 1)

    def add(self, value):
        i = value + 1
        while i <= self.size:
            self.tree[i] += 1
            i += i & -i

    def kth(self, k):
```

```
        # walk down the tree to the k-th smallest value
        pos, step = 0, 1 << self.size.bit_length()
        while step:
            nxt = pos + step
            if nxt <= self.size and self.tree[nxt] < k:
                pos = nxt
                k -= self.tree[nxt]
            step >>= 1
        return pos


# FOLLOW-UP 4: The median of only the last K values.
# Answer: Keep the window in a deque (to know what
# expires) and a sorted list (to read the middle); insert
# and remove with bisect. O(K) per update because of list
# shifting; two heaps with lazy deletion give O(log K).
class WindowMedian:
    """Follow-up: median of only the last k values."""

    def __init__(self, k):
        self.k, self.order, self.sorted = k, [], []

    def add(self, x):
        self.order.append(x)
        insort(self.sorted, x)
        if len(self.order) > self.k:
            old = self.order.pop(0)
            del self.sorted[bisect_left(self.sorted, old)]

    def median(self):
        s, n = self.sorted, len(self.sorted)
        return (s[(n - 1) // 2] + s[n // 2]) / 2


if __name__ == "__main__":
    m = MedianFinder()
    for x in [5, 2, 8]:
        m.add(x)
    assert m.median() == 5
    m.add(1)
    assert m.median() == 3.5
    c = CountingMedian(100)
    for x in [5, 2, 8, 1]:
        c.add(x)
    assert c.median() == 3.5
```

```
    f = Fenwick(100)
    for x in [5, 2, 8, 1]:
        f.add(x)
    ranks = [f.kth(k) for k in (1, 2, 3, 4)]
    assert ranks == [1, 2, 5, 8]
    w = WindowMedian(3)
    for x in [9, 1, 5, 7]:
        w.add(x)
    # window is [1, 5, 7]
    assert w.median() == 5
```

NOTES FROM CANDIDATES

Keep the lower half in a max-heap and the upper half in a min-heap. Maintain a size difference of at most one and keep every lower-half value no greater than every upper-half value.
For an even-sized stream, average the two heap roots.
Each insertion costs O(log n), a median query costs O(1), and the heaps use O(n) space.
For a small bounded domain, a frequency array gives O(1) updates and an O(U) order-statistic scan, where U is the domain size. For arbitrary k-th-smallest queries, use an order-statistic tree or a Fenwick/segment tree over a known or compressed domain for logarithmic update and selection.
The base problem required complete working code plus a dry run and complexity analysis.
The first follow-up stopped at design and pseudocode; the second was discussion-only because the round ran out of time.

PREPARATION

Implement the two-heap median structure and dry-run odd and even stream lengths.
Sketch a bounded-domain alternative and state its update and query costs.
Compare what must change when the requested order statistic is an arbitrary k rather than a fixed median.

STUDY HERE: topic 1, Heaps, Stacks and Queues

Site tags: string, stack, parsing, medium

MEDIUM | IN APP LC 394 | 1Point3Acres | onsite-coding | 10 of 33

★ TOP 75: rank 60 in the Medium and Hard priority list

★ Decode String

ASKED 2026-07-29 | 83 DAYS AGO | LAST 90 DAYS

STUDY: TYPE | ALL TIME 57.7 | 30 DAYS 0 | PRIORITY 45/228

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues

Optimal answer from: your app's interview approach (LeetCode #394)

LeetCode match: LC 394 Decode String

Frequency: Low | Last asked: 2026-07-29 | Roles: SWE, MLE

Original: https://www.1point3acres.com/interview/problems/company/google/decode-string

QUESTION

EXAMPLES

```
"3[a]2[bc]" -> "aaabcbc".
"2[abc]3[cd]ef" -> "abcabccdcdcdef".
"3[a2[c]]" -> "accaccacc".
```

APPROACH

Write the iterative-stack version from memory in under 10 min. Practice the postfix-syntax variant (parse direction reversed). Drill the decode side together with the encode-shortest dual problem as a pair.

HOW TO THINK ABOUT IT

Nested repeats like 3[a2[c]] are a bracket structure, which means a stack: when you see '[' save what you have built and the repeat count; when you see ']' repeat the inner string and glue it to the saved prefix. Read counts as full numbers (more than one digit).

Basis: LeetCode 394, Decode String.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def parse():
    result = ""
    while pos < len(s) and s[pos] != "]":
        if s[pos].isdigit():
            n = read_number()
            pos += 1  # skip "["
            inner = parse()
            pos += 1  # skip "]"
            result += inner * n
        else:
            result += s[pos]
            pos += 1
    return result
```

BRUTE FORCE

TIME	SPACE
O(output length)	O(depth)

WHY IT IS SLOW

Time is proportional to the output, which is optimal; the cost is the recursion depth, equal to the nesting depth. A stack of (text so far, repeat count) avoids recursion.

OPTIMAL SOLUTION · Stack of (prefix, repeat) frames

TIME	SPACE
O(output length)	O(output + nesting depth)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How would you do it recursively, and what is the risk?

Parse with an index: read letters and digits, on '[' recurse to parse the inner string and return where it ended, on ']' return. It reads naturally but the call depth equals the nesting depth, which can overflow the stack for adversarial inputs. The stack version trades that for explicit memory.

FOLLOW-UP 2 What if the decoded string is enormous?

Do not materialize it. Return a generator, or compute the character at a given index by walking the structure with lengths (as in LeetCode 880, Decoded String at Index), using index modulo the segment length at each level.

FOLLOW-UP 3 The decoded string would be enormous.

Compute its length (or a given character) from the stack without building it: keep (length so far, repeat count) frames and combine as prev + inner * times. The length can be computed for strings that could never fit in memory.

FOLLOW-UP 4 Return the character at position k only.

Walk the structure with lengths: at each repeat reduce k modulo the inner length to find where in the inner string it falls (LeetCode 880 style).

FOLLOW-UP 5 Invalid input (missing bracket, count with no bracket).

Detect an unmatched ']' (empty stack), a '[' left at the end, and digits not followed by '['. Decide whether to raise or to treat a malformed part as literal text.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Decode String
#
# Examples: "3[a]2[bc]" -> "aaabcbc". "2[abc]3[cd]ef" ->
# "abcabccdcdcdef". "3[a2[c]]" -> "accaccacc".
#
# FOLLOW-UPS (answered below, in this order):
# 1. How would you do it recursively, and what is the
# risk?
# 2. What if the decoded string is enormous?
# 3. The decoded string would be enormous.
# 4. Return the character at position k only.
# 5. Invalid input (missing bracket, count with no
# bracket).
#
# INTERVIEW NOTES
# Nested repeats like 3[a2[c]] are a bracket structure,
# which means a stack: when you see '[' save what you have
# built and the repeat count; when you see ']' repeat the
# inner string and glue it to the saved prefix. Read
# counts as full numbers (more than one digit).
#
# BRUTE FORCE (say it first): Recursive descent over the
# string
# A function that reads letters until it meets a digit
# (then reads the number, the bracketed substring
# recursively, and repeats it) or a closing bracket. It
# mirrors the grammar and is the oracle; the recursion
# depth equals the nesting depth. Time O(output length).
# Space O(depth).


# BRUTE FORCE (say it first): Recursive descent over the
# string | time O(output length)
def decode_recursive(s):
    pos = 0

    def parse():
        nonlocal pos
        out = []
        while pos < len(s) and s[pos] != "]":
            if s[pos].isdigit():
                num = 0
                while s[pos].isdigit():
                    num = num * 10 + int(s[pos])
                    pos += 1
```

```
                pos += 1  # skip '['
                inner = parse()
                pos += 1  # skip ']'
                out.append(inner * num)
            else:
                out.append(s[pos])
                pos += 1
        return "".join(out)

    return parse()


# BASE QUESTION (optimal): Stack of (prefix, repeat)
# frames
# Idea: On '[' push the text built so far and the repeat
# count, then start fresh. On ']' pop and append the
# current text repeated count times to the saved prefix.
def decode(s):
    # stack of (text so far, repeat count) for each open [
    stack, cur, num = [], [], 0
    for ch in s:
        if ch.isdigit():
            num = num * 10 + int(ch)
        elif ch == "[":
            # remember what we have, start the inner text
            stack.append((cur, num))
            cur, num = [], 0
        elif ch == "]":
            prev, times = stack.pop()
            prev.append("".join(cur) * times)
            cur = prev
        else:
            cur.append(ch)
    return "".join(cur)


# NESTED FORM (the recursive version)
def parse(s):
    # nested form: items are text or (times, items)
    stack, cur, num = [], [], 0
    for ch in s:
        if ch.isdigit():
            num = num * 10 + int(ch)
        elif ch == "[":
            if num == 0:
```

```
                raise ValueError("[ needs a count")
            stack.append((cur, num))
            cur, num = [], 0
        elif ch == "]":
            if not stack:
                raise ValueError("unmatched ]")
            prev, times = stack.pop()
            prev.append((times, cur))
            cur = prev
        else:
            if num:
                raise ValueError("count without [")
            cur.append(ch)
    if stack:
        raise ValueError("missing ]")
    return cur


# FOLLOW-UP 3: The decoded string would be enormous.
# Answer: Compute its length (or a given character) from
# the stack without building it: keep (length so far,
# repeat count) frames and combine as prev + inner *
# times. The length can be computed for strings that could
# never fit in memory.
def length(items):
    # decoded length without building the string
    total = 0
    for it in items:
        if isinstance(it, str):
            total += 1
        else:
            total += it[0] * length(it[1])
    return total


# FOLLOW-UP 4: Return the character at position k only.
# Answer: Walk the structure with lengths: at each repeat
# reduce k modulo the inner length to find where in the
# inner string it falls (LeetCode 880 style).
def char_at(items, k):
    # the k-th decoded character (0-based), LeetCode 880
    # style
    for it in items:
        if isinstance(it, str):
            if k == 0:
```

```
                return it
            k -= 1
        else:
            times, inner = it
            inner_len = length(inner)
            if k < times * inner_len:
                # which copy does not matter: use k modulo
                return char_at(inner, k % inner_len)
            k -= times * inner_len
    return None


if __name__ == "__main__":
    assert decode("3[a]2[bc]") == "aaabcbc"
    assert decode("2[abc]3[cd]ef") == "abcabccdcdcdef"
    assert decode("3[a2[c]]") == "accaccacc"
    items = parse("3[a2[c]]")
    assert length(items) == 9
    assert [char_at(items, i) for i in range(9)] == list(
        decode("3[a2[c]]"))
    # a huge result is measured, never built
    big = parse("999[999[999[ab]]]")
    assert length(big) == 2 * 999 ** 3
    for bad in ("3[a", "a]", "[a]", "3a"):
        try:
            parse(bad)
            raise AssertionError(bad)
        except ValueError:
            pass
```

NOTES FROM CANDIDATES

Google variants include alternative syntax (group){k} (postfix repeat), nested grouping with curly braces, and escape characters.
A scale follow-up asks how the interface and memory strategy should change when the encoded string is extremely long.
Recursive solution is cleaner but the iterative stack is more robust to bizarre input.

PREPARATION

Write the iterative-stack version from memory in under 10 min.
Practice the postfix-syntax variant (parse direction reversed).
Drill the decode side together with the encode-shortest dual problem as a pair.

ALSO REPORTED · 1 more

String Decompression with Nested Counts | 1Point3Acres

https://www.1point3acres.com/interview/problems/company/google/string-decompression-nested-counts

Recursion with an index pointer is the cleanest implementation: each call returns the decoded substring of a single (...){k} block plus literals up to its terminator.
An iterative stack-based approach also works: push (StringBuilder, repeatCount) on (, pop and append on){k}.
The 2-digit k matters only for parsing — read until non-digit.
Watch out for literals between two groups: a(b){2}c(d){3} -> abbcddd.

ALSO REPORTED · 1 more

Preparation: Drill the canonical k[group] decode-string problem to muscle memory; the input grammar there is k[group] instead of (group){k}, but the algorithm is identical.
Practice both recursive and iterative versions — the iterative stack version generalizes more cleanly if the interviewer asks about iterative streaming.
Test with: deeply nested groups, groups containing only literals, two-digit counts ({12}).

STUDY HERE: topic 1, Heaps, Stacks and Queues

Site tags: dedup, streaming, heap, sorting, medium

MEDIUM | DESIGN-STYLE (skippable) | 1Point3Acres | onsite-coding | 11 of 33

Batch and Streaming Log Deduplication

ASKED 2026-07-23 | 89 DAYS AGO | LAST 90 DAYS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 226/228**

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Frequency: Single report | Last asked: 2026-07-23 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/batch-streaming-log-deduplication

QUESTION

Process **`(timestamp, message)`** logs by deduplicating on message and sorting retained entries by timestamp, first keeping the first-seen entry and then the latest timestamp. The streaming extension adds **`add(timestamp, message)`** and **`get_next()`**, requiring a map plus min-heap, lazy deletion, and a discussion of watermarks and bounded state.SWEdedupstreamingheapsortingmediumFrequencySingle reportLast asked2026-07-23Stageonsite-coding

APPROACH

Implement both batch retention policies and write deterministic tie tests.
Build the streaming interface with a membership map, min-heap, and lazy deletion, then state the amortized operation costs.
Rehearse a bounded-memory design using expiry and heap rebuilding, plus a partitioning strategy that keeps each message on one shard.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask which entry to keep in each policy (first seen or latest timestamp), how ties on timestamp break, whether output is sorted by timestamp, and how the streaming interface returns results (get_next).

BRUTE FORCE TO SAY OUT LOUD

Sort all logs and dedupe with a set. Correct for the batch version.

OPTIMAL IDEA

Batch: keep a dict message to chosen entry (keep first-seen, or replace if the new timestamp is later) and sort the values by (timestamp, message). Streaming: a dict for membership plus a min-heap of (timestamp, message); on update of a message push a new entry and mark the old one stale (lazy deletion), and get_next pops until it finds a non-stale one. Amortized O(log n).

DRY-RUN THESE CASES

The same message twice at different times, equal timestamps, an empty stream, get_next on an empty queue.

LIKELY FOLLOW-UPS

Watermarks for late data, bounded memory with eviction by age.

PITFALLS

Returning a stale heap entry. Forgetting deterministic tie-breaking.

HOW TO THINK ABOUT IT

Dedup is a dictionary keyed by the message. Sorting is only for the output order. For a stream, a heap gives the smallest timestamp quickly, but when an entry is replaced you cannot delete from the middle of a heap, so leave the stale entry and skip it when it surfaces (lazy deletion).

Basis: Written. Assumed rules (confirm them): dedupe on the message; policy 'first' keeps the earliest in input order, policy 'latest' keeps the largest timestamp; output sorted by (timestamp, message); get_next returns the retained entry with the smallest timestamp and removes it.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(logs, keep_first=True):
    kept = []
    for entry in logs:
        same = [x for x in logs
                if x[1] == entry[1]]
        pick = max(same)
        if keep_first:
            pick = same[0]
        if entry == pick:
            kept.append(entry)
    return sorted(kept)  # by timestamp
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Each line scans the others: O(n^2). A dictionary from message to the kept entry is one pass, and a heap with lazy deletion serves the streaming version.

OPTIMAL SOLUTION · Dictionary on the message, then sort

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Streaming: map + min-heap with lazy deletion

TIME	SPACE
O(log n) amortized per add and get_next	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Late data: how do you decide when it is safe to emit?

Use a watermark: only release entries whose timestamp is at most (the largest timestamp seen minus the allowed lateness). get_next then refuses to return anything newer than the watermark, and anything arriving older than it is dropped or sent to a late-data side output.

FOLLOW-UP 2 Memory must stay bounded.

Evict messages older than a retention window (a second heap or the same heap keyed on timestamp) and accept that duplicates arriving after eviction are no longer detected. **Cost:** O(log n) per eviction with a heap; memory O(entries inside the retention window).

FOLLOW-UP 3 Keep the entry with the most occurrences instead.

Store (count, timestamp) per message and update the count on every duplicate; the heap key changes only if you rank by count, so push a new entry when the rank changes and rely on lazy deletion again.

FOLLOW-UP 4 Only release entries once they are safe (late data may still arrive).

Add a watermark: release only entries whose timestamp is at most (the largest timestamp seen minus the allowed lateness), in order; anything older than the watermark that arrives later goes to a late-data path. The lazy-deletion heap pops stale entries as it goes.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Batch and Streaming Log Deduplication
#
# Process (timestamp, message) logs by deduplicating on
# message and sorting retained entries by timestamp, first
# keeping the first-seen entry and then the latest
# timestamp. The streaming extension adds add(timestamp,
# message) and get_next(), requiring a map plus min-heap,
# lazy deletion, and a discussion of watermarks and
# bounded state.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Late data: how do you decide when it is safe to emit?
# 2. Memory must stay bounded.
# 3. Keep the entry with the most occurrences instead.
# 4. Only release entries once they are safe (late data
# may still arrive).
#
# INTERVIEW NOTES
# Dedup is a dictionary keyed by the message. Sorting is
# only for the output order. For a stream, a heap gives
# the smallest timestamp quickly, but when an entry is
# replaced you cannot delete from the middle of a heap, so
# leave the stale entry and skip it when it surfaces (lazy
# deletion).
#
# BRUTE FORCE (say it first): Quadratic scan for earlier
# or later duplicates
# For each log line, scan the rest (or the earlier lines)
# to see if it is the one to keep. Clear logic, quadratic
# cost. Time O(n^2). Space O(n).


import heapq


# BRUTE FORCE (say it first): Quadratic scan for earlier
# or later duplicates | time O(n^2)
def dedupe_brute(logs, policy="first"):
    keep = []
    for i, (ts, msg) in enumerate(logs):
        if policy == "first":
            ok = all(
                logs[j][1] != msg for j in range(i)
            )
        else:
            ok = all(
                logs[j][1] != msg
                or logs[j][0] < ts
                or (logs[j][0] == ts and j < i)
                for j in range(len(logs))
                if j != i
            )
        if ok:
            keep.append((ts, msg))
    return sorted(keep, key=lambda e: (e[0], e[1]))


# BASE QUESTION (optimal): Dictionary on the message, then
# sort
# Idea: One pass with a dict from message to the retained
# timestamp (first seen: keep the first; latest: replace
# when the new timestamp is larger), then sort the
# retained entries for the output.
def dedupe(logs, policy="first"):
    # logs: (timestamp, message) pairs; keep one per
```

```
    # message
    kept = {}
    for ts, msg in logs:
        # "first" keeps the first seen, "latest" the
        # newest
        if msg not in kept or (
                policy == "latest" and ts > kept[msg]):
            kept[msg] = ts
    # retained entries sorted by time, then message
    return sorted((ts, msg) for msg, ts in kept.items())


# FOLLOW-UP 4: Only release entries once they are safe
# (late data may still arrive).
# Answer: Add a watermark: release only entries whose
# timestamp is at most (the largest timestamp seen minus
# the allowed lateness), in order; anything older than the
# watermark that arrives later goes to a late-data path.
class StreamDedup:
    """add(ts, msg) and get_next() with a lazy heap."""

    def __init__(self, policy="first", lateness=0):
        self.policy, self.lateness = policy, lateness
        # message -> the timestamp currently kept
        self.kept = {}
```

```
        # (timestamp, message), may hold outdated entries
        self.heap = []
        self.max_ts = None

    def watermark(self):
        # everything at or below this is safe to release
        if self.max_ts is None:
            return None
        return self.max_ts - self.lateness

    def add(self, ts, msg):
        wm = self.watermark()
        # too late: older than the watermark, drop it
        if wm is not None and ts < wm:
            return False
        if self.max_ts is None or ts > self.max_ts:
            self.max_ts = ts
        old = self.kept.get(msg)
        if old is None or (
                self.policy == "latest" and ts > old):
            self.kept[msg] = ts
            heapq.heappush(self.heap, (ts, msg))
        return True

    def get_next(self):
        wm = self.watermark()
        while self.heap:
            ts, msg = self.heap[0]
            # lazy deletion: skip entries that were
            # replaced
            if self.kept.get(msg) != ts:
                heapq.heappop(self.heap)
                continue
            # not safe yet: later data may still arrive
            if ts > wm:
                return None
            heapq.heappop(self.heap)
            del self.kept[msg]
            return ts, msg
        return None


if __name__ == "__main__":
    logs = [(5, "b"), (1, "a"), (9, "a"), (3, "b")]
    assert dedupe(logs, "first") == [(1, "a"), (5, "b")]
    assert dedupe(logs, "latest") == [(5, "b"), (9, "a")]
    s = StreamDedup("first", lateness=2)
    for ts, msg in [(1, "a"), (3, "c"), (5, "b")]:
        assert s.add(ts, msg)
    # watermark is 5 - 2 = 3: release 1, then 3
    assert s.get_next() == (1, "a")
    assert s.get_next() == (3, "c")
    # 5 is still above the watermark
    assert s.get_next() is None
    # data older than the watermark is dropped
    assert not s.add(2, "late")
    # a "latest" policy replaces the kept entry
    t = StreamDedup("latest")
    t.add(1, "x")
    t.add(4, "x")
    assert t.get_next() == (4, "x")
    assert t.get_next() is None
```

NOTES FROM CANDIDATES

Empty input, identical entries, equal timestamps, out-of-order arrivals, updates, and removals all need explicit semantics.
For keep-latest streaming behavior, stale heap entries must be detected against the current timestamp stored for that message.
Arbitrarily late arrivals make globally final timestamp ordering impossible without a watermark or maximum-lateness guarantee.

PREPARATION

Implement both batch retention policies and write deterministic tie tests.
Build the streaming interface with a membership map, min-heap, and lazy deletion, then state the amortized operation costs.
Rehearse a bounded-memory design using expiry and heap rebuilding, plus a partitioning strategy that keeps each message on one shard.

STUDY HERE: topic 1, Heaps, Stacks and Queues

Site tags: heap, hashmap, scaling, medium

MEDIUM | IN APP LC 347 | 1Point3Acres | phone-screen, onsite-coding | 12 of 33

★ TOP 75: rank 29 in the Medium and Hard priority list

★ Top-K Frequent in Large Logs

ASKED 2026-07-19 | 93 DAYS AGO | LAST 6 MONTHS

STUDY: TYPE | **ALL TIME 58.8** | **30 DAYS 41.1** | **PRIORITY 21/228**

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues

Optimal answer from: your app's interview approach (LeetCode #347)

LeetCode match: LC 347 Top K Frequent Elements

Frequency: Medium | Last asked: 2026-07-19 | Roles: SWE, MLE

Original: https://www.1point3acres.com/interview/problems/company/google/top-k-frequent-from-logs

QUESTION

EXAMPLES

```
Input:  chatLog = ["A says hi", "B says hi", "A says
    bye", "C says hi", "A says yo"], k=2
Counts: A=3, B=1, C=1
Output: [A, B]  // or [A, C] — ties allowed
```

APPROACH

Practice writing quickselect with the partition step (and explain how to recover from a bad pivot).
Be able to state amortized vs worst-case time for each approach.
Have a one-line answer for streaming: "hashmap counts + heap of size K maintained online".
Drill the distributed answer once: per-shard top-K, then K-way merge of the partial heaps.

HOW TO THINK ABOUT IT

Count the speakers first; a log line is just a way to read the user id. Then you only need the K largest counts: a heap of size K does that without sorting all users. For logs spread over many machines the local top K are not enough on their own, so say what extra information you send.

Basis: LeetCode 347, Top K Frequent Elements (here the key is the speaker in each chat line).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(chat_log, k):
    names = [l.split()[0] for l in chat_log]
    counts = Counter(names)
    def rank(name):
        return (-counts[name], name)
    return sorted(counts, key=rank)[:k]
```

BRUTE FORCE

TIME	SPACE
O(n + d log d)	O(d)

WHY IT IS SLOW

Sorts all d distinct speakers, O(d log d), when only K are wanted; a size-K heap is O(d log k), and a stream needs a counter sketch instead of exact counts.

OPTIMAL SOLUTION · Count, then keep a size-K heap

TIME	SPACE
O(n + d log k)	O(d)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How do you do it across many machines when the log does not fit on one?

Count per shard and send each shard's local top-K to a coordinator. That is not exact in general (a user can be 11th everywhere yet first overall), so for an exact answer use the threshold algorithm: gather the local top-K, compute a global lower bound, ask every shard for all users above bound / numShards, and merge. If an approximate answer is acceptable, merge count-min sketches (they add) and keep a heap of the K largest estimates.

FOLLOW-UP 2 What if the log arrives as an unbounded stream?

Use the Misra-Gries or Space-Saving summary with m counters. Any user with more than n / (m + 1) messages is guaranteed to be in the summary, and a second pass (or a count-min sketch for the estimate) can confirm the real counts. **Cost:** O(1) per message with O(m) memory for m counters (Misra-Gries), instead of O(d) memory for the exact counts.

FOLLOW-UP 3 Which approach would you choose and why?

A heap is the safe default: simple, O(n + d log k) and streaming-friendly. Quickselect wins when k is a large fraction of d and you need all K but not their order. Sorting is fine only when d is small. Say that out loud, with the complexities, because the interviewer is grading the trade-off discussion.

FOLLOW-UP 4 The log is a stream and memory is limited.

Misra-Gries keeps m counters; any user with more than n / (m + 1) messages is guaranteed to remain in the summary (checked above). A second pass, or a count-min sketch, then gives the real counts of the survivors.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Top-K Frequent in Large Logs
#
# Examples: Input: chatLog = ["A says hi", "B says hi", "A
# says bye", "C says hi", "A says yo"], k=2 Counts: A=3,
# B=1, C=1 Output: [A, B] // or [A, C] — ties allowed
#
# FOLLOW-UPS (answered below, in this order):
# 1. How do you do it across many machines when the log
# does not fit on one?
# 2. What if the log arrives as an unbounded stream?
# 3. Which approach would you choose and why?
# 4. The log is a stream and memory is limited.
#
# INTERVIEW NOTES
# Count the speakers first; a log line is just a way to
# read the user id. Then you only need the K largest
# counts: a heap of size K does that without sorting all
# users. For logs spread over many machines the local top
# K are not enough on their own, so say what extra
# information you send.
#
# BRUTE FORCE (say it first): Count everything, then sort
# all users
# Count the speakers and sort every distinct user by
# (count, name). Correct and the oracle; sorting all d
# users does more work than keeping the best K. Time O(n +
# d log d). Space O(d).


import heapq
from collections import Counter


# BRUTE FORCE (say it first): Count everything, then sort
# all users | time O(n + d log d)
def top_k_speakers_sort(chat_log, k):
    c = Counter(
        line.split(" ", 1)[0] for line in chat_log
    )
    return sorted(c, key=lambda u: (-c[u], u))[:k]


# HELPERS for the base question
def speaker(line):
    # "A says hi" -> "A"
    return line.split(" ", 1)[0]


def rank_key(item):
    # more messages first; ties by name
    return (-item[1], item[0])


# BASE QUESTION (optimal): Count, then keep a size-K heap
# Idea: Parse the speaker from each line and count in a
# hash map. nsmallest keeps a heap of size k, so it never
# sorts all d distinct users. Ties break by user name so
# the output is deterministic.
def top_k_speakers(chat_log, k):
    counts = Counter(speaker(line) for line in chat_log)
    # a heap of size k: O(n + d log k)
    items = counts.items()
    best = heapq.nsmallest(k, items, key=rank_key)
    return [user for user, _ in best]


# FOLLOW-UP 1: How do you do it across many machines when
# the log does not fit on one?
# Answer: Count per shard and send each shard's local
# top-K to a coordinator.
def merge_shards(shard_logs, k):
    # exact answer: add the full counts of every shard
    total = Counter()
    for log in shard_logs:
        total.update(speaker(line) for line in log)
    best = heapq.nsmallest(k, total.items(), key=rank_key)
    return [user for user, _ in best]


# FOLLOW-UP 2: What if the log arrives as an unbounded
# stream?
# Answer: Use the Misra-Gries or Space-Saving summary with
# m counters. Any user with more than n / (m + 1) messages
# is guaranteed to be in the summary, and a second pass
# (or a count-min sketch for the estimate) can confirm the
# real counts.
# FOLLOW-UP 4: The log is a stream and memory is limited.
# Answer: Misra-Gries keeps m counters; any user with more
# than n / (m + 1) messages is guaranteed to remain in the
# summary (checked above). A second pass, or a count-min
# sketch, then gives the real counts of the survivors.
class MisraGries:
    """m counters: a user with over n / (m + 1) of the
    n messages always stays in the summary."""

    def __init__(self, m):
        self.m, self.counts = m, {}

    def add(self, user):
        if user in self.counts:
            self.counts[user] += 1
        elif len(self.counts) < self.m:
            self.counts[user] = 1
        else:
            # no room: every counter pays one
            for u in list(self.counts):
                self.counts[u] -= 1
                if self.counts[u] == 0:
                    del self.counts[u]

    def candidates(self):
        return set(self.counts)


if __name__ == "__main__":
    log = ["A says hi", "B says hi", "A says bye",
           "C says hi", "A says yo"]
    assert top_k_speakers(log, 2) == ["A", "B"]
    assert top_k_speakers(log, 5) == ["A", "B", "C"]
    assert top_k_speakers([], 3) == []
    # two shards that each rank someone else first
    s1 = ["A says x", "B says x"]
    s2 = ["B says y", "B says z"]
    assert merge_shards([s1, s2], 1) == ["B"]
    mg = MisraGries(2)
    for u in "AAABBC":
        mg.add(u)
    # A has 3 of 6 messages (> 6 / 3), so it must survive
    assert "A" in mg.candidates()
```

NOTES FROM CANDIDATES

The interviewer typically wants a discussion of the three implementations + complexity trade-off, not just one working solution.
For the distributed follow-up, candidates that mention count-min sketch and topK per shard -> global merge got positive feedback.
The same family surfaces as "Top-K IPs", "Top-K chatters", and "Top-K words"; the technique is interchangeable.
A recent variant uses a gigabyte-scale IP list and explicitly requires exact time and space complexity analysis.

PREPARATION

Practice writing quickselect with the partition step (and explain how to recover from a bad pivot).
Be able to state amortized vs worst-case time for each approach.
Have a one-line answer for streaming: "hashmap counts + heap of size K maintained online".
Drill the distributed answer once: per-shard top-K, then K-way merge of the partial heaps.

STUDY HERE: topic 1, Heaps, Stacks and Queues

MEDIUM | IN APP LC 1102 | PracHub | Technical Screen | 13 of 33

Find A Threshold-Limited Path With Minimum Required Safety

POSTED 2026-07-06 | 106 DAYS AGO | LAST 6 MONTHS

STUDY: TALK | ALL TIME 25.5 | 30 DAYS 0 | PRIORITY 136/228

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #1102 Path With Maximum Minimum Value

LeetCode match: LC 1102 Path With Maximum Minimum Value (inferred from overview)

Posted on PracHub: 2026-07-06 (updated 2026-07-08)

Original: https://prachub.com/coding-questions/find-a-threshold-limited-path-with-minimum-required-safety

QUESTION

Overview: Practice a graph path interview problem that asks for the minimum required safety value along a threshold-limited path. The prompt targets minimax path reasoning, filtered edges, Dijkstra-style relaxation, disconnected graphs, and edge-case handling.

APPROACH

Minimax Dijkstra: heap keyed by the best bottleneck (min safety along the path) while skipping edges beyond the threshold; or binary search the safety value + BFS.

HOW TO THINK ABOUT IT

When the quality of a path is decided by its weakest edge, the best path does not need to be short, only strong. Two equivalent tools: add edges from strongest to weakest with Union-Find until source and target connect (that edge's value is the answer), or run Dijkstra with a max-heap where extending a path takes the minimum so far. Disconnected graphs must return the 'no path' value.

Basis: LeetCode 1102 (Path With Maximum Minimum Value) and the bottleneck path on a graph. Assumed form (confirm it): edges (u, v, safety); the safety of a path is its smallest edge; find the best achievable path safety from source to target, or -1 if disconnected.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(graph, src, dst):
    best = -INF
    for path in all_paths(graph, src, dst):
        weakest = min(edge_safety(path))
        best = max(best, weakest)
    return best
```

BRUTE FORCE

TIME	SPACE
exponential	O(path)

WHY IT IS SLOW

Exponential. Adding edges from strongest to weakest in a Union-Find until the source and target connect is O(E log E).

OPTIMAL SOLUTION · Union-Find adding edges from strongest to weakest

TIME	SPACE
O(E log E)	O(V)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Use a max-heap search instead of Union-Find.

Dijkstra where the value of a path is the minimum edge so far and you maximise it: pop the best bottleneck, extend with min(value, edge). O(E log V); Union-Find is simpler when you need only the value.

FOLLOW-UP 2 Answer many (source, target) queries.

Build the maximum spanning tree once (Kruskal) and answer each query as the minimum edge on the tree path, with binary lifting for O(log V) per query.

FOLLOW-UP 3 Edges below a threshold are forbidden (threshold-limited).

Filter them out before the search (or stop adding edges below the threshold in the Union-Find loop); if source and target are still disconnected the answer is -1.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find A Threshold-Limited Path With
# Minimum Required Safety
#
# Practice a graph path interview problem that asks for
# the minimum required safety value along a threshold-
# limited path. The prompt targets minimax path reasoning,
# filtered edges, Dijkstra-style relaxation, disconnected
# graphs, and edge-case handling.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Use a max-heap search instead of Union-Find.
# 2. Answer many (source, target) queries.
# 3. Edges below a threshold are forbidden (threshold-
# limited).
#
# INTERVIEW NOTES
# When the quality of a path is decided by its weakest
```

```
# edge, the best path does not need to be short, only
# strong. Two equivalent tools: add edges from strongest
# to weakest with Union-Find until source and target
# connect (that edge's value is the answer), or run
# Dijkstra with a max-heap where extending a path takes
# the minimum so far. Disconnected graphs must return the
# 'no path' value.
#
# BRUTE FORCE (say it first): Try every simple path
# Depth-first search over simple paths from the source,
# tracking the minimum edge safety, and keep the best at
# the target. Exact and the oracle; exponential. Time
# exponential. Space O(path).


from collections import defaultdict
import heapq


# BRUTE FORCE (say it first): Try every simple path | time
# exponential
def bottleneck_brute(n, edges, s, t):
    adj = defaultdict(list)
    for u, v, w in edges:
        adj[u].append((v, w))
        adj[v].append((u, w))
    best = [-1]
```

```
    def go(u, seen, worst):
        if u == t:
            best[0] = max(best[0], worst)
            return
        for v, w in adj[u]:
            if v not in seen:
                go(v, seen | {v}, min(worst, w))

    go(s, {s}, float("inf"))
    return (
        best[0]
        if best[0] != float("inf")
        else float("inf")
    )


# HELPERS for the base question
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))

    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def union(self, a, b):
        self.parent[self.find(a)] = self.find(b)


# BASE QUESTION (optimal): Union-Find adding edges from
# strongest to weakest
# Idea: Sort the edges by safety, strongest first, and
# union their endpoints one by one.
def best_path_safety(n, edges, s, t, threshold=None):
    # edges are (u, v, safety). The safety of a path is
    # its
    # weakest edge; return the best safety, or -1 if s and
    # t
    # are not connected through edges >= threshold
    if s == t:
        return float("inf")
    uf = UnionFind(n)
    # add the safest edges first: the edge that joins s
    # and t
    # is the weakest edge of the best path
    for u, v, w in sorted(edges, key=lambda e: -e[2]):
        if threshold is not None and w < threshold:
            break
        uf.union(u, v)
        if uf.find(s) == uf.find(t):
            return w
    return -1


# FOLLOW-UP 1: Use a max-heap search instead of Union-
# Find.
# Answer: Dijkstra where the value of a path is the
# minimum edge so far and you maximise it: pop the best
# bottleneck, extend with min(value, edge). O(E log V);
# Union-Find is simpler when you need only the value.
def best_path_safety_heap(n, edges, s, t, threshold=None):
    # same answer with a max-heap (Dijkstra on the
    # bottleneck)
    graph = [[] for _ in range(n)]
    for u, v, w in edges:
        if threshold is None or w >= threshold:
            graph[u].append((v, w))
            graph[v].append((u, w))
    best = [-1] * n
    best[s] = float("inf")
    heap = [(-best[s], s)]
    while heap:
        neg, u = heapq.heappop(heap)
        if -neg < best[u]:
            continue
        for v, w in graph[u]:
            # extending a path can only lower its weakest
            # edge
            cand = min(-neg, w)
            if cand > best[v]:
                best[v] = cand
                heapq.heappush(heap, (-cand, v))
    return best[t]


if __name__ == "__main__":
    edges = [(0, 1, 5), (1, 2, 3), (0, 2, 1), (2, 3, 4)]
    # 0-1-2-3 has weakest edge 3; 0-2-3 only 1
```

```
    assert best_path_safety(4, edges, 0, 3) == 3
    assert best_path_safety_heap(4, edges, 0, 3) == 3
    # edges below the threshold are forbidden
    assert best_path_safety(4, edges, 0, 3, 4) == -1
    assert best_path_safety_heap(4, edges, 0, 3, 4) == -1
    # disconnected, and source equals target
    assert best_path_safety(3, [(0, 1, 9)], 0, 2) == -1
    assert best_path_safety(3, [], 1, 1) == float("inf")
    one = [(0, 1, 9)]
    assert best_path_safety_heap(3, one, 0, 2) == -1
```

FUNCTION AND TEST CASES

Sample call

```
solution(4, [(0,1,5),(1,3,4),(0,2,2),(2,3,8)], 0, 3, 6)
```

Test cases

Input	Expected
(4, [(0,1,5),(1,3,4),(0,2,2),(2,3,8)], 0, 3, 6)	5
(3, [(0,1,7),(1,2,3)], 0, 2, 5)	-1

STUDY HERE: topic 1, Heaps, Stacks and Queues

Site tags: parsing, recursion, stack, medium

MEDIUM | 1Point3Acres | onsite-coding | 14 of 33

Nested Function Expression Evaluator

ASKED 2026-06-10 | 132 DAYS AGO | LAST 6 MONTHS

STUDY: TALK **ALL TIME 5** **30 DAYS 0** **PRIORITY 175/228**

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 770 Basic Calculator IV (related)

Frequency: Low | Last asked: 2026-06-10 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/nested-function-expression-evaluator

QUESTION

EXAMPLES

```
add(1, sub(1, 0)) -> 2
add(2, mul(3, pow(4, 5))) -> 3074
```

APPROACH

Write a small tokenizer for identifiers, integers, commas, and parentheses.
Practice a recursive parseExpr() that returns both the value and the next index.
Drill the variable-arity follow-up by collecting arguments until) instead of assuming exactly two.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask which functions exist, whether they take a fixed or variable number of arguments, whether numbers can be negative, how division behaves, and how errors are reported.

BRUTE FORCE TO SAY OUT LOUD

Repeatedly find the innermost call with a regex and replace it with its value. Quick but fragile.

OPTIMAL IDEA

Tokenize into identifiers, integers, commas and parentheses. Write parse(i) returning (value, next index): a number returns itself; a name must be followed by '(', then loop reading arguments until ')', then apply the function. A function table keeps new operators one line each. O(n), recursion depth equals nesting depth.

DRY-RUN THESE CASES

add(1, sub(1, 0)) is 2. add(2, mul(3, pow(4, 5))) is 3074. A bare number. Variable argument counts. Unknown function names.

LIKELY FOLLOW-UPS

Variables, iterative parsing with two stacks, or division semantics.

PITFALLS

INTERVIEW-DAY PLAN

Losing the next index after parsing an argument. Treating '-' in '-5' as an operator.

HOW TO THINK ABOUT IT

An expression with function calls inside function calls has a grammar: expression = number | name '(' expression, expression, ... ')'. Each call can be evaluated only after its arguments, so write a function that parses one expression and reports where it stopped (recursive descent). A tokenizer first, then parse(i) returning (value, next index), keeps the bookkeeping small.

Basis: Recursive descent parsing, in the spirit of LeetCode 224 / 726; closest on LeetCode is 770, Basic Calculator IV.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def evaluate(text):
    while has_call(text):
        call = find_innermost_call(text)
        value = apply(call)  # e.g. add(2, 3)
        text = text.replace(call, str(value))
    return int(text)
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Every replacement rescans and rebuilds the string, O(n) per call and O(n^2) in total, and it cannot report errors well. A tokenizer plus recursive descent reads the text once, O(n).

OPTIMAL SOLUTION · Tokenizer + recursive descent over name(arg, arg, ...)

TIME	SPACE
O(n)	O(depth)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Support variadic functions and variables.

Collect arguments until ')' instead of assuming two (as the code does). For variables, pass an environment dict into parse and treat an identifier that is not followed by '(' as a lookup. Define the error behaviour for undefined names.

FOLLOW-UP 2 How do you make it iterative?

Use two stacks: one of pending function names and one of lists of evaluated arguments. On an identifier push the name and a new list, on a number append to the top list, on ')' pop both, apply the function, and append the result to the new top list. This removes the recursion-depth limit but is easier to get wrong off by one.

FOLLOW-UP 3 What about division?

State the contract: integer or float, truncate toward zero or floor (Python's // floors, which differs for negatives), and divide by zero handling. The implementation above truncates toward zero.

FOLLOW-UP 4 Add variables (an identifier with no parentheses is a lookup).

In parse, check whether the identifier is followed by '('; if not, return env[name]. Decide what an undefined name does (raise, or default to 0).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```python
# MAIN QUESTION: Nested Function Expression Evaluator
#
# Examples: add(1, sub(1, 0)) -> 2 add(2, mul(3, pow(4,
# 5))) -> 3074
#
# FOLLOW-UPS (answered below, in this order):
# 1. Support variadic functions and variables.
# 2. How do you make it iterative?
# 3. What about division?
# 4. Add variables (an identifier with no parentheses is a
# lookup).
#
# INTERVIEW NOTES
# An expression with function calls inside function calls
# has a grammar: expression = number | name '('
# expression, expression, ... ')'. Each call can be
# evaluated only after its arguments, so write a function
# that parses one expression and reports where it stopped
# (recursive descent). A tokenizer first, then parse(i)
# returning (value, next index), keeps the bookkeeping
# small.
#
# BRUTE FORCE (say it first): Repeatedly replace the
# innermost call with its value
# Find a call whose arguments contain no parentheses (a
# regular expression can do this), compute it, replace the
# text with the result, and repeat until only a number is
# left. It needs no parser, and it is quadratic because
# every replacement rescans and rebuilds the string. Time
# O(n^2). Space O(n).


import math
import re
```

```python
# BRUTE FORCE (say it first): Repeatedly replace the
# innermost call with its value | time O(n^2)
_OPS = {
    "add": lambda *a: sum(a),
    "sub": lambda a, b: a - b,
    "mul": lambda *a: math.prod(a),
    "pow": lambda a, b: a**b,
    "div": lambda a, b: int(a / b),
    "min": min,
    "max": max,
}


# HELPERS for the base question
TOKEN = re.compile(r"-?\d+|[A-Za-z_]\w*|[(),]")


# FOLLOW-UP 3: What about division?
# Answer: State the contract: integer or float, truncate
# toward zero or floor (Python's // floors, which differs
# for negatives), and divide by zero handling. The
# implementation above truncates toward zero.
# NEEDED BY THE BASE QUESTION: it is also the answer to
# the follow-up below
def divide(a, b):
    # integer division that truncates toward zero
    if b == 0:
        raise ZeroDivisionError("division by zero")
    return int(a / b)


OPS = {
    "add": lambda *a: sum(a),
    "sub": lambda a, b: a - b,
    "mul": lambda *a: math.prod(a),
    "pow": lambda a, b: a ** b,
    "div": divide,
    "min": min,
    "max": max,
}


def evaluate_by_replacement(expr):
    pattern = re.compile(r"([A-Za-z_]\w*)\(([^()]*)\)")
    while True:
        m = pattern.search(expr)
        if not m:
            return int(expr.strip())
        args = [int(x) for x in m.group(2).split(",")]
        expr = (
            expr[: m.start()]
            + str(_OPS[m.group(1)](*args))
            + expr[m.end() :]
        )
```

```python
# BASE QUESTION (optimal): Tokenizer + recursive descent
# over name(arg, arg, ...)
# Idea: Tokenize into identifiers, integers, commas and
# parentheses.
def evaluate(expr, env=None):
    env = env or {}
    tokens = TOKEN.findall(expr)

    def parse(i):
        # returns (value, index after this expression)
        tok = tokens[i]
        if re.fullmatch(r"-?\d+", tok):
            return int(tok), i + 1
        # FOLLOW-UP 4: variables come from env
        # a name not followed by "(" is a variable
```

```
        if i + 1 >= len(tokens) or tokens[i + 1] != "(":
            if tok not in env:
                raise ValueError(f"undefined {tok!r}")
            return env[tok], i + 1
        if tok not in OPS:
            raise ValueError(f"unknown function {tok!r}")
        i += 2
        args = []
        # read arguments until the closing bracket
        while tokens[i] != ")":
            value, i = parse(i)
            args.append(value)
            if tokens[i] == ",":
                i += 1
        return OPS[tok](*args), i + 1

    value, end = parse(0)
    if end != len(tokens):
        raise ValueError("trailing tokens")
    return value


# FOLLOW-UP 2: How do you make it iterative?
# Answer: Use two stacks: one of pending function names
# and one of lists of evaluated arguments.
def evaluate_iterative(expr):
    # two stacks: function names and their argument lists
    names, args, tokens = [], [[]], TOKEN.findall(expr)
```

```
    for tok in tokens:
        if re.fullmatch(r"-?\d+", tok):
            args[-1].append(int(tok))
        elif tok == "(":
            # the name just pushed now owns a new list
            args.append([])
        elif tok == ")":
            result = OPS[names.pop()](*args.pop())
            args[-1].append(result)
        elif tok != ",":
            names.append(tok)
    return args[0][0]


if __name__ == "__main__":
    assert evaluate("add(1, sub(1, 0))") == 2
    assert evaluate("add(2, mul(3, pow(4, 5)))") == 3074
    # variadic functions
    assert evaluate("add(1, 2, 3, 4)") == 10
    assert evaluate("max(3, min(9, 4), 1)") == 4
    # variables come from an environment
    assert evaluate("add(x, mul(x, 2))", {"x": 5}) == 15
    # division truncates toward zero
    assert evaluate("div(-7, 2)") == -3
    for bad in ("foo(1)", "add(1", "add(1) 2", "y"):
        try:
            evaluate(bad)
            raise AssertionError(bad)
        except (ValueError, IndexError):
            pass
    # the iterative version agrees and has no depth limit
    big = "add(2, mul(3, pow(4, 5)))"
    assert evaluate_iterative(big) == 3074
    deep = "add(1, " * 3000 + "0" + ")" * 3000
    assert evaluate_iterative(deep) == 3000
```

NOTES FROM CANDIDATES

Tokenizer + recursive descent is the cleanest approach: parse operator, consume (, parse comma-separated child expressions, consume), then evaluate.
Iterative parsing works but is easy to make off-by-one errors under interview pressure; one candidate was specifically flagged for minor syntax and bounds issues.
Clarify division behavior for negative values and non-divisible integers.

PREPARATION

Write a small tokenizer for identifiers, integers, commas, and parentheses.
Practice a recursive parseExpr() that returns both the value and the next index.
Drill the variable-arity follow-up by collecting arguments until) instead of assuming exactly two.

STUDY HERE: topic 1, Heaps, Stacks and Queues

MEDIUM | PracHub | Technical Screen | 15 of 33

★ TOP 75: rank 69 in the Medium and Hard priority list

★ Solve Rooms and Top-K Streams

POSTED 2026-06-05 | 137 DAYS AGO | LAST 6 MONTHS

STUDY: TYPE | ALL TIME 58 | 30 DAYS 0 | PRIORITY 40/228

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #253 Meeting Rooms II, LeetCode #703 Kth Largest Element in a Stream

LeetCode match: LC 253 Meeting Rooms II + LC 703 Kth Largest Element in a Stream (inferred from overview)

Posted on PracHub: 2026-06-05 (updated 2026-06-14)

Original: https://prachub.com/coding-questions/solve-rooms-and-top-k-streams

QUESTION

Overview: This question evaluates algorithmic problem-solving skills across interval scheduling/resource allocation and streaming top-k maintenance, testing competency in data structures for ordered queries, tie-breaking logic, stateful stream processing, and analysis of time and space trade-offs.

APPROACH

Rooms: min-heap of end times. Stream: size-k min-heap, push and pop when over k.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Two parts. Rooms: is a meeting that starts exactly when another ends a conflict (the scraped solution treats it as no conflict)? Stream: top-K means the K largest values or K most frequent, and what ties do. The scraped solution covers only the rooms part.

BRUTE FORCE TO SAY OUT LOUD

Rooms: assign meetings to rooms and try all orders. Stream: store everything and sort on each query.

OPTIMAL IDEA

Rooms: sort starts and ends separately and sweep with two pointers: if the next start is at least the earliest end, reuse that room, otherwise open a new one; or keep a min-heap of room end times. O(n log n). Stream of the K largest (LeetCode 703): a min-heap of size K, push each value and pop when the size exceeds K, so the root is the K-th largest. O(log K) per value.

DRY-RUN THESE CASES

Rooms: [[0,30],[5,10],[15,20]] gives 2, an empty list gives 0, back-to-back meetings. Stream: fewer than K values so far, duplicates, K = 1.

LIKELY FOLLOW-UPS

Return the room assignment, or a sliding window of the last N values for the stream.

PITFALLS

Sorting starts and ends together by start. Returning from the heap before it holds K items.

INTERVIEW-DAY PLAN

HOW TO THINK ABOUT IT

Rooms: the number of rooms needed is the largest number of meetings happening at the same time, so sort the starts and the ends separately and sweep: a start that comes before the earliest end needs a new room. Stream: only the K largest values can matter, so keep exactly those in a min-heap; the root is the K-th largest.

Basis: Two tasks: LeetCode 253 (minimum meeting rooms; back-to-back meetings may share a room) and LeetCode 703 (Kth Largest Element in a Stream).

BRUTE FORCE · Rooms: count the overlap at every time unit

BRUTE FORCE · PSEUDO-CODE

```
def rooms(meetings):
    lo = min(s for s, e in meetings)
    hi = max(e for s, e in meetings)
    best = 0
    for t in range(lo, hi):
        busy = 0
        for s, e in meetings:
            if s <= t < e:
                busy += 1
        best = max(best, busy)
    return best
```

TIME	SPACE
O(range * n)	O(1)

WHY IT IS SLOW

Depends on the length of the time range, not only on n: O(range * n). Sorted starts and ends with two pointers (or a heap of end times) are O(n log n).

BRUTE FORCE · Stream: sort everything on every query

BRUTE FORCE · PSEUDO-CODE

```
values = []
def add(x):
    values.append(x)
    values.sort(reverse=True)
    return values[K - 1]
```

BRUTE FORCE · Stream: sort everything on every query

TIME	SPACE
O(n log n) per add	O(n)

WHY IT IS SLOW

Every add re-sorts the whole history, O(n log n). A min-heap of size K keeps the K-th largest at its root in O(log K) per add.

OPTIMAL SOLUTION · Rooms: sorted starts and ends with two pointers

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Stream: min-heap of size K

TIME	SPACE
O(log k) per add	O(k)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Rooms: return which room each meeting uses.

Sort by start and keep a min-heap of (end time, room). If the earliest-ending room is free at the next start, reuse it, otherwise open a new room. The heap size never exceeds the answer.

FOLLOW-UP 2 Stream: K changes over time, or you need the median.

A fixed-size heap cannot answer a different K. Use a sorted structure with order statistics (balanced tree or Fenwick tree over values), or two heaps for the median. **Cost:** a balanced tree or Fenwick tree gives O(log n) per insert and per k-th query; two heaps give O(log n) per insert and O(1) per median.

FOLLOW-UP 3 Rooms: meetings on several floors with capacities.

Group meetings by required capacity, schedule each group separately with the sweep, and assign rooms from the smallest sufficient capacity (best fit) when capacities differ.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Solve Rooms and Top-K Streams
#
# This question evaluates algorithmic problem-solving
# skills across interval scheduling/resource allocation
# and streaming top-k maintenance, testing competency in
# data structures for ordered queries, tie-breaking logic,
# stateful stream processing, and analysis of time and
# space trade-offs.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Rooms: return which room each meeting uses.
# 2. Stream: K changes over time, or you need the median.
# 3. Rooms: meetings on several floors with capacities.
#
# INTERVIEW NOTES
# Rooms: the number of rooms needed is the largest number
# of meetings happening at the same time, so sort the
# starts and the ends separately and sweep: a start that
# comes before the earliest end needs a new room. Stream:
# only the K largest values can matter, so keep exactly
# those in a min-heap; the root is the K-th largest.
#
# BRUTE FORCE (say it first): Rooms: count the overlap at
# every time unit
# For each integer time t count the meetings with start <=
# t < end and take the maximum. Correct and the oracle
# (integer times only); it depends on the length of the
# time range, not just n. Time O(range * n). Space O(1).


import heapq


# BRUTE FORCE (say it first): Rooms: count the overlap at
# every time unit | time O(range * n)
def rooms_by_time(intervals):
    if not intervals:
        return 0
    return max(
        sum(1 for s, e in intervals if s <= t < e)
        for t in range(
            min(s for s, _ in intervals),
            max(e for _, e in intervals),
        )
    )
```

```
# BASE QUESTION (optimal): Rooms: sorted starts and ends
# with two pointers
# Idea: Sort starts and ends separately. For each start:
# if it is at or after the earliest unfinished end, that
# room is reused (advance the end pointer); otherwise a
# new room is needed.
def min_rooms(intervals):
    # two sorted lists: every start with no room free
    # needs
    # a new room, otherwise it reuses one that just ended
    starts = sorted(s for s, _ in intervals)
    ends = sorted(e for _, e in intervals)
    rooms = j = 0
    for s in starts:
        if s >= ends[j]:
            # the earliest-ending meeting is over: reuse
            # it
            j += 1
```

```
        else:
            rooms += 1
    return rooms


# PART 2: top K values of a stream
class TopKStream:
    """The k largest values seen so far (fixed k)."""

    def __init__(self, k):
        self.k, self.heap = k, []

    def add(self, x):
        # a min-heap of size k: the root is the k-th
        # largest
        if len(self.heap) < self.k:
            heapq.heappush(self.heap, x)
        elif x > self.heap[0]:
            heapq.heapreplace(self.heap, x)

    def top(self):
        return sorted(self.heap, reverse=True)


# VARIANT: median of the stream
```

```
class MedianStream:
    """Follow-up: the median instead of a fixed top-k."""

    def __init__(self):
        # low is a max-heap (negated), high is a min-heap
        self.low, self.high = [], []

    def add(self, x):
        heapq.heappush(self.low, -x)
        top = -heapq.heappop(self.low)
        heapq.heappush(self.high, top)
        if len(self.high) > len(self.low):
            back = heapq.heappop(self.high)
            heapq.heappush(self.low, -back)

    def median(self):
        if len(self.low) > len(self.high):
            return float(-self.low[0])
        return (-self.low[0] + self.high[0]) / 2


# FOLLOW-UP 1: Rooms: return which room each meeting uses.
# Answer: Sort by start and keep a min-heap of (end time,
# room). If the earliest-ending room is free at the next
# start, reuse it, otherwise open a new room. The heap
# size never exceeds the answer.
def assign_rooms(intervals):
    # which room does each meeting use? a heap of (end,
    # room)
    order = sorted(range(len(intervals)),
                   key=lambda i: intervals[i])
    heap, rooms, used = [], [0] * len(intervals), 0
    for i in order:
        start, end = intervals[i]
        if heap and heap[0][0] <= start:
            _, room = heapq.heappop(heap)
        else:
            room, used = used, used + 1
        rooms[i] = room
        heapq.heappush(heap, (end, room))
    return rooms


if __name__ == "__main__":
    meetings = [(0, 30), (5, 10), (15, 20)]
    assert min_rooms(meetings) == 2
    # a meeting may start the moment another ends
    assert min_rooms([(1, 3), (3, 5)]) == 1
    assert min_rooms([]) == 0
    rooms = assign_rooms(meetings)
    assert rooms[0] != rooms[1] and rooms[1] == rooms[2]
    t = TopKStream(3)
    for x in [5, 1, 9, 7, 3, 8]:
        t.add(x)
    assert t.top() == [9, 8, 7]
    m = MedianStream()
    for x in [5, 2, 8, 1]:
        m.add(x)
    assert m.median() == 3.5
```

FUNCTION AND TEST CASES

Signature

```
def solution(intervals: list[list[int]]) -> int:
```

Sample call

```
solution([[0, 30], [5, 10], [15, 20]])
```

Test cases

Input	Expected
`([[0, 30], [5, 10], [15, 20]],)`	`2`
`([],)`	`0`

STUDY HERE: topic 1, Heaps, Stacks and Queues

MEDIUM | IN APP LC 621 | PracHub | Technical Screen | 16 of 33

Rearrange Tasks With Cooldown

POSTED 2026-06-03 | 139 DAYS AGO | LAST 6 MONTHS

STUDY: TALK ALL TIME 40.3 30 DAYS 0 PRIORITY 92/228

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #621 Task Scheduler

LeetCode match: LC 621 Task Scheduler (inferred from overview)

Posted on PracHub: 2026-06-03 (updated 2026-06-06)

Original: https://prachub.com/coding-questions/rearrange-tasks-with-cooldown

QUESTION

Overview: This question evaluates task-scheduling and constraint-handling skills, testing algorithmic reasoning about arranging repeated jobs with cooldown constraints along with analysis of schedule optimality and algorithmic complexity.

APPROACH

Greedy with max-heap + cooldown queue, or the formula (maxFreq-1)*(n+1)+countOfMax vs len(tasks). Rearranged output: always take the available task with highest remaining count.

HOW TO THINK ABOUT IT

The most frequent task is the bottleneck. Lay it out with gaps of n between its copies; every other task can fill gaps. Total length is the larger of 'that skeleton' and 'just the number of tasks' (when there are so many tasks the gaps are all filled and no idle time is needed). The skeleton has (maxcount - 1) full blocks of size n + 1 plus one last block holding every task tied for the maximum.

Basis: LeetCode 621, Task Scheduler (minimum time), with LeetCode 358, Rearrange String k Distance Apart as the construction follow-up. Assumed rule: the same task needs n idle-or-other slots between runs.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def brute(tasks, n):
    left = Counter(tasks)
    free_at = {}  # task -> time it is ready
    t = 0
    while any(left.values()):
        ready = [x for x in left if left[x]
                 and free_at.get(x, 0) <= t]
        if ready:
            task = max(ready, key=left.get)
            left[task] -= 1
            free_at[task] = t + n + 1
        t += 1  # idle if none ready
    return t
```

TIME	SPACE
O(T log k) where T is the answer	O(k)

WHY IT IS SLOW

It simulates every slot, including long idle stretches, O(T log k). The answer follows from the most frequent task: max(len(tasks), (maxCount - 1) * (n + 1) + number of tasks with that count).

OPTIMAL SOLUTION · Closed form from the most frequent task

TIME	SPACE
O(len(tasks))	O(1) for a fixed alphabet

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Print an actual schedule, not just the time.

Use the heap-with-cooldown-queue construction: run the available task with the most copies left, park it for n slots, and emit an idle marker when nothing is available. Or rearrange a string so equal letters are at least k apart.

FOLLOW-UP 2 Different cooldowns per task.

The closed form no longer applies. Simulate with a heap, storing for each task the time it becomes available; pick the ready task with the most remaining copies, and jump the clock to the earliest availability when nothing is ready.

FOLLOW-UP 3 Tasks have priorities or deadlines.

Then it is a scheduling problem; say what you optimise (lateness, completion time) and use a heap ordered by deadline for the ready tasks.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Rearrange Tasks With Cooldown
#
# This question evaluates task-scheduling and constraint-
# handling skills, testing algorithmic reasoning about
# arranging repeated jobs with cooldown constraints along
# with analysis of schedule optimality and algorithmic
# complexity.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Print an actual schedule, not just the time.
# 2. Different cooldowns per task.
# 3. Tasks have priorities or deadlines.
#
# INTERVIEW NOTES
# The most frequent task is the bottleneck. Lay it out
# with gaps of n between its copies; every other task can
# fill gaps. Total length is the larger of 'that skeleton'
# and 'just the number of tasks' (when there are so many
# tasks the gaps are all filled and no idle time is
# needed). The skeleton has (maxcount - 1) full blocks of
# size n + 1 plus one last block holding every task tied
# for the maximum.
#
# BRUTE FORCE (say it first): Simulate each time slot,
# always running the available task with the most copies
# left
# At every time unit pick, among tasks not cooling down,
# the one with the most remaining copies; if none is
# available the slot is idle. A heap for ready tasks and a
# queue of tasks in cooldown make it work. Correct, but it
# simulates every slot, including long idle stretches.
# Time O(T log k) where T is the answer. Space O(k).


import heapq
from collections import Counter, deque


# BRUTE FORCE (say it first): Simulate each time slot,
# always running the available task with the most copies
# left | time O(T log k) where T is the answer
def brute(tasks, n):
    left = Counter(tasks)
    free_at = {}  # task -> time it is ready
    t = 0
```

```
    while any(left.values()):
        ready = [
            x
            for x in left
            if left[x] and free_at.get(x, 0) <= t
        ]
        if ready:
            task = max(ready, key=left.get)
            left[task] -= 1
            free_at[task] = t + n + 1
        t += 1  # idle if none ready
    return t


# BASE QUESTION (optimal): Closed form from the most
# frequent task
# Idea: Let m be the highest frequency and k the number of
# tasks that have that frequency. The skeleton needs (m -
# 1) * (n + 1) + k slots.
def least_interval(tasks, n):
```

```
    # closed form: the most frequent task sets the frame
    counts = Counter(tasks).values()
    most = max(counts, default=0)
    # how many tasks are tied for the most
    tied = sum(1 for c in counts if c == most)
    if not tasks:
        return 0
    # (most - 1) full frames of n + 1 slots, plus the last
    # row
    return max(len(tasks), (most - 1) * (n + 1) + tied)


# FOLLOW-UP 1: Print an actual schedule, not just the
# time.
# Answer: Use the heap-with-cooldown-queue construction:
# run the available task with the most copies left, park
# it for n slots, and emit an idle marker when nothing is
# available. Or rearrange a string so equal letters are at
# least k apart.
def schedule(tasks, n):
    # an actual schedule, with "idle" where nothing may
    # run
    heap = [(-c, t) for t, c in Counter(tasks).items()]
    heapq.heapify(heap)
    # tasks cooling down: (time ready, -left, task)
```

```
    cooling, out, now = deque(), [], 0
    while heap or cooling:
        # bring back tasks whose cooldown is over
        while cooling and cooling[0][0] <= now:
            _, left, task = cooling.popleft()
            heapq.heappush(heap, (left, task))
        if heap:
            # run the task with the most copies left
            left, task = heapq.heappop(heap)
            out.append(task)
            if left + 1 < 0:
                ready = now + n + 1
                cooling.append((ready, left + 1, task))
        else:
            out.append("idle")
        now += 1
    return out


# HELPERS for the checks below
def valid(plan, n):
    # equal tasks must be more than n slots apart
    last = {}
    for i, task in enumerate(plan):
        if task == "idle":
            continue
        if task in last and i - last[task] <= n:
            return False
        last[task] = i
    return True


if __name__ == "__main__":
    tasks = list("AAABBB")
    assert least_interval(tasks, 2) == 8
    assert least_interval(tasks, 0) == 6
    assert least_interval([], 3) == 0
    # many different tasks: no idle time needed
    assert least_interval(list("ABCDEFG"), 2) == 7
    plan = schedule(tasks, 2)
    assert len(plan) == 8 and valid(plan, 2)
    assert plan.count("idle") == 2
    for t, n in ((list("AAAABBC"), 3), (list("AB"), 5)):
        plan = schedule(t, n)
        assert valid(plan, n)
```

```
        assert len(plan) == least_interval(t, n)
```

FUNCTION AND TEST CASES

Signature

```
def solution(tasks: list[str], cooldown: int) -> str:
```

Sample call

```
solution(['A', 'A', 'A', 'B', 'B', 'C'], 2)
```

Test cases

Input	Expected
`(['A', 'A', 'A', 'B', 'B', 'C'], 2)`	`'ABCAB_A'`
`(['A', 'A', 'A', 'B'], 2)`	`'AB_A__A'`

ALSO REPORTED · 1 more

Design a task scheduler in Python. | Exponent

https://www.aced.io/questions/3504/design-task-scheduler-python

STUDY HERE: topic 1, Heaps, Stacks and Queues

MEDIUM | PracHub | Technical Screen | 17 of 33

Determine if a 14-tile hand is winning

POSTED 2026-03-09 | 225 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 19.6 | 30 DAYS 0 | PRIORITY 149/228

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #659 Split Array into Consecutive Subsequences

LeetCode match: No direct match; related LC 659 Split Array into Consecutive Subsequences

Posted on PracHub: 2026-03-09 (updated 2026-04-19)

Original: https://prachub.com/coding-questions/determine-if-a-14-tile-hand-is-winning

QUESTION

You are given an integer array **`tiles`** of length 14 representing a Mahjong-like hand. Each integer is a tile value from 1 to 9 (single suit). You may reorder the tiles. A hand is **winning** if it can be partitioned into:

- **1 pair**: two identical tiles, and - **4 melds** (each meld uses 3 tiles), where each meld is either: - a **triplet**: three identical tiles (e.g., 7,7,7), or - a **sequence**: three consecutive values (e.g., 2,3,4). Each tile can be used at most once. Return **`true`** if the hand is winning, otherwise return **`false`**. Assumptions/constraints:
- **`tiles.length == 14`**
- **`1 <= tiles[i] <= 9`**
- Each value can appear multiple times (as in a real tile set).

APPROACH

Count array of tiles 1..9; try each value as the pair, then backtrack: take the smallest remaining tile as a triplet or a run of three. Tiny search space.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether sequences are only consecutive in one suit (values 1 to 9), whether a tile can be used only once, and whether other special hands exist (assume not).

BRUTE FORCE TO SAY OUT LOUD

Try every permutation of the 14 tiles and test the grouping. Far too slow.

OPTIMAL IDEA

Keep a count array of tiles 1 to 9. Try each value that has at least two tiles as the pair, remove it, then backtrack on the remaining 12 tiles: take the smallest value that still has tiles and either remove a triplet of it or a run of three consecutive values starting at it; if neither is possible, fail. Always handling the smallest value makes the search small. Time is tiny for 14 tiles.

DRY-RUN THESE CASES

Four triplets and a pair, four runs and a pair, a hand with two possible groupings, a hand with a pair but impossible melds, all 14 tiles equal (invalid since each value has at most four copies).

LIKELY FOLLOW-UPS

More suits, seven pairs, or all possible winning tiles for a 13-tile hand.

PITFALLS

Greedy without backtracking: a run versus a triplet choice can matter. Forgetting to restore counts.

INTERVIEW-DAY PLAN

HOW TO THINK ABOUT IT

Work with counts per value (at most nine numbers), not with the tile list. Try every value as the pair, then handle the lowest remaining value: it must belong to a triplet or to a sequence starting at it (nothing smaller is left to pair with it). Always processing the smallest value makes the search tiny and avoids trying the same grouping in different orders.

Basis: Written; a small exhaustive search like the Mahjong 'standard hand' check. Rule: 14 tiles of one suit with values 1..9 split into one pair and four melds, each meld a triplet (x, x, x) or a sequence (x, x + 1, x + 2).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def is_win(counts):
    for v in range(1, 10):
        if counts[v] >= 2:
            counts[v] -= 2  # the pair
            ok = remove_melds(counts)
            counts[v] += 2
            if ok:
                return True
    return False
def remove_melds(counts):
    v = first_value_with_tiles(counts)
    if v is None:
        return True
    for meld in (triple(v), run(v)):
        if can_take(counts, meld):
            take(counts, meld)
            ok = remove_melds(counts)
            give_back(counts, meld)
            if ok:
                return True
    return False
```

TIME	SPACE
exponential but small for...	**O(depth)**
exponential but small for 14 tiles	

BRUTE FORCE

WHY IT IS SLOW

It tries the same groupings in many different orders (still tiny for 14 tiles). Always grouping the smallest remaining tile tries each grouping once.

OPTIMAL SOLUTION · Counts, try each pair, then always group the smallest remaining tile

TIME	SPACE
O(9 * small branching) = constant for 14 tiles	**O(1)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Which tile would complete a 13-tile hand?

Try each of the nine values as the 14th tile and run the same check: 9 checks of a tiny search. That is how a player finds the tiles they are waiting for.

FOLLOW-UP 2 More suits or special hands (seven pairs).

Run the single-suit check on each suit separately (melds never cross suits) after splitting the hand; seven pairs is a separate one-line count check (seven values with count 2).

FOLLOW-UP 3 Why is the 'smallest remaining value' rule enough?

The smallest remaining value cannot be the middle or end of a sequence (the smaller values are gone), so it must be in a triplet or the start of a sequence. Trying those two options covers every legal grouping exactly once.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Determine if a 14-tile hand is winning
#
# You are given an integer array tiles of length 14
# representing a Mahjong-like hand. Each integer is a tile
# value from 1 to 9 (single suit). You may reorder the
# tiles. A hand is **winning** if it can be partitioned
# into: - **1 pair**: two identical tiles, and - **4
# melds** (each meld uses 3 tiles), where each meld is
# either: - a **triplet**: three identical tiles (e.g.,
# 7,7,7), or - a **sequence**: three consecutive values
# (e.g., 2,3,4). Each tile can be used at most once.
# Return true if the hand is winning, otherwise return
# false. Assumptions/constraints: - tiles.length == 14 - 1
# <= tiles[i] <= 9 - Each value can appear multiple times
# (as in a real tile set).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Which tile would complete a 13-tile hand?
# 2. More suits or special hands (seven pairs).
# 3. Why is the 'smallest remaining value' rule enough?
#
# INTERVIEW NOTES
# Work with counts per value (at most nine numbers), not
# with the tile list. Try every value as the pair, then
# handle the lowest remaining value: it must belong to a
# triplet or to a sequence starting at it (nothing smaller
# is left to pair with it). Always processing the smallest
# value makes the search tiny and avoids trying the same
# grouping in different orders.
#
# BRUTE FORCE (say it first): Remove any triplet or
# sequence from anywhere, exhaustively
# For each pair candidate, repeatedly try removing any
# triplet or any sequence available among the counts (in
# any order) and see whether everything can be removed.
# Correct and the oracle; it tries the same groupings in
# many orders, unlike the smallest-first search. Time
# exponential but small for 14 tiles. Space O(depth).


from collections import Counter


# BRUTE FORCE (say it first): Remove any triplet or
# sequence from anywhere, exhaustively | time exponential
# but small for 14 tiles
def winning_brute(tiles):
    cnt = [0] * 12
    for t in tiles:
        cnt[t] += 1

    def melds(c):
        if sum(c) == 0:
            return True
        for v in range(1, 10):
            if c[v] >= 3:
                c[v] -= 3
                ok = melds(c)
                c[v] += 3
                if ok:
                    return True
            if (
                v <= 7
                and c[v]
                and c[v + 1]
                and c[v + 2]
            ):
                c[v] -= 1
                c[v + 1] -= 1
                c[v + 2] -= 1
                ok = melds(c)
                c[v] += 1
                c[v + 1] += 1
                c[v + 2] += 1
                if ok:
                    return True
        return False

    for v in range(1, 10):
        if cnt[v] >= 2:
            cnt[v] -= 2
            ok = melds(cnt)
            cnt[v] += 2
            if ok:
                return True
    return False


# HELPERS for the base question
def melds_ok(cnt, v=1):
```

```
    # can the tiles in cnt (value -> count) form only
    # melds?
    while v <= 9 and cnt[v] == 0:
        v += 1
    if v > 9:
        return True
    # the smallest tile is either in a triplet ...
    if cnt[v] >= 3:
        cnt[v] -= 3
        ok = melds_ok(cnt, v)
        cnt[v] += 3
        if ok:
            return True
    # ... or the start of a run v, v + 1, v + 2
    if v <= 7 and cnt[v + 1] and cnt[v + 2]:
        for t in (v, v + 1, v + 2):
            cnt[t] -= 1
        ok = melds_ok(cnt, v)
        for t in (v, v + 1, v + 2):
            cnt[t] += 1
        return ok
    return False


# HELPERS for follow-up 2
def count_of(values):
    cnt = Counter({v: 0 for v in range(1, 12)})
    for v in values:
        cnt[v] += 1
    return cnt


def is_winning_hand_with_len(values):
    cnt = count_of(values)
    for pair in range(1, 10):
        if cnt[pair] >= 2:
            cnt[pair] -= 2
            ok = melds_ok(cnt)
            cnt[pair] += 2
            if ok:
                return True
    return False


# BASE QUESTION (optimal): Counts, try each pair, then
```

```
# always group the smallest remaining tile
# Idea: Keep counts of 1..9. For each value with at least
# two tiles, take it as the pair.
def is_winning_hand(tiles):
    # one pair plus four melds, single suit, values 1..9
    cnt = Counter({v: 0 for v in range(1, 12)})
    for t in tiles:
        cnt[t] += 1
    for pair in range(1, 10):
        if cnt[pair] >= 2:
            cnt[pair] -= 2
            ok = melds_ok(cnt)
            cnt[pair] += 2
            if ok:
                return True
    return False


# FOLLOW-UP 1: Which tile would complete a 13-tile hand?
# Answer: Try each of the nine values as the 14th tile and
# run the same check: 9 checks of a tiny search. That is
# how a player finds the tiles they are waiting for.
def waiting_tiles(tiles13):
    # follow-up: which 14th tile completes the hand?
    return [t for t in range(1, 10)
            if is_winning_hand(tiles13 + [t])]
```

```
# FOLLOW-UP 2: More suits or special hands (seven pairs).
# Answer: Run the single-suit check on each suit
# separately (melds never cross suits) after splitting the
# hand; seven pairs is a separate one-line count check
# (seven values with count 2).
def is_seven_pairs(tiles):
    # a special hand: seven different pairs
    counts = Counter(tiles)
    pairs = all(c == 2 for c in counts.values())
    return len(tiles) == 14 and pairs


# FOLLOW-UP 2: More suits or special hands (seven pairs).
# Answer: Run the single-suit check on each suit
# separately (melds never cross suits) after splitting the
# hand; seven pairs is a separate one-line count check
# (seven values with count 2).
```

```
def is_winning_multi(hand):
    # several suits: hand is a list of (suit, value);
    # melds
    # never cross suits, and exactly one suit holds the
    # pair
    suits = {}
    for suit, v in hand:
        suits.setdefault(suit, []).append(v)
    pair_suits = [s for s in suits
                  if len(suits[s]) % 3 == 2]
    if len(pair_suits) != 1:
        return False
    for s, vs in suits.items():
        if s == pair_suits[0]:
            if not is_winning_hand_with_len(vs):
                return False
        elif len(vs) % 3 or not melds_ok(count_of(vs)):
            return False
    return True


if __name__ == "__main__":
    win = [1, 1, 1, 2, 3, 4, 5, 5, 5, 6, 7, 8, 9, 9]
    assert is_winning_hand(win)
    # 3 and 6 are missing, so the runs cannot be formed
    lose = [1, 2, 4, 5, 7, 8, 1, 2, 4, 5, 7, 8, 9, 9]
    assert not is_winning_hand(lose)
    # thirteen tiles waiting for a 9 (or a 6)
    wait = [1, 1, 1, 2, 3, 4, 5, 5, 5, 6, 7, 8, 9]
    assert 9 in waiting_tiles(wait)
    assert is_seven_pairs([1, 1, 2, 2, 3, 3, 4, 4, 5, 5,
                           6, 6, 7, 7])
    hand = [("m", v) for v in (1, 1, 1, 2, 3)] + [
        ("p", v) for v in (4, 5, 6, 7, 7, 7)] + [
        ("s", v) for v in (2, 2, 2)]
    assert is_winning_multi(hand)
```

FUNCTION AND TEST CASES

Signature

```
def solution(tiles: list[int]) -> bool:
```

Sample call

```
solution([1,1,1,2,3,4,2,3,4,5,6,7,9,9])
```

Test cases

Input	Expected
([1,1,1,2,3,4,2,3,4,5,6,7,9,9],)	True
([2,2,2,3,3,3,4,4,4,5,5,5,9,9],)	True

ALSO REPORTED · 1 more

Mahjong Winning Hand Detection (14 Tiles) | 1Point3Acres | onsite-coding | asked 2026-07-20 | Low

https://www.1point3acres.com/interview/problems/company/google/mahjong-winning-hand-check

Clarify the rules — the prompt is intentionally sparse. Confirm: are honor tiles (winds/dragons) included? Do flowers count? Multi-suit?
Greedy + backtracking is the standard approach:
Try every possible pair (tile, tile); remove the pair.
On the remaining 12 tiles, repeatedly take the smallest remaining tile and try to form a triplet of it or a run starting at it; backtrack on failure.
For 14 tiles this runs in well under 1 ms; the interviewer cares more about the case analysis and clean recursion than micro-optimizations.
The same algorithm extends to special hands (seven pairs / ????) if the interviewer adds them as a follow-up.

Preparation: Practice writing the backtracking decomposer for tiles / counters; the canonical "expression add operators" recursion trains the same backtracking skeleton.
Have a tile-frequency-map mental model — sorting is the alternative but harder to reason about for runs across suits.
Pre-plan for the inevitable rules clarification: pull out at least 3 questions before coding.

STUDY HERE: topic 1, Heaps, Stacks and Queues

MEDIUM | PracHub | Technical Screen | 18 of 33

★ TOP 75: rank 28 in the Medium and Hard priority list

★ Implement frequency + distance top-K queries

POSTED 2026-02-12 | 250 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 58.8 | 30 DAYS 41.1 | PRIORITY 20/228

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #347 Top K Frequent Elements, LeetCode #973 K Closest Points to Origin

LeetCode match: LC 347 Top K Frequent Elements + LC 973 K Closest Points to Origin

Posted on PracHub: 2026-02-12 (updated 2026-04-28)

Original: https://prachub.com/coding-questions/implement-frequency-plus-distance-top-k-queries

QUESTION

You will implement solutions for two coding interview questions.

Question

1: Return the top-K points by frequency (with distance tie-break) You are given an array **`points`** of 2D integer points, where each point is represented as **`[x, y]`**. The array may contain duplicates (the same **`[x, y]`** can appear multiple times). Define:

- **`freq(p)`** = number of times point **p** appears in the array.
- **`dist(p)`** = squared Euclidean distance to the origin, **`x^2 + y^2`**. Return **k distinct points** sorted by the following priority:

1. Higher frequency first (**`freq`** descending)
2. If frequencies tie, smaller distance first (**`dist`** ascending)
3. If still tied, sort by **x** ascending, then **y** ascending

Input

- **`points`**: list of length **n**, each element **`[x, y]`**
- **k**: integer, **`1 <= k <= (# of distinct points)`**

Output

- A list of **k** distinct points following the ranking above.

Constraints (typical interview assumptions)

- **`1 <= n <= 2 * 10^5`**
- **`-10^4 <= x, y <= 10^4`** ---

Question

2: Time-based key-value store with “closest timestamp” lookup Design a data structure supporting the following operations:

- **`set(key, value, timestamp)`**: store the string **`value`** for string **`key`** at integer **`timestamp`**.

QUESTION

- Assume timestamps for the same **`key`** are inserted in **non-decreasing** order.
- **`getClosest(key, timestamp) -> value or ""`**:
- If **`key`** has no stored values, return **`""`**.
- Otherwise, return the **`value`** whose stored timestamp is **closest** to the query **`timestamp`**.
- If there is a tie (equal distance to two timestamps), return the value with the **smaller** timestamp.

Example

If for key **`"a"`** you stored timestamps **`[2, 10, 15]`** with values **`["x", "y", "z"]`**:

- **`getClosest("a", 11)`** returns **`"y"`** (10 is closest)
- **`getClosest("a", 14)`** returns **`"z"`** (15 is closest)
- **`getClosest("a", 12)`** returns **`"y"`** (10 and 15 are equally far; choose smaller timestamp)

What to discuss

- Time and space complexity of your approach - How you would improve a “naive” solution (e.g., linear scan) to a more optimal one

Constraints (typical interview assumptions)

- Up to **`2 * 10^5`** total operations - Keys and values are strings; timestamps are integers within typical 32-bit range

Part 1: Top-K Points by Frequency with Distance Tie-BreakGiven a list of 2D integer points, where duplicate points may appear, return the top **k** distinct points ranked by these rules: (1) higher frequency first, (2) if frequencies tie, smaller squared distance to the origin first, and (3) if still tied, smaller **x** first, then smaller **y**. The result must contain exactly **k** distinct points in ranked order.
Constraints
1 <= len(points) <= 2 * 10^5-10^4 <= x, y <= 10^41 <= k <= number of distinct points

EXAMPLES

```
Input: ([[1,1],[2,2],[2,2],[1,1],[2,2],[3,3]], 2)
Expected Output: [[2, 2], [1, 1]]
Explanation: Point [2,2] appears 3 times and [1,1]
    appears 2 times, so they are the top 2 distinct
    points.
Input: ([[1,2],[0,2],[1,2],[2,1],[0,2],[2,1],[3,0]],
    3)
Expected Output: [[0, 2], [1, 2], [2, 1]]
Explanation: The points [0,2], [1,2], and [2,1] each
    appear twice. Their squared distances are 4, 5,
    and 5, so [0,2] comes first. Between [1,2] and
    [2,1], x is smaller for [1,2].
```

QUESTION

Hints

First count how many times each point appears. After that, each distinct point only needs to be ranked once.Python can sort tuples directly, so think about building a sort key like `(-frequency, distance, x, y)`.

Part 2: Time-Based Key-Value Store with Closest Timestamp QueriesProcess a sequence of operations for a time-based key-value store. Each operation is either `['set', key, value, timestamp]` or `['getClosest', key, timestamp]`. For every `getClosest` operation, return the stored value whose timestamp is closest to the query tim

APPROACH

Count frequencies in a hash map; use a size-k heap with key (frequency desc, distance asc) for Q1, and a max-heap of size k on distance for Q2.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Two questions. Q1 (top-K points): confirm the full ordering (higher frequency, then smaller distance, then smaller x, then smaller y) and that K is at most the number of distinct points. Q2 (time-based store): timestamps for a key arrive in non-decreasing order, and a tie in distance returns the smaller timestamp.

BRUTE FORCE TO SAY OUT LOUD

Q1: sort all distinct points with the full key. Q2: linear scan of a key's timestamps. Both are correct and easy to state.

OPTIMAL IDEA

Q1: Counter over tuples, then heapq.nsmallest(k, distinct, key=(-freq, x*x + y*y, x, y)), O(n + d log k). Q2: per key keep parallel lists of timestamps and values; bisect_left finds the first timestamp at least the query; compare it with the previous one and return the closer value, the smaller timestamp on a tie. O(log m) per query.

DRY-RUN THESE CASES

Q1: duplicates, ties on frequency and distance, K equal to the number of distinct points. Q2: query before the first timestamp, after the last, exactly on a timestamp, the tie example (a, 12 returns y), an unknown key.

LIKELY FOLLOW-UPS

Q1: top-K in a stream. Q2: an out-of-order insert (then use a sorted container), deletes, and memory limits.

PITFALLS

Q1: sorting by distance only. Q2: off-by-one at the ends of the list.

HOW TO THINK ABOUT IT

Q1 is a ranking: say the full sort key out loud, count with a hash map, and then you only need the K best (heap), not a full sort. Q2 keeps each key's timestamps in a sorted list, so the nearest one is next to the insertion point: binary search, then compare the two neighbours.

Basis: Two tasks: top K points by (frequency desc, squared distance asc, x asc, y asc), a mix of LeetCode 347 and 973; and a time-based store whose getClosest returns the value at the nearest timestamp (ties to the smaller one), a variant of LeetCode 981.

BRUTE FORCE · Q1: sort all distinct points by the full key

BRUTE FORCE · PSEUDO-CODE

```
def top_k_points(points, k):
    counts = Counter(map(tuple, points))
    def rank(p):
        x, y = p
        return (-counts[p], x * x + y * y, x, y)
    return sorted(counts, key=rank)[:k]
```

TIME	SPACE
O(d log d)	O(d)

WHY IT IS SLOW

Sorts all d distinct points, O(d log d); a size-K heap with the same key is O(d log k).

BRUTE FORCE · Q2: scan every timestamp of the key

BRUTE FORCE · PSEUDO-CODE

```
def get_closest(key, t):
    best = None
    for ts, value in store[key]:
        gap = abs(ts - t)
        if best is None or (gap, ts) < best[:2]:
            best = (gap, ts, value)
    return best[2]
```

TIME	SPACE
O(m) per query	O(total entries)

WHY IT IS SLOW

Linear in the number of stored timestamps per query. Timestamps arrive in order, so binary search on the sorted list finds the neighbours in O(log m).

OPTIMAL SOLUTION · Q1: heap with the full key

TIME	SPACE
O(n + d log k)	O(d)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Q2: parallel sorted lists and bisect

TIME	SPACE
O(1) per set, O(log m) per getClosest	**O(total entries)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Q1 with points arriving over time.

Keep a running Counter and compute the top K on demand with a heap (O(d log k) per query), or maintain a sorted structure updated on every add if queries dominate.

FOLLOW-UP 2 Q2 with timestamps arriving out of order.

The append keeps the list sorted only for in-order input. For arbitrary order keep a sorted list and insert with bisect.insort (O(m) per insert) or a balanced tree (O(log m)); the query is unchanged.

FOLLOW-UP 3 Q2 returns the closest value at or before the timestamp (LeetCode 981).

Use bisect_right(timestamps, ts) - 1 and return that value or an empty string if the index is negative; there is no tie question.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Implement frequency + distance top-K
# queries
#
# You will implement solutions for two coding interview
# questions. ## Question 1: Return the top-K points by
# frequency (with distance tie-break) You are given an
# array points of 2D integer points, where each point is
# represented as [x, y]. The array may contain duplicates
# (the same [x, y] can appear multiple times). Define: -
# freq(p) = number of times point p appears in the array.
# - dist(p) = squared Euclidean distance to the origin,
# x^2 + y^2. Return **k distinct points** sorted by the
# following priority: 1. Higher frequency first (freq
# descending) 2. If frequencies tie, smaller distance
# first (dist ascending) 3. If still tied, sort by x
# ascending, then y ascending ### Input - points: list of
# length n, each element [x, y] - k: integer, 1 <= k <= (#
# of distinct points) ### Output - A list of k distinct
# points following the ranking above.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Q1 with points arriving over time.
# 2. Q2 with timestamps arriving out of order.
# 3. Q2 returns the closest value at or before the
# timestamp (LeetCode 981).
#
# INTERVIEW NOTES
# Q1 is a ranking: say the full sort key out loud, count
# with a hash map, and then you only need the K best
# (heap), not a full sort. Q2 keeps each key's timestamps
# in a sorted list, so the nearest one is next to the
# insertion point: binary search, then compare the two
# neighbours.
#
# BRUTE FORCE (say it first): Q1: sort all distinct points
# by the full key
# Count the points, sort the distinct ones by (-frequency,
# distance, x, y), take K. Correct; sorting all d distinct
# points does more work than a heap of size K. Time O(d
# log d). Space O(d).


import heapq
from bisect import bisect_left, bisect_right, insort
from collections import Counter, defaultdict
```

```
# BRUTE FORCE (say it first): Q1: sort all distinct points
# by the full key | time O(d log d)
def top_points_sort(points, k):
    c = Counter(map(tuple, points))
    return [
        list(p)
        for p in sorted(
            c,
            key=lambda p: (
                -c[p],
                p[0] ** 2 + p[1] ** 2,
                p[0],
                p[1],
            ),
        )[:k]
    ]


# BASE QUESTION (optimal): Q1: heap with the full key
# Idea: Count with a Counter, then heapq.nsmallest(k, ...)
# with the key (-frequency, squared distance, x, y).
# Squared distances avoid square roots and are exact for
# integers.
def top_points(points, k):
    # more frequent first, then closer to the origin, then
    # x, y
    counts = Counter(map(tuple, points))

    def rank(p):
        dist = p[0] ** 2 + p[1] ** 2
        return (-counts[p], dist, p[0], p[1])

    best = heapq.nsmallest(k, counts, key=rank)
    return [list(p) for p in best]


# PART 2: closest timestamp value
class TimeStore:
    """set(key, value, ts) and closest-time reads."""

    def __init__(self):
        # key -> sorted [(timestamp, value)]
        self.data = defaultdict(list)
```

```
    def set(self, key, value, ts):
        # insort also handles timestamps arriving out of
        # order
        insort(self.data[key], (ts, value))

    def get_closest(self, key, ts):
        items = self.data.get(key)
        if not items:
            return ""
```

```
        # the first stored time at or after ts, and the
        # one before
        i = bisect_left(items, (ts,))
        options = items[max(0, i - 1):i + 1]
        # closest wins; on a tie the smaller timestamp
        # wins
        def gap(item):
            return (abs(item[0] - ts), item[0])
        best = min(options, key=gap)
        return best[1]

    def get_at_or_before(self, key, ts):
        # LeetCode 981: the latest value not after ts
        items = self.data.get(key, [])
        i = bisect_right(items, (ts, chr(0x10FFFF))) - 1
        return items[i][1] if i >= 0 else ""


# FOLLOW-UP 1: Q1 with points arriving over time.
# Answer: Keep a running Counter and compute the top K on
# demand with a heap (O(d log k) per query), or maintain a
# sorted structure updated on every add if queries
# dominate.
class TopPointsStream:
    # follow-up: points arrive one at a time. The counts
    # are
    # kept as points arrive, so top_k never recounts
    # history.
    def __init__(self):
        self.counts = Counter()

    def add(self, x, y):
        self.counts[(x, y)] += 1

    def top_k(self, k):
```

```
        def rank(p):
            dist = p[0] ** 2 + p[1] ** 2
            return (-self.counts[p], dist, p[0], p[1])

        best = heapq.nsmallest(k, self.counts, key=rank)
        return [list(p) for p in best]


if __name__ == "__main__":
    pts = [[1, 1], [1, 1], [2, 0], [0, 2], [2, 0], [3, 3]]
    # (1,1) and (2,0) appear twice; (2,0) is farther? 4 vs
    # 2
    assert top_points(pts, 2) == [[1, 1], [2, 0]]
    four = top_points(pts, 4)
    assert four == [[1, 1], [2, 0], [0, 2], [3, 3]]
    assert top_points([[5, 5]], 1) == [[5, 5]]
    live = TopPointsStream()
    for x, y in pts:
        live.add(x, y)
    assert live.top_k(2) == [[1, 1], [2, 0]]
    live.add(3, 3)
    live.add(3, 3)
    live.add(3, 3)
    # (3, 3) is now the most frequent point
    assert live.top_k(1) == [[3, 3]]
    s = TimeStore()
    for ts, v in ((2, "x"), (10, "y"), (15, "z")):
        s.set("a", v, ts)
    assert s.get_closest("a", 11) == "y"
    assert s.get_closest("a", 14) == "z"
    # a tie: 10 and 15 are both 2.5 away from 12.5, 12 ->
    # 10
    assert s.get_closest("a", 12) == "y"
    assert s.get_closest("b", 1) == ""
    assert s.get_closest("a", 100) == "z"
    assert s.get_closest("a", -5) == "x"
    assert s.get_at_or_before("a", 14) == "y"
```

```
    assert s.get_at_or_before("a", 1) == ""
    s.set("a", "late", 5)
    assert s.get_closest("a", 6) == "late"
```

FUNCTION AND TEST CASES

Signature

```
def solution(points: list[list[int]], k: int) -> list[list
[int]]:
```

Sample call

```
solution([[1,1],[2,2],[2,2],[1,1],[2,2],[3,3]], 2)
```

Test cases

Input	Expected
([[1,1],[2,2],[2,2],[1,1],[2,2],[3,3]], 2)	[[2, 2], [1, 1]]
([[1,2],[0,2],[1,2],[2,1],[0,2],[2,1] ,[3,0]], 3)	[[0, 2], [1, 2], [2, 1]]

STUDY HERE: topic 1, Heaps, Stacks and Queues

MEDIUM | IN APP LC 253 | PracHub | Technical Screen | 19 of 33

★ TOP 75: rank 50 in the Medium and Hard priority list

★ Compute minimum number of rooms needed

POSTED 2026-02-12 | 250 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 58 | 30 DAYS 0 | PRIORITY 42/228

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #253 Meeting Rooms II

LeetCode match: LC 253 Meeting Rooms II

Posted on PracHub: 2026-02-12 (updated 2026-05-04)

Original: https://prachub.com/coding-questions/compute-minimum-number-of-rooms-needed

QUESTION

Problem

You are given a list of meetings, each with a start time and end time. A single room can host only one meeting at a time. Two meetings **overlap** if one starts before the other ends (treat meetings as half-open intervals **[start, end)** so a meeting ending at time `t` does not conflict with another starting at time `t`).

Task

Return the **minimum number of rooms** required to host all meetings.

Input

- `intervals`: an array of `n` pairs `[start, end]` where `0 <= start <= end`.

Output

- An integer: the minimum number of rooms needed.

Constraints (typical interview bounds)

- `1 <= n <= 200000`
- Times are integers in a reasonable range (e.g., `0 . 10^9`).

Example

- Input: `[[0,30],[5,10],[15,20]]`
- Output: `2` (One room can host `[0,30]`, the other can host `[5,10]` then `[15,20]`.)

You are given a list of meetings, where each meeting is represented as [start, end]. A single room can host only one meeting at a time. Meetings are treated as half-open intervals [start, end), so a meeting ending at time t does not conflict with another meeting starting at time t. Return the minimum number of rooms required to host all meetings. If start == end, the meeting occupies no time and does not require a room.
Constraints
0 <= n <= 2000000 <= start <= end <= 10^9

QUESTION

EXAMPLES

```
Input: ([[0,30],[5,10],[15,20]],)
Expected Output: 2
Explanation: [0,30] overlaps with both [5,10] and
    [15,20], so at most 2 rooms are needed at the
    same time.
Input: ([[7,10],[2,4],[4,7],[10,12]],)
Expected Output: 1
```

Explanation: These meetings only touch at endpoints, so one room can host all of them in sequence.

Hints

Try turning each meeting into two events: one when it starts and one when it ends.Because intervals are half-open, if a meeting ends at the same time another starts, process the ending event first.

APPROACH

Sort by start, min-heap of end times (reuse a room when earliest end <= start), or two-pointer sweep on sorted starts/ends. O(n log n).

HOW TO THINK ABOUT IT

The rooms needed equal the largest number of meetings that overlap at one moment. Two ways to find that without checking every moment: sort by start and keep a min-heap of the end times of rooms in use (reuse the room that frees up first), or sort starts and ends separately and sweep with two pointers.

Basis: LeetCode 253, Meeting Rooms II (half-open intervals [start, end): a meeting ending at t does not conflict with one starting at t).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(meetings):
    lo = min(s for s, e in meetings)
    hi = max(e for s, e in meetings)
    best = 0
    for t in range(lo, hi):
        busy = 0
        for s, e in meetings:
            if s <= t < e:
                busy += 1
        best = max(best, busy)
    return best
```

TIME	SPACE
O(range * n)	O(1)

WHY IT IS SLOW

Depends on the length of the time range, O(range * n). A min-heap of room end times, or sorted starts and ends with two pointers, is O(n log n).

OPTIMAL SOLUTION · Sort by start with a min-heap of end times

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Can one person attend all meetings (LeetCode 252)?

Sort by start and check that each meeting ends no later than the next one starts. O(n log n), O(1) extra.

FOLLOW-UP 2 Return which room each meeting uses.

Keep a heap of (end time, room id): reuse the room that frees first, otherwise open the next id, and record the id per meeting.

FOLLOW-UP 3 Meetings arrive online.

Keep the end-time heap between calls: pop every end at or before the new start (those rooms are free), then push the new end. The heap size is the number of rooms in use right now; the peak is the answer so far.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Compute minimum number of rooms needed
#
# ## Problem You are given a list of meetings, each with a
# start time and end time. A single room can host only one
# meeting at a time. Two meetings **overlap** if one
# starts before the other ends (treat meetings as half-
# open intervals **[start, end)** so a meeting ending at
# time t does not conflict with another starting at time
# t). ### Task Return the **minimum number of rooms**
# required to host all meetings. ### Input - intervals: an
# array of n pairs [start, end] where 0 <= start <= end.
# ### Output - An integer: the minimum number of rooms
# needed. ### Constraints (typical interview bounds) - 1
# <= n <= 200000 - Times are integers in a reasonable
# range (e.g., 0 . 10^9).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Can one person attend all meetings (LeetCode 252)?
# 2. Return which room each meeting uses.
# 3. Meetings arrive online.
#
# INTERVIEW NOTES
# The rooms needed equal the largest number of meetings
# that overlap at one moment. Two ways to find that
# without checking every moment: sort by start and keep a
# min-heap of the end times of rooms in use (reuse the
# room that frees up first), or sort starts and ends
# separately and sweep with two pointers.
#
# BRUTE FORCE (say it first): Count the overlap at every
# time unit
# For each integer time count the meetings with start <= t
# < end and take the maximum. Correct and the oracle
# (integer times); it depends on the length of the time
# range, not only on n. Time O(range * n). Space O(1).


import heapq


# BRUTE FORCE (say it first): Count the overlap at every
# time unit | time O(range * n)
def rooms_by_time(intervals):
    if not intervals:
        return 0
    return max(
        sum(1 for s, e in intervals if s <= t < e)
        for t in range(
            min(s for s, _ in intervals),
            max(e for _, e in intervals) + 1,
        )
    )


# BASE QUESTION (optimal): Sort by start with a min-heap
# of end times
# Idea: Process meetings by start time. If the earliest-
# ending room has finished by this meeting's start, reuse
# it (pop and push the new end); otherwise open a new room
# (push).
def min_meeting_rooms(intervals):
    # half-open [start, end): a meeting ending at t does
    # not
    # clash with one starting at t. A heap holds the end
    # times
    # of the rooms in use.
    ends = []
    meetings = sorted(m for m in intervals if m[0] < m[1])
    for start, end in meetings:
        if ends and ends[0] <= start:
            # the earliest room is free: reuse it
            heapq.heapreplace(ends, end)
        else:
            heapq.heappush(ends, end)
    return len(ends)


# FOLLOW-UP 1: Can one person attend all meetings
# (LeetCode 252)?
# Answer: Sort by start and check that each meeting ends
# no later than the next one starts. O(n log n), O(1)
# extra.
def can_attend_all(intervals):
    # LeetCode 252: one person, no overlap at all
    ordered = sorted(intervals)
    pairs = zip(ordered, ordered[1:])
    return all(a[1] <= b[0] for a, b in pairs)


# FOLLOW-UP 2: Return which room each meeting uses.
# Answer: Keep a heap of (end time, room id): reuse the
# room that frees first, otherwise open the next id, and
# record the id per meeting.
def assign_rooms(intervals):
    # which room does each meeting use?
    order = sorted(range(len(intervals)),
                   key=lambda i: intervals[i])
    heap, rooms, used = [], [None] * len(intervals), 0
    for i in order:
        start, end = intervals[i]
        if heap and heap[0][0] <= start:
```

```
            _, room = heapq.heappop(heap)
        else:
            room, used = used, used + 1
        rooms[i] = room
        heapq.heappush(heap, (end, room))
    return rooms


# FOLLOW-UP 3: Meetings arrive online.
# Answer: Keep the end-time heap between calls: pop every
# end at or before the new start (those rooms are free),
# then push the new end. The heap size is the number of
# rooms in use right now; the peak is the answer so far.
class RoomTracker:
    """Meetings arrive online (in order of start time)."""

    def __init__(self):
        self.ends, self.peak = [], 0

    def book(self, start, end):
        # rooms whose meeting already ended are free again
        while self.ends and self.ends[0] <= start:
            heapq.heappop(self.ends)
        heapq.heappush(self.ends, end)
        # the heap size is the rooms in use right now
        self.peak = max(self.peak, len(self.ends))
        return len(self.ends)


if __name__ == "__main__":
    meetings = [(0, 30), (5, 10), (15, 20)]
    assert min_meeting_rooms(meetings) == 2
    assert min_meeting_rooms([(7, 10), (2, 4)]) == 1
    # touching meetings share a room; empty ones need none
```

```
    touching = [(1, 3), (3, 5), (4, 4)]
    assert min_meeting_rooms(touching) == 1
    assert min_meeting_rooms([]) == 0
    assert not can_attend_all(meetings)
    assert can_attend_all([(7, 10), (2, 4)])
    rooms = assign_rooms(meetings)
    assert rooms[0] != rooms[1] and rooms[1] == rooms[2]
    t = RoomTracker()
    in_use = [t.book(s, e) for s, e in meetings]
    assert in_use == [1, 2, 2]
    assert t.peak == 2
```

FUNCTION AND TEST CASES

Signature

```
def solution(intervals: list[list[int]]) -> int:
```

Sample call

```
solution([[0,30],[5,10],[15,20]])
```

Test cases

Input	Expected
`([[0,30],[5,10],[15,20]],)`	`2`
`([[7,10],[2,4],[4,7],[10,12]],)`	`1`

ALSO REPORTED · 4 more

Find Minimum Rooms Needed | PracHub | Technical Screen

https://prachub.com/coding-questions/find-minimum-rooms-needed

Compute minimum rooms for time intervals | PracHub | Technical Screen

https://prachub.com/coding-questions/compute-minimum-rooms-for-time-intervals

Given a list of car rental requests with start and end times, assign cars to rentals such | Exponent

https://www.aced.io/questions/5425/assign-cars-minimize-total-cars-rented

Meeting Rooms / Interval Scheduling Variants | 1Point3Acres | phone-screen, tech-screen, onsite-coding | asked 2026-09-20 | Very high

https://www.1point3acres.com/interview/problems/company/google/meeting-rooms-variants

Asked in Round 1 coding, Round 2 onsite coding, phone screen, and as a warm-up in the FDE loop. Almost every other coding loop contains some flavor of this family.
Interviewer commonly chains two follow-ups: (1) reconstruct assignment, (2) scale out to many shards / regions.
For the assignment follow-up, min-heap keyed on end time is the clean answer: pop the earliest-ending room, push the new interval.
For "distributed across shards", expect open-ended discussion: per-shard sweep, cross-shard merge, reconciliation; the interviewer often acknowledges the question is intentionally vague.
The on-call metadata variant includes detailed complexity follow-ups: account for heap operations, stored employee information, and output size separately, and explain each symbol in the complexity expression.

Preparation: Drill three solutions cold: sweep-line / prefix delta, min-heap on end time, sort + greedy room assignment.
Be ready to convert between the three when interviewers add follow-ups ("give me the assignment", "now handle cyclic days").
Prepare a 1-minute pitch on map-reduce style sharding (per-shard intervals -> merged timeline -> global max).

STUDY HERE: topic 1, Heaps, Stacks and Queues

MEDIUM | PracHub | Technical Screen | 20 of 33

Find longest increasing contiguous subarray

POSTED 2026-01-06 | 287 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 19.6 | 30 DAYS 0 | PRIORITY 146/228

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues

Optimal answer from: the interview site's own solution

LeetCode match: LC 674 Longest Continuous Increasing Subsequence (variant: one modification) (inferred from overview)

Posted on PracHub: 2026-01-06 (updated 2026-06-26)

Original: https://prachub.com/coding-questions/find-longest-increasing-contiguous-subarray

QUESTION

Overview: This question evaluates proficiency in array processing and algorithmic optimization, focusing on recognition of contiguous subarray properties and handling a single-value modification to achieve strict monotonicity.

APPROACH

Compute run lengths ending at i (left) and starting at i (right); for each i try changing nums[i] to bridge: left[i-1] + 1 + right[i+1] if nums[i+1] - nums[i-1] >= 2. O(n).

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether increasing is strict, whether the modified value can be any integer or any real, and whether you must change exactly one element or at most one.

BRUTE FORCE TO SAY OUT LOUD

Try changing each element to every useful value and recompute the run. O(n^2).

OPTIMAL IDEA

Compute left[i], the length of the strictly increasing run ending at i, and right[i], the run starting at i. Changing nums[i] can bridge the neighbors if nums[i + 1] - nums[i - 1] >= 2 (integers), giving left[i - 1] + 1 + right[i + 1]. Otherwise you can only extend one side: left[i - 1] + 1 or right[i + 1] + 1. Take the maximum over all i, and also the run with no change. O(n).

DRY-RUN THESE CASES

n = 1, a strictly decreasing array, a plateau like [1, 2, 2, 3], neighbors that differ by exactly 1 (cannot bridge with integers), the change at the first or last index.

LIKELY FOLLOW-UPS

Allow k changes (needs a sliding window with at most k bad positions), allow non-strict increase, or return the subarray bounds.

INTERVIEW-DAY PLAN

PITFALLS

Forgetting the integer gap condition. Index errors at the array ends.

HOW TO THINK ABOUT IT

A run that has a single 'bad' spot can be repaired by changing one element, so look at each position i as the element to change. The best run through i joins the increasing run that ends just before it and the one that starts just after it, and the join is legal only if the values leave enough room between the two neighbours (for integers, a[i + 1] - a[i - 1] >= 2).

Basis: LeetCode 674 (Longest Continuous Increasing Subsequence) with one allowed modification. Assumed rule (confirm it): strictly increasing, you may change at most one element to any integer.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(a):
    best = 0
    for i in range(len(a)):
        for j in range(i, len(a)):
            part = a[i:j + 1]
            if fixable_with_one_change(part):
                best = max(best, len(part))
    return best
def fixable_with_one_change(part):
    bad = [k for k in range(1, len(part))
           if part[k] <= part[k - 1]]
    if not bad:
        return True
    if len(bad) > 1:
        return False
    return has_room(part, bad[0])
```

TIME	SPACE
O(n^3)	O(1)

WHY IT IS SLOW

O(n^2) subarrays, each tested in O(n): O(n^3). Run lengths ending and starting at each index, then trying each index as the changed element, is O(n).

OPTIMAL SOLUTION · Left and right run lengths, try every position as the changed element

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 No modification allowed (LeetCode 674).

One pass keeping the length of the current strictly increasing run; reset to 1 when a[i] <= a[i - 1]. O(n), O(1).

FOLLOW-UP 2 Up to k modifications.

Use a sliding window over the array that allows at most k positions where the increasing property fails and cannot be repaired; careful with the integer room condition between neighbours. Often handled with DP over (position, changes used).

FOLLOW-UP 3 Values are real numbers, not integers.

Then there is always room between two different neighbours, so the join condition becomes a[i + 1] > a[i - 1] instead of a gap of at least 2.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find longest increasing contiguous
# subarray
#
# This question evaluates proficiency in array processing
# and algorithmic optimization, focusing on recognition of
# contiguous subarray properties and handling a single-
# value modification to achieve strict monotonicity.
#
# FOLLOW-UPS (answered below, in this order):
# 1. No modification allowed (LeetCode 674).
# 2. Up to k modifications.
# 3. Values are real numbers, not integers.
#
# INTERVIEW NOTES
# A run that has a single 'bad' spot can be repaired by
# changing one element, so look at each position i as the
# element to change. The best run through i joins the
# increasing run that ends just before it and the one that
# starts just after it, and the join is legal only if the
# values leave enough room between the two neighbours (for
# integers, a[i + 1] - a[i - 1] >= 2).
#
# BRUTE FORCE (say it first): Test every subarray for
# 'increasing after at most one change'
# For each subarray count the places where an element is
# not larger than its predecessor, and check whether one
# change can fix it (one such spot and the neighbours
# leave room, or none). Exact and slow; the oracle for the
# O(n) method. Time O(n^3). Space O(1).


# BRUTE FORCE (say it first): Test every subarray for
# 'increasing after at most one change' | time O(n^3)
def longest_one_change_brute(a):
    n = len(a)
    best = 0
    for i in range(n):
        for j in range(i, n):
            sub = a[i : j + 1]
            ok = any(
                all(
                    x[k] < x[k + 1]
                    for k in range(len(x) - 1)
                )
                for x in (
                    [sub]
                    + [
                        sub[:k] + [v] + sub[k + 1 :]
                        for k in range(len(sub))
                        for v in range(
                            min(a) - len(a) - 2,
                            max(a) + len(a) + 3,
                        )
                    ]
                )
            )
            if ok:
                best = max(best, len(sub))
    return best


# BASE QUESTION (optimal): Left and right run lengths, try
# every position as the changed element
# Idea: left[i] is the length of the strictly increasing
# run ending at i and right[i] the length of the one
# starting at i.
def longest_one_change(a, integers=True):
    # strictly increasing after changing AT MOST one value
    n = len(a)
    if n == 0:
        return 0
    # left[i]: run ending at i; right[i]: run starting at
    # i
    left, right = [1] * n, [1] * n
    for i in range(1, n):
        if a[i] > a[i - 1]:
            left[i] = left[i - 1] + 1
    for i in range(n - 2, -1, -1):
        if a[i] < a[i + 1]:
            right[i] = right[i + 1] + 1
    # the gap needed between two neighbours: 2 for whole
    # numbers (one number must fit between), > 0 for reals
    best = max(left)
    for i in range(n):
        # change a[i] to join the run on its left ...
        if i > 0:
            best = max(best, left[i - 1] + 1)
        # ... or the run on its right ...
        if i < n - 1:
            best = max(best, right[i + 1] + 1)
        # ... or both, when a value fits between the
        # neighbours
        if 0 < i < n - 1:
            room = a[i + 1] - a[i - 1]
            fits = room >= 2 if integers else room > 0
            if fits:
                joined = left[i - 1] + 1 + right[i + 1]
                best = max(best, joined)
    return min(best, n)


# FOLLOW-UP 1: No modification allowed (LeetCode 674).
# Answer: One pass keeping the length of the current
# strictly increasing run; reset to 1 when a[i] <= a[i -
# 1]. O(n), O(1).
def longest_increasing(a):
    # LeetCode 674: no change allowed, one pass
    best = run = 0
    for i, x in enumerate(a):
        run = run + 1 if i and a[i - 1] < x else 1
        best = max(best, run)
    return best
```

```
if __name__ == "__main__":
    assert longest_increasing([1, 3, 5, 4, 7]) == 3
    assert longest_increasing([]) == 0
    assert longest_one_change([1, 3, 5, 4, 7]) == 5
    # equal neighbours leave no room for a whole number
    assert longest_one_change([1, 2, 2, 3]) == 3
    nums = [1, 2, 2, 3]
    reals = longest_one_change(nums, integers=False)
    assert reals == 4
    assert longest_one_change([5, 4, 3, 2, 1]) == 2
    assert longest_one_change([7]) == 1
    assert longest_one_change([]) == 0
    # never more than the array length
    assert longest_one_change([1, 2, 3]) == 3
```

FUNCTION AND TEST CASES

Signature

```
def longestIncreasing(nums: list[int]) -> int:
```

Sample call

```
longestIncreasing([1, 2, 2, 3, 4])
```

Test cases

Input	Expected
`([1, 2, 2, 3, 4],)`	`3`
`([1, 2, 3, 4, 5],)`	`5`

STUDY HERE: topic 1, Heaps, Stacks and Queues

MEDIUM | IN APP LC 2402 | PracHub | Technical Screen | 21 of 33

Simulate room assignments for scheduled meetings

POSTED 2026-01-06 | 287 DAYS AGO | OLDER

STUDY: TALK ALL TIME 40 30 DAYS 0 PRIORITY 95/228

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues

Optimal answer from: your app's interview approach (LeetCode #2402)

LeetCode match: LC 2402 Meeting Rooms III (inferred from overview)

Posted on PracHub: 2026-01-06 (updated 2026-03-29)

Original: https://prachub.com/interview-questions/simulate-room-assignments-for-scheduled-meetings

QUESTION

Overview: This question evaluates scheduling and event-simulation skills, including time-ordered processing, resource allocation with tie-breaking, and efficient tracking of room availability and usage counts using appropriate data structures.

APPROACH

Sort meetings by start, min-heap of free rooms (by index) and a heap of busy rooms (end time, room); assign the lowest free room, else wait for the earliest finish and delay.

HOW TO THINK ABOUT IT

Simulate in start order with two questions per meeting: which rooms are free now (lowest index first), and if none, which room frees up first (the meeting is delayed but keeps its length). Two heaps answer both quickly: one of free room ids, one of (end time, room id) for busy rooms.

Basis: LeetCode 2402, Meeting Rooms III.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(n, meetings):
    end = [0] * n
    count = [0] * n
    for start, finish in sorted(meetings):
        free = [r for r in range(n)
                if end[r] <= start]
        if free:
            r = min(free)
            end[r] = finish
        else:
            r = end.index(min(end))  # earliest
            wait = end[r] - start
            end[r] = finish + wait
        count[r] += 1
    return count.index(max(count))
```

BRUTE FORCE

TIME	SPACE
O(M * N)	O(N)

WHY IT IS SLOW

Every meeting scans all N rooms, O(M * N). Two heaps (free rooms and busy rooms by end time) make each meeting O(log N).

OPTIMAL SOLUTION · Two heaps: free rooms and busy rooms

TIME	SPACE
O(M log M + M log N)	O(N)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What if meetings arrive online and cannot be reordered?

The same two heaps work if you process meetings in arrival order and treat the arrival time as the earliest allowed start, but you can no longer rely on sorted starts, so you must first move every room whose end time is <= the arrival into the free heap. Each call stays O(log N).

FOLLOW-UP 2 How do you report the busiest room by total busy time instead of meeting count?

Add the meeting's actual duration to used[room] instead of 1 when you assign it. Rooms are chosen the same way; only the tally changes, and ties still break toward the lowest index.

FOLLOW-UP 3 Rank rooms by total busy time instead of the number of meetings.

Add each meeting's duration (its original length, not the delay) to the room's tally; the choice of room is unchanged.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Simulate room assignments for scheduled
# meetings
#
# This question evaluates scheduling and event-simulation
# skills, including time-ordered processing, resource
# allocation with tie-breaking, and efficient tracking of
# room availability and usage counts using appropriate
# data structures.
#
# FOLLOW-UPS (answered below, in this order):
# 1. What if meetings arrive online and cannot be
# reordered?
```

```python
# 2. How do you report the busiest room by total busy time
# instead of meeting count?
# 3. Rank rooms by total busy time instead of the number
# of meetings.
#
# INTERVIEW NOTES
# Simulate in start order with two questions per meeting:
# which rooms are free now (lowest index first), and if
# none, which room frees up first (the meeting is delayed
# but keeps its length). Two heaps answer both quickly:
# one of free room ids, one of (end time, room id) for
# busy rooms.
#
# BRUTE FORCE (say it first): Scan all rooms for every
# meeting
# Keep the end time of every room in an array. For each
# meeting in start order, take the lowest-index room that
# has finished by the start; if none, take the room with
# the earliest end (lowest index on ties) and delay the
# meeting. Correct and the oracle; each meeting scans all
# N rooms. Time O(M * N). Space O(N).


import heapq


# BRUTE FORCE (say it first): Scan all rooms for every
# meeting | time O(M * N)
def most_booked_scan(n, meetings):
    end, used = [0] * n, [0] * n
    for start, finish in sorted(meetings):
        free = [r for r in range(n) if end[r] <= start]
        if free:
            r = free[0]
            end[r] = finish
        else:
            r = min(range(n), key=lambda x: (end[x], x))
            end[r] += finish - start
        used[r] += 1
    return used.index(max(used))


# BASE QUESTION (optimal): Two heaps: free rooms and busy
# rooms
# Idea: Process meetings in start order. Move rooms that
# finished by the meeting's start into the free heap
# (lowest index first). If a room is free, use it.
# FOLLOW-UP 3: Rank rooms by total busy time instead of
# the number of meetings.
# Answer: Add each meeting's duration (its original
# length, not the delay) to the room's tally; the choice
# of room is unchanged.
def most_booked_room(n, meetings, by="count"):
    # a meeting takes the lowest-numbered free room; if
    # none is
    # free it waits for the earliest room and keeps its
    # length
    free = list(range(n))
    heapq.heapify(free)
    # busy rooms: (time it becomes free, room)
    busy = []
    used = [0] * n
    for start, end in sorted(meetings):
        # rooms that finished by now are free again
        while busy and busy[0][0] <= start:
            _, room = heapq.heappop(busy)
            heapq.heappush(free, room)
        if free:
            room = heapq.heappop(free)
            heapq.heappush(busy, (end, room))
        else:
            # delayed: starts when the earliest room frees
            # up
            finish, room = heapq.heappop(busy)
            delayed_end = finish + (end - start)
            heapq.heappush(busy, (delayed_end, room))
        # FOLLOW-UP 3: by="time" ranks by total busy time
        # count meetings, or total length (not the delay)
        used[room] += 1 if by == "count" else end - start
    # ties go to the lowest room number
    return used.index(max(used))


if __name__ == "__main__":
    assert most_booked_room(2, [[0, 10], [1, 5], [2, 7],
                                [3, 4]]) == 0
    assert most_booked_room(3, [[1, 20], [2, 10], [3, 5],
                                [4, 9], [6, 8]]) == 1
    # a single room takes everything
    assert most_booked_room(1, [[0, 1], [0, 1]]) == 0
    # by busy time instead of the number of meetings
    meetings = [[0, 100], [1, 2], [2, 3], [3, 4]]
    assert most_booked_room(2, meetings, "count") == 1
    assert most_booked_room(2, meetings, "time") == 0
```

FULL QUESTION TEXT FROM THE SITE

Problem

You are given n meeting rooms labeled 0..n-1 and a list of meetings meetings, where each meeting is [start, end] with start < end. Meetings are processed in increasing order of start time.

Assign each meeting to a room using these rules:

If one or more rooms are free at start , assign the meeting to the free room with the smallest index .
If no rooms are free at start , the meeting must wait until a room becomes free. It is then scheduled in the room that becomes available earliest (break ties by smallest room index). If it waits, its duration stays the same , so if the original duration is end - start and the room becomes free at time t , the meeting runs on [t, t + (end - start)] .

Each time a meeting is assigned to a room (even after waiting), that room’s meeting count increases by 1.

Task

Return the index of the room that hosted the most meetings. If there is a tie, return the smallest index.

Input/Output
Input: n (int), meetings (list of [start,end])
Output: an integer room index
Constraints (reasonable interview assumptions)
1 <= n <= 10^5
1 <= len(meetings) <= 10^5
Times fit in 32-bit signed integer; waiting can increase end times beyond original but still fits in 64-bit.

ALSO REPORTED · 1 more

Assign meetings to rooms with delays | PracHub | Technical Screen

https://prachub.com/interview-questions/assign-meetings-to-rooms-with-delays

STUDY HERE: topic 1, Heaps, Stacks and Queues

MEDIUM | IN APP LC 253 | PracHub | Technical Screen | 22 of 33

★ TOP 75: rank 58 in the Medium and Hard priority list

★ Compute servers needed for daily recurring jobs

POSTED 2025-11-05 | 349 DAYS AGO | OLDER

STUDY: TYPE ALL TIME 58 30 DAYS 0 PRIORITY 41/228

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #253 Meeting Rooms II

LeetCode match: LC 253 Meeting Rooms II

Posted on PracHub: 2025-11-05 (updated 2026-06-25)

Original: https://prachub.com/coding-questions/compute-servers-needed-for-daily-recurring-jobs

QUESTION

You operate a cluster of identical servers that run recurring **daily** jobs. For a single day, you are given a list of job execution intervals. Each interval has:

- **`job_id`** (string or integer),
- **`start_time`** (timestamp within that day),
- **`end_time`** (timestamp within that day), with **`start_time < end_time`**. Interpret each interval as a half-open time range **`[start_time, end_time)`** during which that job must be running on some server **on that day**. Rules and constraints:
- Each physical server can execute at most **one job at a time**.
- However, if multiple intervals in the input correspond to the **same `job_id`**, they may overlap in time **on the same server** (they are considered parts or retries of the same job and do not consume extra capacity).
- If a job's true execution would extend past midnight into the next day, it is conceptually **truncated at the end of the current day** for scheduling purposes; for each day you only see the portion that falls within that day's 24-hour window.

Tasks

1. Given all job intervals for a single day (potentially with repeated **`job_id`**s and overlapping intervals), compute the **minimum number of servers** required to schedule all the jobs under the above rules.
2. Describe the time and space complexity of your algorithm in terms of the number of intervals (m).

Follow-up

QUESTION

Some rare jobs are extremely long-running and conceptually span **multiple days** without restart. For such a multi-day job, you decide to reserve a **dedicated server** that is not shared with any other jobs, but which may still handle different intervals of that same **`job_id`**. Explain how you would extend or adapt your solution to:

- Detect which jobs require a dedicated server based on their multi-day duration or configuration.
- Account for these dedicated servers when computing the total number of servers needed for the system.

Part 1: Minimum Servers for One Day with Overlapping Same-Job IntervalsYou are given all job execution intervals for a single day. Each interval is half-open, meaning [start_time, end_time), and represents a time range during which a job must be running. A server can run at most one job at a time. However, if multiple intervals have the same job_id, they are considered parts or retries of the same job, so overlapping intervals with the same job_id consume only one server total during the overlap. Return the minimum number of servers required to run all intervals for the day.
Constraints
0 <= len(intervals) <= 2000000 <= job_id <= 10^90 <= start_time < end_time <= 1440Intervals are half-open: an interval ending at time t does not overlap an interval starting at time t.Multiple intervals may have the same job_id.

EXAMPLES

```
Input: ([],)
Expected Output: 0
```

Explanation: There are no jobs, so no servers are needed.

```
Input: ([[5, 100, 200], [5, 150, 250], [5, 240,
    300]],)
Expected Output: 1
```

Explanation: All intervals belong to the same job_id. Even though they overlap, they can share one server.

Hints

At any moment, one server is needed for each distinct active job_id, not for each active interval.Try merging all intervals belonging to the same job_id first, then run a sweep-line algorithm over the merged intervals.
Part 2: Total Servers with Dedicated Multi-Day JobsYou are given true job execution intervals that may span multiple days. Some jobs require dedicated servers. A job is dedicated if its job_id is listed in configured_dedicated, or if any one of its true intervals has duration at least min_dedicated_duration minutes. A dedicated job uses one reserved server for that job_id and does not share that server with other jobs, though multiple intervals of the same dedicated job_id may use the same dedicated server. For a given target_day, compute the total number of

APPROACH

Max concurrent jobs = servers needed; sweep line over start/end events, or min-heap of end times. Handle intervals wrapping midnight by splitting.

HOW TO THINK ABOUT IT

The number of servers is the largest number of jobs running at the same time. First remove the twist: overlapping intervals of the same job are really one longer interval, so merge the intervals of each job_id. Then it is the plain maximum overlap: sort start and end events and keep a running count.

Basis: LeetCode 253 (Meeting Rooms II) with a twist from the statement: intervals of the same job_id may overlap on one server and count once. Intervals are half-open [start, end) within one day.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def servers_needed(intervals):
    best = 0
    for t in range(DAY):
        jobs = set()
        for job_id, s, e in intervals:
            if s <= t < e:
                jobs.add(job_id)
        best = max(best, len(jobs))
    return best
```

TIME	SPACE
O(T * m)	O(m)

WHY IT IS SLOW

Depends on the length of the day, O(T * m). Merge each job's own intervals first, then sort start and end events and keep a running count, O(m log m).

OPTIMAL SOLUTION · Merge each job's intervals, then sweep start and end events

TIME	SPACE
O(m log m)	O(m)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Some rare jobs run for several days without restart and get a dedicated server.

Remove those jobs from the daily intervals and add one server per such job: servers = (number of dedicated jobs) + (maximum overlap of the remaining jobs).

FOLLOW-UP 2 Jobs that cross midnight.

Truncate at the end of the day for each day's schedule (as the statement says). For a recurring daily job that wraps, split it into two pieces (the end of the day and the start of the day); the sweep then handles both on the same circular day.

FOLLOW-UP 3 Return which server each job runs on.

Sort merged intervals by start and keep a min-heap of (end time, server id): reuse the server that frees up first if it is free, otherwise open a new one. The number of servers opened equals the maximum overlap.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Compute servers needed for daily
# recurring jobs
#
# You operate a cluster of identical servers that run
# recurring **daily** jobs. For a single day, you are
# given a list of job execution intervals. Each interval
# has: - job_id (string or integer), - start_time
# (timestamp within that day), - end_time (timestamp
# within that day), with start_time < end_time. Interpret
# each interval as a half-open time range [start_time,
# end_time) during which that job must be running on some
# server **on that day**. Rules and constraints: - Each
# physical server can execute at most **one job at a
# time**. - However, if multiple intervals in the input
# correspond to the **same job_id**, they may overlap in
# time **on the same server** (they are considered parts
# or retries of the same job and do not consume extra
# capacity).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Some rare jobs run for several days without restart
# and get a dedicated server.
# 2. Jobs that cross midnight.
# 3. Return which server each job runs on.
#
# INTERVIEW NOTES
# The number of servers is the largest number of jobs
# running at the same time. First remove the twist:
# overlapping intervals of the same job are really one
# longer interval, so merge the intervals of each job_id.
# Then it is the plain maximum overlap: sort start and end
# events and keep a running count.
#
# BRUTE FORCE (say it first): For every time unit count
# the distinct jobs running
# For each integer time t collect the set of job ids with
# an interval covering t and take the largest set size.
# Correct and the oracle; it depends on the length of the
# day, not only on m. Time O(T * m). Space O(m).


import heapq
from collections import defaultdict
```

```
# BRUTE FORCE (say it first): For every time unit count
# the distinct jobs running | time O(T * m)
def servers_by_time(intervals):
```

```
    if not intervals:
        return 0
    lo, hi = min(s for _, s, _ in intervals), max(
        e for _, _, e in intervals
    )
    return max(
        len({j for j, s, e in intervals if s <= t < e})
        for t in range(lo, hi)
    )


# HELPERS for the base question
def merge_by_job(intervals):
    # intervals of the SAME job may overlap on one server,
    # so
    # merge them first: (job, start, end) -> disjoint
    # blocks
    by_job = defaultdict(list)
    for job, start, end in intervals:
        by_job[job].append((start, end))
    blocks = []
    for job, spans in by_job.items():
        cur = None
        for s, e in sorted(spans):
            if cur and s <= cur[1]:
                cur[1] = max(cur[1], e)
            else:
                if cur:
                    blocks.append((cur[0], cur[1], job))
                cur = [s, e]
        blocks.append((cur[0], cur[1], job))
    return blocks


# BASE QUESTION (optimal): Merge each job's intervals,
# then sweep start and end events
# Idea: Group the intervals by job_id, sort and merge each
# group's overlapping or touching intervals, then collect
# all merged intervals.
# FOLLOW-UP 1: Some rare jobs run for several days without
# restart and get a dedicated server.
# Answer: Remove those jobs from the daily intervals and
```

```
# add one server per such job: servers = (number of
# dedicated jobs) + (maximum overlap of the remaining
# jobs).
def servers_needed(intervals, dedicated=()):
    # FOLLOW-UP 1: rare multi-day jobs get their own
    # server
    # dedicated jobs (they span several days) get their
    # own
    # server; the rest share servers: the answer is the
    # largest number of blocks running at the same moment
    dedicated = set(dedicated)
    shared = [i for i in intervals
              if i[0] not in dedicated]
    events = []
    for start, end, _ in merge_by_job(shared):
        events.append((start, 1))
        # [start, end): at equal times an end comes first
        events.append((end, -1))
    run = best = 0
    for _, delta in sorted(events):
        run += delta
        best = max(best, run)
    used = {i[0] for i in intervals if i[0] in dedicated}
    return best + len(used)


# FOLLOW-UP 2: Jobs that cross midnight.
# Answer: Truncate at the end of the day for each day's
# schedule (as the statement says).
def split_over_midnight(start, end, day=24 * 60):
    # a daily job that wraps past midnight: two pieces
    if start < end:
        return [(start, end)]
    return [(start, day), (0, end)]


# FOLLOW-UP 3: Return which server each job runs on.
# Answer: Sort merged intervals by start and keep a min-
# heap of (end time, server id): reuse the server that
# frees up first if it is free, otherwise open a new one.
# The number of servers opened equals the maximum overlap.
def assign_servers(intervals):
    # which server runs each block of each job?
    heap, plan, opened = [], [], 0
    blocks = sorted(merge_by_job(intervals))
```

```
    for start, end, job in blocks:
        if heap and heap[0][0] <= start:
            _, server = heapq.heappop(heap)
        else:
            server, opened = opened, opened + 1
        plan.append((job, server))
        heapq.heappush(heap, (end, server))
    return plan, opened


if __name__ == "__main__":
    jobs = [("a", 0, 10), ("a", 5, 12), ("b", 3, 8),
            ("c", 10, 15)]
    # job a is one block [0, 12); b overlaps it; c starts
    # at 10
    assert servers_needed(jobs) == 2
    # touching half-open intervals share a server
    assert servers_needed([("a", 0, 5), ("b", 5, 9)]) == 1
    assert servers_needed([]) == 0
    # job b runs for days: it gets a dedicated server
    assert servers_needed(jobs, dedicated=["b"]) == 3
    assert servers_needed(jobs, dedicated=["a", "b"]) == 3
    plan, count = assign_servers(jobs)
    assert count == 2 and len(plan) == 3
    assert split_over_midnight(1380, 60) == [(1380, 1440),
                                             (0, 60)]
```

FUNCTION AND TEST CASES

Signature

```
def solution(intervals: list[list[int]]) -> int:
```

Sample call

```
solution([])
```

Test cases

Input	Expected
`([],)`	`0`
`([[5, 100, 200], [5, 150, 250], [5, 240, 300]],)`	`1`

ALSO REPORTED · 1 more

Compute minimum servers for cyclic tasks | PracHub | Technical Screen

https://prachub.com/coding-questions/compute-minimum-servers-for-cyclic-tasks

STUDY HERE: topic 1, Heaps, Stacks and Queues

MEDIUM | PracHub | Technical Screen | 23 of 33

★ TOP 75: rank 65 in the Medium and Hard priority list

★ Compute shortest paths with blocked nodes

POSTED 2025-09-06 | 409 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 37.9 | 30 DAYS 33 | PRIORITY 60/228

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #743 Network Delay Time

LeetCode match: LC 743 Network Delay Time (Dijkstra with blocked nodes skipped)

Posted on PracHub: 2025-09-06 (updated 2026-06-27)

Original: https://prachub.com/coding-questions/compute-shortest-paths-with-blocked-nodes

QUESTION

Given a graph with nodes and edges and a designated source node s, compute the shortest distance from s to every other node. Some nodes are inaccessible (blocked) and cannot be traversed or reached; treat them as removed from the graph. Return a distance for each node where unreachable nodes (including blocked ones) are -1. Describe your algorithm, data structures, and time/space complexity, and note any differences for directed vs. undirected graphs.

Given n nodes labeled 0..n-1, edges, a source, and blocked nodes, return shortest unweighted distances from the source. Blocked and unreachable nodes must be -1. Set directed=True to treat edges as directed.

Constraints

Nodes are labeled 0 through n-1Edges are unweighted

EXAMPLES

```
Input: (5, [[0, 1], [1, 2], [0, 3], [3, 4]], 0, [3],
    False)
Expected Output: [0, 1, 2, -1, -1]
Input: (4, [[0, 1], [1, 2], [2, 3]], 0, [], True)
Expected Output: [0, 1, 2, 3]
```

Hints

Treat blocked nodes as removed before BFS.For directed graphs, only add the given edge direction.

APPROACH

Run Dijkstra (BFS if unweighted) from s, never relaxing into blocked nodes; blocked/unreachable nodes stay at infinity. O((V+E) log V).

HOW TO THINK ABOUT IT

Blocked nodes are simply not in the graph, so skip them when you queue neighbours and never start from one. After that it is plain BFS from the source: first discovery gives the shortest distance. State the edge cases: a blocked source makes everything unreachable, and directed edges must only be added in their given direction.

Basis: Unweighted shortest distances (BFS) with some nodes removed from the graph. Output: a distance per node, -1 for unreachable or blocked nodes; directed=True uses edges one way only.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(n, edges, blocked, s):
    keep = []
    for u, v, w in edges:
        if u not in blocked and v not in blocked:
            keep.append((u, v, w))
    d = floyd_warshall(n, keep)  # all pairs
    return d[s]
```

TIME	SPACE
O(V^3)	O(V^2)

WHY IT IS SLOW

All-pairs distances cost O(V^3) and O(V^2) memory when one source is needed. A BFS from the source that never enters a blocked node is O(V + E).

OPTIMAL SOLUTION · BFS that never enters a blocked node

TIME	SPACE
O(V + E)	O(V + E)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The edges have weights.

BFS is no longer correct: use Dijkstra with a heap and skip blocked nodes the same way. O((V + E) log V). Directed and undirected differ only in whether you add the reverse edge.

FOLLOW-UP 2 Directed versus undirected.

For an undirected graph add each edge in both directions; for a directed one only as given. A node may be reachable in one case and not in the other, so state which you assume.

FOLLOW-UP 3 Nodes get blocked over time and you must keep distances up to date.

Decremental shortest paths is hard in general; for few updates rerun BFS, and for many offline queries process the changes in reverse order (unblocking only improves distances).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Compute shortest paths with blocked nodes
#
# Given a graph with nodes and edges and a designated
# source node s, compute the shortest distance from s to
# every other node. Some nodes are inaccessible (blocked)
# and cannot be traversed or reached; treat them as
# removed from the graph. Return a distance for each node
# where unreachable nodes (including blocked ones) are -1.
# Describe your algorithm, data structures, and time/space
# complexity, and note any differences for directed vs.
# undirected graphs. Overview: This question evaluates
# proficiency with graph algorithms, shortest-path and
# reachability concepts when nodes are removed or blocked,
# and the ability to analyze time and space complexity
# across directed and undirected graphs.Given n nodes
# labeled 0..n-1, edges, a source, and blocked nodes,
# return shortest unweighted distances from the source.
# Blocked and unreachable nodes must be -1. Set
# directed=True to treat edges as directed.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The edges have weights.
# 2. Directed versus undirected.
# 3. Nodes get blocked over time and you must keep
# distances up to date.
#
# INTERVIEW NOTES
# Blocked nodes are simply not in the graph, so skip them
# when you queue neighbours and never start from one.
# After that it is plain BFS from the source: first
# discovery gives the shortest distance. State the edge
# cases: a blocked source makes everything unreachable,
# and directed edges must only be added in their given
# direction.
#
# BRUTE FORCE (say it first): Floyd-Warshall on the graph
# without the blocked nodes
# Compute all-pairs distances ignoring blocked nodes and
# read the row of the source. Correct and the oracle;
# cubic in the number of nodes. Time O(V^3). Space O(V^2).


import heapq
from collections import defaultdict, deque
```

```
# HELPERS for the brute force
INF = float("inf")


# BRUTE FORCE (say it first): Floyd-Warshall on the graph
# without the blocked nodes | time O(V^3)
def blocked_distances_floyd(
    n, edges, source, blocked, directed
):
    INF = float("inf")
    bl = set(blocked)
    d = [
        [0 if i == j else INF for j in range(n)]
        for i in range(n)
    ]
    for u, v in edges:
        if u in bl or v in bl:
            continue
        d[u][v] = min(d[u][v], 1)
        if not directed:
            d[v][u] = min(d[v][u], 1)
    for k in range(n):
        for i in range(n):
            for j in range(n):
                if d[i][k] + d[k][j] < d[i][j]:
                    d[i][j] = d[i][k] + d[k][j]
    if source in bl:
        return [-1] * n
    return [
        -1 if x == INF or i in bl else x
        for i, x in enumerate(d[source])
    ]


# BASE QUESTION (optimal): BFS that never enters a blocked
# node
# Idea: Build adjacency lists (one direction only if
# directed). If the source is blocked return all -1.
def blocked_distances(n, edges, source, blocked,
                      directed=False):
    # BFS that treats blocked nodes as removed;
    # unreachable
    # and blocked nodes get -1
    bad = set(blocked)
    if source in bad:
        return [-1] * n
    adj = defaultdict(list)
    for u, v in edges:
        adj[u].append(v)
        if not directed:
            adj[v].append(u)
    dist = [-1] * n
    dist[source] = 0
    queue = deque([source])
    while queue:
        u = queue.popleft()
        for v in adj[u]:
            if v not in bad and dist[v] == -1:
                dist[v] = dist[u] + 1
                queue.append(v)
    return dist


# FOLLOW-UP 1: The edges have weights.
# Answer: BFS is no longer correct: use Dijkstra with a
# heap and skip blocked nodes the same way. O((V + E) log
# V). Directed and undirected differ only in whether you
# add the reverse edge.
def blocked_distances_weighted(n, edges, source, blocked,
                               directed=False):
    # edges (u, v, w): BFS is wrong now, use Dijkstra
    bad = set(blocked)
    if source in bad:
        return [-1] * n
    adj = defaultdict(list)
    for u, v, w in edges:
        adj[u].append((v, w))
        if not directed:
            adj[v].append((u, w))
    dist = [INF] * n
    dist[source] = 0
    heap = [(0, source)]
    while heap:
```

```
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        for v, w in adj[u]:
            if v not in bad and d + w < dist[v]:
                dist[v] = d + w
                heapq.heappush(heap, (d + w, v))
    return [-1 if d == INF else d for d in dist]


if __name__ == "__main__":
    edges = [(0, 1), (1, 2), (0, 3), (3, 2), (2, 4)]
    free = blocked_distances(5, edges, 0, [])
    assert free == [0, 1, 2, 1, 3]
    # blocking 1 forces the route through 3
    assert blocked_distances(5, edges, 0, [1])[2] == 2
    # blocking 2 cuts node 4 off; blocked nodes are -1
    cut = blocked_distances(5, edges, 0, [2])
    assert cut == [0, 1, -1, 1, -1]
    start = blocked_distances(3, edges[:2], 0, [0])
    assert start == [-1, -1, -1]
    # directed edges only go one way
    path = [(0, 1), (1, 2)]
    one_way = blocked_distances(3, path, 2, [], True)
    assert one_way == [-1, -1, 0]
    weighted = [(0, 1, 5), (1, 2, 1), (0, 2, 10)]
    wd = blocked_distances_weighted
    assert wd(3, weighted, 0, []) == [0, 5, 6]
    assert wd(3, weighted, 0, [1])[2] == 10
```

STUDY HERE: topic 1, Heaps, Stacks and Queues

MEDIUM | HackTheRounds | Phone Screen | 24 of 33

★ TOP 75: rank 59 in the Medium and Hard priority list

★ Meeting Rooms III Variant

STUDY: TYPE | ALL TIME 58 | 30 DAYS 0 | PRIORITY 39/228

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #253 Meeting Rooms II

LeetCode match: LC 253 (cyclic variant)

QUESTION

Given an array of meeting time intervals where each interval is [start, end], determine the minimum number of conference rooms required to hold all meetings without overlap. Variant: times are cyclic within a 24-hour period (meetings can wrap around midnight).

EXAMPLES

Example 1	
Input	`intervals = [[0, 30], [5, 10], [15, 20]]`
Output	`2`
Why	`Meeting [0,30] overlaps with [5,10] and [15,20], so we need 2 rooms.`

Example 2	
Input	`intervals = [[7, 10], [2, 4]]`
Output	`1`
Why	`No overlap, 1 room is sufficient.`

Example 3	
Input	`intervals = [[22, 2], [23, 1], [0, 3]]`
Output	`3`
Why	`All three meetings overlap around midnight.`

CONSTRAINTS

- 1 <= intervals.length <= 10^4
- 0 <= start < end <= 24 * 60 (minutes in a day)
- Meetings may wrap around (e.g., [23:00, 01:00])

APPROACH

Sweep line; split intervals that wrap past midnight

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the 24-hour day repeats cyclically, whether a meeting that wraps midnight must stay in one room the whole time, and whether a meeting ending exactly when another starts conflicts (usually not).

BRUTE FORCE TO SAY OUT LOUD

Try assigning meetings to rooms one by one with backtracking. Exponential.

OPTIMAL IDEA

Split a wrapping meeting [s, e] with e < s into [s, 24h] and [0, e] and run a sweep line over the events: sort start and end events, +1 at a start, -1 at an end (process ends first at equal times), and take the maximum running count. That gives the minimum number of rooms when pieces may use different rooms. If a wrapping meeting must stay in a single room the exact answer is circular-arc coloring, which can be larger than the maximum overlap (five arcs forming an odd cycle need three rooms with overlap two) and is hard in general, so state the assumption and mention a heuristic.

DRY-RUN THESE CASES

No wrap, one wrapping meeting, a wrap that overlaps the beginning of the day, back-to-back meetings, all meetings the same.

LIKELY FOLLOW-UPS

Return which room each meeting uses, or check feasibility with a given number of rooms.

PITFALLS

Counting back-to-back meetings as overlapping. Splitting a wrap but treating the two pieces as two meetings in the output.

HOW TO THINK ABOUT IT

Split a wrapped meeting into two pieces (from start to the end of the day, and from midnight to its end) and run the normal sweep line: the largest number of meetings active at once is the number of rooms needed. That is exact when pieces may sit in different rooms; if a meeting must keep one room, the arcs form a circular-arc graph and the true minimum can exceed the maximum overlap (five overlapping arcs around a circle need 3 rooms with overlap 2). Say which reading you assume.

Basis: LeetCode 253 (Meeting Rooms II) on a circular day. Meetings are half-open [start, end) in minutes; a meeting with end < start wraps past midnight. Two readings: the usual interview answer (split the wrapped meeting and take the maximum overlap) and the exact answer when a wrapped meeting must keep one room.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def rooms_needed(meetings):
    slots = [minutes_used(m) for m in meetings]
    for k in range(1, len(meetings) + 1):
        rooms = [set() for _ in range(k)]
        if assign(0, rooms, slots):
            return k
def assign(i, rooms, slots):
    if i == len(slots):
        return True
    for room in rooms:
        busy = slots[i]
        if not room & busy:  # no shared minute
            room |= busy
            if assign(i + 1, rooms, slots):
                return True
            room -= busy
    return False
```

TIME	SPACE
exponential	O(n)

WHY IT IS SLOW

Backtracking over room assignments is exponential. Splitting wrapped meetings and sweeping start and end events gives the maximum overlap in O(n log n); that equals the exact answer when meetings do not wrap.

OPTIMAL SOLUTION · Split wrapped meetings and sweep (maximum overlap)

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 A wrapped meeting must stay in a single room.

Then it is circular-arc colouring, NP-hard in general. A common practical method: cut the circle at the minute with the fewest active meetings; the meetings crossing the cut use fixed rooms, and the rest are scheduled greedily with a heap. It can use more rooms than the maximum overlap, and the five-arc example shows why.

FOLLOW-UP 2 Return which room each meeting uses.

Sort by start and keep a min-heap of (end time, room id); reuse the earliest-ending room if it is free, otherwise open a new one. For wrapped meetings schedule them first so their rooms are fixed.

FOLLOW-UP 3 Days can repeat (a wrapped meeting recurs every day).

A recurring meeting occupies its slots every day, so the circle model above is exactly right, and the no-wrap shortcut of ordering by start time is not.

FOLLOW-UP 4 Return which room each meeting uses.

Sort by start and keep a heap of (end time, room): reuse the earliest-ending room if it is free, otherwise open a new one. For meetings that do not wrap this uses exactly the minimum number of rooms (verified against the sweep); wrapped meetings need the cut-the-circle method.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Meeting Rooms III Variant
#
# Given an array of meeting time intervals where each
# interval is [start, end], determine the minimum number
# of conference rooms required to hold all meetings
# without overlap. Variant: times are cyclic within a
# 24-hour period (meetings can wrap around midnight).
#
# FOLLOW-UPS (answered below, in this order):
# 1. A wrapped meeting must stay in a single room.
# 2. Return which room each meeting uses.
# 3. Days can repeat (a wrapped meeting recurs every day).
# 4. Return which room each meeting uses.
#
# INTERVIEW NOTES
# Split a wrapped meeting into two pieces (from start to
# the end of the day, and from midnight to its end) and
# run the normal sweep line: the largest number of
# meetings active at once is the number of rooms needed.
# That is exact when pieces may sit in different rooms; if
# a meeting must keep one room, the arcs form a circular-
# arc graph and the true minimum can exceed the maximum
# overlap (five overlapping arcs around a circle need 3
# rooms with overlap 2). Say which reading you assume.
#
# BRUTE FORCE (say it first): Exact minimum with one room
# per meeting, by backtracking
# Treat each meeting as the set of minute slots it
# occupies (a wrapped meeting is two ranges of slots) and
# try to colour the meetings with k rooms for k = 1, 2,
# ...; two meetings conflict when their slot sets
# intersect. Exact for 'a meeting keeps its room', and the
# oracle for small cases; it is exponential. Time
# exponential. Space O(n).


import heapq


# HELPERS for the brute force
DAY = 24


def hours(meeting, day=DAY):
    # the hours a meeting occupies on the 24 hour circle
```

```
    s, e = meeting
    length = (e - s) % day or day
    return {(s + k) % day for k in range(length)}


def brute_min_rooms(meetings, day=DAY):
    return max((sum(1 for m in meetings
                    if h in hours(m, day))
                for h in range(day)), default=0)


# HELPERS for the base question
def pieces(meetings, day=DAY):
    # a meeting whose end is before its start wraps past
    # midnight: split it into the part before midnight and
    # the part after it
    for s, e in meetings:
        if e < s:
            yield s, day
            yield 0, e
        else:
            yield s, e


# BASE QUESTION (optimal): Split wrapped meetings and
# sweep (maximum overlap)
# Idea: Turn every meeting into one or two ordinary
# intervals, create +1 events at starts and -1 events at
# ends, sort them (ends before starts at the same minute,
# because intervals are half-open), and take the maximum
# running count.
def min_rooms_sweep(meetings, day=DAY):
    # sweep the time line: +1 at a start, -1 at an end. At
    # equal times an end comes first (a meeting ending at
    # 5
    # and one starting at 5 can share a room).
    events = []
    for s, e in pieces(meetings, day):
        events.append((s, 1))
        events.append((e, -1))
    events.sort()
    cur = best = 0
    for _, step in events:
        cur += step
        best = max(best, cur)
```

```
    return best


# FOLLOW-UP 1: A wrapped meeting must stay in a single
# room.
# Answer: Then it is circular-arc colouring, NP-hard in
# general.
def assign_single_rooms(meetings, day=DAY):
    # follow-up: a meeting that wraps must stay in ONE
    # room.
    # That is circular-arc colouring (NP-hard), so use the
    # practical method: cut the circle at the hour where
    # the
    # fewest meetings are running, give those meetings
    # their
    # own rooms, and place the rest greedily.
    busy = [sum(1 for m in meetings
                if cut in hours(m, day))
            for cut in range(day)]
    cut = busy.index(min(busy))
    # rotate so that the cut is time 0: (start, end) on a
    # line
    line = []
    for s, e in meetings:
        start = (s - cut) % day
        length = (e - s) % day or day
        line.append((start, start + length))
    free_at, deadline = [], []     # per room
```

```
    room_of = {}
    for i, (s, e) in enumerate(line):
        if e > day:
            # crosses the cut: busy until e - day, and
            # again
            # from s on, so the room is blocked after s
            room_of[i] = len(free_at)
            free_at.append(e - day)
            deadline.append(s)
    rest = [i for i in range(len(line))
            if i not in room_of]
    others = sorted(rest, key=lambda i: line[i])
    for i in others:
        s, e = line[i]
        # the room that has been free the longest and fits
        fits = [r for r in range(len(free_at))
                if free_at[r] <= s and e <= deadline[r]]
```

```
        if fits:
            room = max(fits, key=lambda r: free_at[r])
        else:
            room = len(free_at)
            free_at.append(0)
            deadline.append(day)
        room_of[i] = room
        free_at[room] = e
    return [room_of[i] for i in range(len(meetings))]


# FOLLOW-UP 2: Return which room each meeting uses.
# Answer: Sort by start and keep a min-heap of (end time,
# room id); reuse the earliest-ending room if it is free,
# otherwise open a new one. For wrapped meetings schedule
# them first so their rooms are fixed.
# FOLLOW-UP 4: Return which room each meeting uses.
# Answer: Sort by start and keep a heap of (end time,
# room): reuse the earliest-ending room if it is free,
# otherwise open a new one.
def assign_rooms(meetings):
    # follow-up: which room for each meeting (no
    # wrapping).
    # Sort by start; a heap of (end, room) gives the room
    # that
    # frees up first; reuse it if it is free, else open
    # one.
    order = sorted(range(len(meetings)),
                   key=lambda i: meetings[i][0])
    heap, room_of, rooms = [], {}, 0
    for i in order:
        s, e = meetings[i]
        if heap and heap[0][0] <= s:
            _, room = heapq.heappop(heap)
        else:
            room, rooms = rooms, rooms + 1
        room_of[i] = room
        heapq.heappush(heap, (e, room))
    return [room_of[i] for i in range(len(meetings))]


# HELPERS for the checks below
def valid_assignment(meetings, rooms, day=DAY):
    # check that no room holds two meetings at the same
    # hour
```

```
    used = set()
    for m, room in zip(meetings, rooms):
        for h in hours(m, day):
            if (room, h) in used:
                return False
            used.add((room, h))
    return True
```

```
if __name__ == "__main__":
    long = [[0, 30], [5, 10], [15, 20]]
    assert min_rooms_sweep(long) == 2
    assert min_rooms_sweep([[7, 10], [2, 4]]) == 1
    night = [[22, 2], [23, 1], [0, 3]]
    assert min_rooms_sweep(night) == 3
    assert min_rooms_sweep([]) == 0
    # one ends exactly when the next starts: same room
    assert min_rooms_sweep([[1, 5], [5, 9]]) == 1
    flat = [[0, 30], [5, 10], [15, 20]]
    rooms = assign_rooms(flat)
    assert rooms == [0, 1, 1]
    assert len(set(rooms)) == min_rooms_sweep(flat)
    two = [(22, 2), (1, 5)]
    assert valid_assignment(two, assign_single_rooms(two))
    import random
    for _ in range(400):
        ms = []
        for _ in range(random.randint(0, 7)):
            s, e = random.sample(range(DAY), 2)
            ms.append((s, e))
        assert min_rooms_sweep(ms) == brute_min_rooms(ms)
        rooms = assign_single_rooms(ms)
        assert valid_assignment(ms, rooms)
        assert len(set(rooms)) >= brute_min_rooms(ms)
        if all(s < e for s, e in ms):
            plain = assign_rooms(ms)
            assert valid_assignment(ms, plain)
            assert len(set(plain)) == brute_min_rooms(ms)
```

STUDY HERE: topic 1, Heaps, Stacks and Queues

MEDIUM | HackTheRounds | Phone Screen | 25 of 33

Playlist Shuffle No Adjacent Artists

STUDY: TALK | ALL TIME 34.9 | 30 DAYS 0 | PRIORITY 110/228

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #767 Reorganize String

LeetCode match: LC 767 / LC 1054

QUESTION

Given a playlist of songs where each song has an artist, implement a shuffle function that minimizes (or eliminates) adjacent songs by the same artist.

EXAMPLES

Example 1	
Input	`[("A", "song1"), ("A", "song2"), ("B", "song3"), ("B", "song4"), ("C", "song5")]`
Output	`[("A", "song1"), ("B", "song3"), ("A", "song2"), ("B", "song4"), ("C", "song5")]`
Why	`No adjacent songs have same artist.`

CONSTRAINTS

- 1 <= songs.length <= 10^4

APPROACH

Max-heap by remaining count per artist

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the shuffle must be random, how to behave when a perfect arrangement is impossible (the most frequent artist has more than half the songs, rounded up), and whether the result must be uniform among valid shuffles.

BRUTE FORCE TO SAY OUT LOUD

Shuffle randomly, then check for adjacent artists and retry. May never finish.

OPTIMAL IDEA

Feasibility: let n be the song count; an arrangement exists if the largest artist group has at most ceil(n / 2) songs. Build it greedily with a max-heap of remaining songs per artist: repeatedly take from the artist with the most remaining songs who is not the previous artist, using random choice among songs of that artist (and optionally among equally big groups) to keep the result varied. If infeasible, place the leftovers at the end or spread them as evenly as possible, and say that adjacency is then unavoidable. O(n log A).

INTERVIEW-DAY PLAN

DRY-RUN THESE CASES

One artist (impossible), two artists with equal counts (alternating), the largest artist at exactly half rounded up, three songs with two from the same artist.

LIKELY FOLLOW-UPS

Uniform randomness among all valid shuffles (harder), or a minimum spacing of K songs between the same artist.

PITFALLS

Choosing the largest group even when it was just played. Assuming the greedy result is uniformly random.

HOW TO THINK ABOUT IT

A perfect arrangement exists exactly when the most frequent artist has at most ceil(n / 2) songs. Build it greedily: always play from the artist with the most songs left, except the artist who just played. A max-heap gives the most frequent artist; hold the previous artist back for one step. If it is impossible, the leftover songs of the dominant artist are the only unavoidable adjacencies.

Basis: LeetCode 767 (Reorganize String) and 1054 (Distant Barcodes) with songs and artists. Minimising adjacent same-artist pairs when a perfect arrangement does not exist.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(songs):
    best, best_pairs = None, INF
    for order in permutations(songs):
        pairs = 0
        for a, b in zip(order, order[1:]):
            if a.artist == b.artist:
                pairs += 1
        if pairs < best_pairs:
            best, best_pairs = order, pairs
    return best
```

TIME	SPACE
O(n! * n)	O(n)

WHY IT IS SLOW

n! orderings. A greedy that always plays the artist with the most songs left (not the one that just played), using a heap, is O(n log A).

OPTIMAL SOLUTION · Greedy with a max-heap, holding back the last artist

TIME	SPACE
O(n log A)	O(A)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The shuffle should look random, not always the same arrangement.

Break ties between equally frequent artists randomly and pick the song within the artist at random, as in the code. The result is valid but not uniform among all valid shuffles; a uniform sampler needs counting arrangements.

FOLLOW-UP 2 Songs by the same artist must be at least K apart.

Same idea with a waiting queue of length K: after playing an artist, hold them for K - 1 more songs before they are eligible again (the LeetCode 358 construction).

FOLLOW-UP 3 What is the minimum number of unavoidable repeats?

With n songs and a dominant artist with m songs, the other n - m songs separate at most n - m + 1 groups, so at least m - (n - m + 1) = 2m - n - 1 neighbouring pairs of that artist are forced (if positive).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Playlist Shuffle No Adjacent Artists
#
# Given a playlist of songs where each song has an artist,
# implement a shuffle function that minimizes (or
# eliminates) adjacent songs by the same artist.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The shuffle should look random, not always the same
# arrangement.
# 2. Songs by the same artist must be at least K apart.
# 3. What is the minimum number of unavoidable repeats?
#
# INTERVIEW NOTES
# A perfect arrangement exists exactly when the most
# frequent artist has at most ceil(n / 2) songs. Build it
# greedily: always play from the artist with the most
# songs left, except the artist who just played. A max-
# heap gives the most frequent artist; hold the previous
# artist back for one step. If it is impossible, the
# leftover songs of the dominant artist are the only
# unavoidable adjacencies.
#
# BRUTE FORCE (say it first): Try every ordering and keep
# the fewest adjacent pairs
# Enumerate all orderings of the artists' songs (as a
# multiset) and count neighbours with the same artist.
# Exact and the oracle; hopeless beyond about ten songs.
# Time O(n! * n). Space O(n).


import heapq
import random
from collections import deque
from itertools import permutations


# HELPERS for the brute force
def adjacent_pairs(playlist):
    pairs = zip(playlist, playlist[1:])
    return sum(1 for a, b in pairs if a[0] == b[0])


# BRUTE FORCE (say it first, it also checks the answers)
def brute_min_pairs(songs):
    # oracle: try every order (small playlists only)
    orders = permutations(songs)
    return min(adjacent_pairs(p) for p in orders)


# HELPERS for the base question
def release(waiting, heap):
    # the oldest waiting artist may play again (if it has
    # any song left)
    entry = waiting.popleft()
    if entry[0]:
        heapq.heappush(heap, entry)


def take_oldest(waiting, heap):
    # break the gap on purpose: the first waiting artist
    # that still has songs plays early
    while waiting:
        entry = waiting.popleft()
        if entry[0]:
            heapq.heappush(heap, entry)
            return


# BASE QUESTION (optimal): Greedy with a max-heap, holding
# back the last artist
# Idea: Keep a heap of (remaining songs, artist). Each
# step pop the artist with the most songs left; play one
# of their songs, and push them back only after the next
# artist has played (so they cannot repeat).
# FOLLOW-UP 1: The shuffle should look random, not always
# the same arrangement.
# Answer: Break ties between equally frequent artists
# randomly and pick the song within the artist at random,
# as in the code. The result is valid but not uniform
# among all valid shuffles; a uniform sampler needs
# counting arrangements.
# FOLLOW-UP 2: Songs by the same artist must be at least K
# apart.
# Answer: Same idea with a waiting queue of length K:
# after playing an artist, hold them for K - 1 more songs
# before they are eligible again (the LeetCode 358
# construction).
def shuffle_playlist(songs, gap=2, strict=False,
                     rng=random):
    # songs are (artist, title). The artist with the most
    # songs left plays next and then waits `gap - 1` other
    # songs before it may play again (gap 2 = never twice
    # in
    # a row). Ties between artists and the song picked
    # inside
    # an artist are random, so every run looks different.
    by_artist = {}
    for artist, title in songs:
        by_artist.setdefault(artist, []).append(title)
    for titles in by_artist.values():
        # FOLLOW-UP 1: random song order inside an artist
        rng.shuffle(titles)
    # FOLLOW-UP 1: random tie-break between artists
    heap = [(-len(t), rng.random(), a)
            for a, t in by_artist.items()]
    heapq.heapify(heap)
    waiting, out = deque(), []
    while len(out) < len(songs):
```

```
        if not heap:
            # everyone left is still waiting
            if strict:
                return None
            take_oldest(waiting, heap)
        left, _, artist = heapq.heappop(heap)
        out.append((artist, by_artist[artist].pop()))
        # an artist stays in the queue even with no songs
        # left: it still marks one position of spacing
        waiting.append((left + 1, rng.random(), artist))
        # FOLLOW-UP 2: gap=K keeps an artist out for K - 1
        # songs
        if len(waiting) >= gap:
            release(waiting, heap)
    return out


# FOLLOW-UP 3: What is the minimum number of unavoidable
# repeats?
# Answer: With n songs and a dominant artist with m songs,
# the other n - m songs separate at most n - m + 1 groups,
# so at least m - (n - m + 1) = 2m - n - 1 neighbouring
# pairs of that artist are forced (if positive).
def forced_repeats(counts):
    # follow-up: the fewest unavoidable neighbouring
    # pairs.
    # The other songs split the dominant artist into at
```

```
    # most
    # (n - m + 1) groups, so 2m - n - 1 pairs touch.
    n, m = sum(counts), max(counts)
    return max(0, 2 * m - n - 1)


if __name__ == "__main__":
    rng = random.Random(7)
    songs = [("A", "song1"), ("A", "song2"),
             ("B", "song3"), ("B", "song4"),
             ("C", "song5")]
    mixed = shuffle_playlist(songs, rng=rng)
    assert sorted(mixed) == sorted(songs)
    assert adjacent_pairs(mixed) == 0
    # 4 songs by A and 1 by B: 2 touching pairs are forced
    heavy = [("A", str(i)) for i in range(4)]
    heavy.append(("B", "x"))
    out = shuffle_playlist(heavy, rng=rng)
    assert adjacent_pairs(out) == forced_repeats([4, 1])
    assert adjacent_pairs(out) == 2
    strict = shuffle_playlist(heavy, strict=True, rng=rng)
    assert strict is None
    assert shuffle_playlist([], rng=rng) == []
    # at least 3 apart: three artists, two songs each
    trio = [(a, a + str(i))
            for a in "ABC" for i in range(2)]
    far = shuffle_playlist(trio, 3, True, rng)
    assert sorted(far) == sorted(trio)
    for i, (artist, _) in enumerate(far):
        recent = far[max(0, i - 2):i]
        assert all(other != artist for other, _ in recent)
    # runs differ from each other but are all valid
    seen = {tuple(shuffle_playlist(songs, rng=rng))
            for _ in range(30)}
    assert len(seen) > 1
    for _ in range(150):
        size = random.randint(1, 7)
        songs = [(random.choice("ABC"), str(i))
                 for i in range(size)]
        counts = [sum(1 for a, _ in songs if a == x)
                  for x in "ABC"]
        got = shuffle_playlist(songs, rng=rng)
        assert sorted(got) == sorted(songs)
        pairs = adjacent_pairs(got)
        assert pairs == forced_repeats(counts)
```

```
        assert pairs == brute_min_pairs(songs)
```

STUDY HERE: topic 1, Heaps, Stacks and Queues

Site tags: binary-search, heap, divide-and-conquer, hard

HARD | IN APP LC 4 | 1Point3Acres | tech-screen | 26 of 33

Median of Two Sorted Arrays (LC 4) with a Follow-up Variant

ASKED 2026-10-09 | 11 DAYS AGO | LAST 30 DAYS

STUDY: TYPE | ALL TIME 74.2 | 30 DAYS 61 | PRIORITY 4/228

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues

Optimal answer from: your app's interview approach (LeetCode #4)

LeetCode match: LC 4 Median of Two Sorted Arrays

Frequency: Single report | Last asked: 2026-10-09 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/median-two-sorted-arrays-k-inputs

QUESTION

L5 R1 coding: LC 4 (Median of Two Sorted Arrays). Heading toward the LC 295 streaming two-heap approach drew a redirect from the interviewer. Follow-up: extend to many sorted inputs, in the spirit of LC 23 (Merge k Sorted Lists).

Per the candidate, the base question is LC 4 (Median of Two Sorted Arrays): given two sorted arrays, return the median of their combined elements.
Follow-up: the input becomes many sorted arrays instead of two, in the spirit of LC 23 (Merge k Sorted Lists).

APPROACH

Drill LC 4 until you can explain the partition invariant and the odd/even median cases without notes.
Drill LC 23 with a min-heap and with divide-and-conquer merging, and compare their costs as the number of inputs grows.
Practice stating, in under a minute, why a streaming two-heap approach is unnecessary when the inputs are static and sorted.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask for the median definition when the total is even (the mean of the two middle values), whether the arrays can be empty, whether duplicates are possible, and the sizes.

BRUTE FORCE TO SAY OUT LOUD

Merge and sort, then read the middle. O((m + n) log(m + n)); merging two sorted arrays with two pointers is O(m + n).

OPTIMAL IDEA

Binary search the cut in the shorter array: choose i in a and j = (m + n + 1) // 2 - i in b so the left parts hold half of the elements. The cut is valid when a[i-1] <= b[j] and b[j-1] <= a[i], with minus and plus infinity at the ends. The answer is the larger left value for an odd total, or the mean of the larger left and the smaller right value. O(log min(m, n)). For k arrays: binary search on the value and count elements at most x with a bisect per array, O(k log n log range), or pop from a min-heap until the middle, O(N log k).

DRY-RUN THESE CASES

One array empty, both of equal length, total odd and even, all elements equal, one array entirely smaller than the other, negative numbers.

LIKELY FOLLOW-UPS

Many sorted arrays (the stated follow-up), a stream (two heaps), or the k-th element instead of the median.

PITFALLS

Starting from the two-heap design of the streaming median (wrong family for static sorted input). Searching the longer array and running past its end. Off-by-one in the half size for odd totals.

HOW TO THINK ABOUT IT

Both inputs are already sorted, so the median is a partition problem, not a streaming one (the two-heap design of LeetCode 295 is the wrong family and the interviewer redirects it). Split the small array at i and the large one at j so that the left halves together hold half the elements; the split is right when every left value is at most every right value, which only needs two comparisons across the cut. That condition is monotone in i, so binary search it in O(log min(m, n)). For k arrays the same idea becomes: binary search on the value and count elements at most x with one bisect per array.

Basis: LeetCode 4, Median of Two Sorted Arrays; the follow-up to many sorted inputs is in the spirit of LeetCode 23, Merge k Sorted Lists.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def median(a, b):
    merged = sorted(a + b)
    n, mid = len(merged), len(merged) // 2
    if n % 2:
        return merged[mid]
    return (merged[mid - 1] + merged[mid]) / 2
```

TIME	SPACE
O((m + n) log(m + n))	O(m + n)

WHY IT IS SLOW

Sorting ignores that the inputs are sorted and costs O((m + n) log(m + n)) time and O(m + n) memory. A binary search for the partition reads only O(log min(m, n)) values.

OPTIMAL SOLUTION · Binary search on the partition

TIME	SPACE
O(log min(m, n))	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Now there are k sorted arrays instead of two. How do you find the median?

Binary search on the answer value. For a candidate x, count the elements at most x with one bisect per array (O(k log n)); the median is the smallest x whose count reaches the middle rank. With integers the search range is the global min to max, so the cost is O(k log n log(range)). The heap alternative pops the smallest element from a k-way merge until it passes the middle, O((N / 2) log k), which is simpler but linear in the total size.

FOLLOW-UP 2 What is the heap version and when would you choose it?

A min-heap holds the front of every array. Pop (N + 1) / 2 times, remembering the one or two middle values. It is O(N log k) at worst, trivial to get right and needs no bound on the values, so choose it when the arrays are small or the values are not integers; choose the value search when N is huge and k is small.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Median of Two Sorted Arrays (LC 4) with a
# Follow-up Variant
#
# L5 R1 coding: LC 4 (Median of Two Sorted Arrays).
# Heading toward the LC 295 streaming two-heap approach
# drew a redirect from the interviewer. Follow-up: extend
# to many sorted inputs, in the spirit of LC 23 (Merge k
# Sorted Lists). Per the candidate, the base question is
# LC 4 (Median of Two Sorted Arrays): given two sorted
# arrays, return the median of their combined elements.
# Follow-up: the input becomes many sorted arrays instead
# of two, in the spirit of LC 23 (Merge k Sorted Lists).
#
# FOLLOW-UPS (answered below, in this order):
# 1. Now there are k sorted arrays instead of two. How do
# you find the median?
# 2. What is the heap version and when would you choose
# it?
#
# INTERVIEW NOTES
# Both inputs are already sorted, so the median is a
# partition problem, not a streaming one (the two-heap
# design of LeetCode 295 is the wrong family and the
# interviewer redirects it). Split the small array at i
# and the large one at j so that the left halves together
# hold half the elements; the split is right when every
# left value is at most every right value, which only
# needs two comparisons across the cut. That condition is
# monotone in i, so binary search it in O(log min(m, n)).
# For k arrays the same idea becomes: binary search on the
# value and count elements at most x with one bisect per
# array.
#
# BRUTE FORCE (say it first): Merge everything and read
# the middle
# Concatenate, sort and take the middle one or two. It is
# the oracle and the baseline to say first. It ignores
# that the inputs are sorted, and it needs the whole
# merged array in memory. Time O((m + n) log(m + n)).
# Space O(m + n).


import bisect
import heapq


# BRUTE FORCE (say it first, it also checks the answers)
def median_brute(*arrays):
    # the oracle: merge, sort, read the middle
    merged = sorted(x for a in arrays for x in a)
    n, mid = len(merged), len(merged) // 2
    if n % 2:
        return merged[mid]
    return (merged[mid - 1] + merged[mid]) / 2


# HELPERS for the base question
INF = float("inf")


# BASE QUESTION (optimal): Binary search on the partition
# Idea: Make a the shorter array. Binary search i in a,
# set j = half - i in b so the left side holds (m + n + 1)
# // 2 elements.
def median_two(a, b):
    """Both arrays are sorted: binary search the cut. The
    left parts must hold half of all the elements."""
    if len(a) > len(b):
        a, b = b, a
    m, n = len(a), len(b)
    if m + n == 0:
        raise ValueError("no elements")
    lo, hi, half = 0, m, (m + n + 1) // 2
    while lo <= hi:
        i = (lo + hi) // 2
        j = half - i
        # values just left and right of each cut
        a_left = a[i - 1] if i > 0 else -INF
        a_right = a[i] if i < m else INF
        b_left = b[j - 1] if j > 0 else -INF
        b_right = b[j] if j < n else INF
        if a_left <= b_right and b_left <= a_right:
            # a valid cut: read the median off its edges
            if (m + n) % 2:
                return max(a_left, b_left)
            return (max(a_left, b_left)
                    + min(a_right, b_right)) / 2
        if a_left > b_right:
            hi = i - 1
        else:
            lo = i + 1


# FOLLOW-UP 1: Now there are k sorted arrays instead of
# two. How do you find the median?
# Answer: Binary search on the answer value. For a
# candidate x, count the elements at most x with one
# bisect per array (O(k log n)); the median is the
# smallest x whose count reaches the middle rank.
```

```
def median_k_value(arrays):
    """k arrays: binary search the value; one bisect per
    array counts how many elements are at most x."""
    arrays = [a for a in arrays if a]
    total = sum(len(a) for a in arrays)
    if total == 0:
        raise ValueError("no elements")

    def kth(rank):
        # smallest x with at least `rank` elements <= x
        lo = min(a[0] for a in arrays)
        hi = max(a[-1] for a in arrays)
        while lo < hi:
            mid = (lo + hi) // 2
            count = sum(bisect.bisect_right(a, mid)
                        for a in arrays)
            if count >= rank:
                hi = mid
            else:
                lo = mid + 1
        return lo

    if total % 2:
        return kth(total // 2 + 1)
    return (kth(total // 2) + kth(total // 2 + 1)) / 2


# FOLLOW-UP 2: What is the heap version and when would you
# choose it?
# Answer: A min-heap holds the front of every array. Pop
# (N + 1) / 2 times, remembering the one or two middle
# values.
def median_k_heap(arrays):
    """k arrays, simple version: pop from a k-way merge
    until the middle one or two values have been seen."""
    heap = [(a[0], i, 0)
            for i, a in enumerate(arrays) if a]
    heapq.heapify(heap)
    total = sum(len(a) for a in arrays)
    if total == 0:
        raise ValueError("no elements")
    low_rank, high_rank = (total - 1) // 2, total // 2
    seen, middle = 0, []
    while heap:
        value, i, j = heapq.heappop(heap)
        if seen >= low_rank:
            middle.append(value)
        if seen >= high_rank:
            break
        seen += 1
        if j + 1 < len(arrays[i]):
            nxt = (arrays[i][j + 1], i, j + 1)
            heapq.heappush(heap, nxt)
    if total % 2:
        return middle[0]
    return (middle[0] + middle[1]) / 2


if __name__ == "__main__":
    assert median_two([1, 3], [2]) == 2
    assert median_two([1, 2], [3, 4]) == 2.5
    assert median_two([], [5]) == 5
    assert median_k_value([[1, 3], [2], [4, 5, 6]]) == 3.5
    assert median_k_heap([[1, 3], [2], [4, 5, 6]]) == 3.5
    import random
    for _ in range(300):
        arrays = []
        for _ in range(random.randint(1, 5)):
            size = random.randint(0, 6)
            arrays.append(sorted(random.randint(-9, 9)
                                 for _ in range(size)))
        if not any(arrays):
            continue
        want = median_brute(*arrays)
        assert median_k_value(arrays) == want
        assert median_k_heap(arrays) == want
        if len(arrays) >= 2:
            rest = sum(arrays[1:], [])
            rest.sort()
            assert median_two(arrays[0], rest) == want
```

NOTES FROM CANDIDATES

Pitfalls: Starting from the LC 295 (Find Median from Data Stream) two-heap design is the wrong family for the base question; the interviewer stepped in with a hint. Both inputs are already sorted — lean on that.

Follow-ups: Many sorted inputs instead of two

PREPARATION

Drill LC 4 until you can explain the partition invariant and the odd/even median cases without notes.
Drill LC 23 with a min-heap and with divide-and-conquer merging, and compare their costs as the number of inputs grows.
Practice stating, in under a minute, why a streaming two-heap approach is unnecessary when the inputs are static and sorted.

STUDY HERE: topic 1, Heaps, Stacks and Queues

HARD | PracHub | Technical Screen | 27 of 33

Compute Turnstile Crossing Times

POSTED 2026-05-04 | 169 DAYS AGO | LAST 6 MONTHS

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 176/228

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues

Optimal answer from: the interview site's own solution

LeetCode match: LeetCode 'Time Taken to Cross the Door' (exact number unverified) (inferred from overview)

Posted on PracHub: 2026-05-04 (updated 2026-05-10)

Original: https://prachub.com/coding-questions/compute-turnstile-crossing-times

QUESTION

Overview: This question evaluates proficiency in queue simulation, event-driven scheduling, and handling priority and tie-breaking rules while achieving linear time complexity.

APPROACH

Two queues (entering, exiting) ordered by arrival; simulate time with the direction-priority and tie-break rules, jumping the clock when idle. O(n).

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask: how many people can pass per second, which direction goes first when both are waiting, whether the rule depends on what happened in the previous second, and how ties on arrival time break (by id). Clarify what to return (time per person).

BRUTE FORCE TO SAY OUT LOUD

Simulate every second from the earliest to the latest time, checking both queues. Correct but its cost grows with the size of the timestamps, not the number of people.

OPTIMAL IDEA

Keep two FIFO queues, enter and exit, and a variable for what the turnstile did in the previous second. Each step apply the priority rule to pick a queue. When both queues are empty, jump the clock to the next arrival and reset the previous-state to idle if at least one second was skipped. That makes the simulation O(n) after sorting.

DRY-RUN THESE CASES

Two people arriving at the same second with opposite directions. A long idle gap followed by a simultaneous pair. All enters, then all exits. A person who arrives while the turnstile was just used by the other direction.

LIKELY FOLLOW-UPS

INTERVIEW-DAY PLAN

Ask what happens when the idle jump leaves exactly one empty second versus more. Count that the 'previous second' state can be none, not only enter or exit.

PITFALLS

Resetting state only when the gap is two or more seconds. Forgetting to push arrivals that happened during the jump. Comparing queues by id instead of arrival order.

HOW TO THINK ABOUT IT

A scheduling rule that looks at 'what happened last second' is a state machine: write the state down (previous direction: enter, exit, none) before coding. If time values can be huge, simulate events, not seconds: jump the clock to the next arrival when both queues are empty, and remember an idle gap resets the state.

Basis: LeetCode 2534, Time Taken to Cross the Door. Assumed rules (confirm them): one person per second; if an enter and an exit compete, the previous second decides (enter if it was an enter, else exit, and exit if the door was idle).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(arrivals):
    t, prev = 0, None
    enter_q, exit_q = deque(), deque()
    while pending(arrivals, enter_q, exit_q):
        add_arrivals(t, arrivals, enter_q,
                     exit_q)
        who = pick_by_priority(prev, enter_q,
                               exit_q)
        if who is not None:
            passed[who] = t
            prev = direction_of(who)
        t += 1
    return passed
```

TIME	SPACE
O(T + n)	O(n)
where T is the largest timestamp	

WHY IT IS SLOW

A timestamp of 10^9 means a billion iterations even if nothing happens. Jumping to the next arrival when both queues are empty is O(n log n).

OPTIMAL SOLUTION · Event-driven simulation with two queues

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The priority when the turnstile was idle is different in this version of the question.

Keep the same loop and make the idle-state winner a parameter. Only the line that picks the queue changes, which is why the state machine should be written first.

FOLLOW-UP 2 Timestamps are up to 10^9.

That is exactly why the simulation jumps between events: the loop runs once per person, so the answer is O(n log n) no matter how large the times are.

FOLLOW-UP 3 Two turnstiles or a capacity of two people per second.

Run the same pick-a-queue step twice per tick (or per turnstile) and carry a previous-state variable per turnstile. Decide whether the two turnstiles share the queues.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Compute Turnstile Crossing Times
#
# This question evaluates proficiency in queue simulation,
# event-driven scheduling, and handling priority and tie-
# breaking rules while achieving linear time complexity.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The priority when the turnstile was idle is different
# in this version of the question.
# 2. Timestamps are up to 10^9.
# 3. Two turnstiles or a capacity of two people per
# second.
#
# INTERVIEW NOTES
# A scheduling rule that looks at 'what happened last
# second' is a state machine: write the state down
# (previous direction: enter, exit, none) before coding.
# If time values can be huge, simulate events, not
# seconds: jump the clock to the next arrival when both
# queues are empty, and remember an idle gap resets the
# state.
#
# BRUTE FORCE (say it first): Simulate every second
# Walk t = 0, 1, 2, ... adding arrivals to two FIFO queues
# and letting one person through per second by the rule.
# It is easy to verify, but it spends time on every empty
# second, so a timestamp of 10^9 is hopeless. Time O(T +
# n) where T is the largest timestamp. Space O(n).


from collections import deque


# BRUTE FORCE (say it first): Simulate every second | time
# O(T + n) where T is the largest timestamp
def turnstile_brute(events):
    n = len(events)
    order = sorted(
        range(n), key=lambda i: (events[i][0], i)
    )
    res, ent, ext, prev, t, k = (
        [None] * n,
        deque(),
        deque(),
        None,
        0,
        0,
    )
    while any(r is None for r in res):
        while k < n and events[order[k]][0] <= t:
            (
                ent
                if events[order[k]][1] == "enter"
                else ext
            ).append(order[k])
            k += 1
        if ent and ext:
            q = ent if prev == "enter" else ext
        else:
            q = ent or ext or None
        if q is None:
            prev = None
        else:
            p = q.popleft()
            res[p], prev = t, events[p][1]
        t += 1
    return res


# BASE QUESTION (optimal): Event-driven simulation with
# two queues
# Idea: Sort arrivals once. When both queues are empty,
# jump straight to the next arrival time; if the jump
# leaves at least one idle second, reset the previous
# state to 'none'.
# FOLLOW-UP 1: The priority when the turnstile was idle is
# different in this version of the question.
# Answer: Keep the same loop and make the idle-state
# winner a parameter. Only the line that picks the queue
# changes, which is why the state machine should be
# written first.
def turnstile_times(events, idle_first="exit"):
    # events[i] = (arrival time, "enter" or "exit");
    # the result is the second each person goes through
    n = len(events)
    order = sorted(range(n),
                   key=lambda i: (events[i][0], i))
    lines = {"enter": deque(), "exit": deque()}
    result = [0] * n
    t, i, prev, last_used = 0, 0, None, -10
    while i < n or lines["enter"] or lines["exit"]:
        if not lines["enter"] and not lines["exit"]:
            # nobody is waiting: jump to the next arrival
            t = max(t, events[order[i]][0])
            # an idle second forgets the last direction
            if t > last_used + 1:
                prev = None
        # everyone who has arrived by now joins a line
        while i < n and events[order[i]][0] <= t:
            person = order[i]
            lines[events[person][1]].append(person)
            i += 1
        if lines["enter"] and lines["exit"]:
            # FOLLOW-UP 1: idle tie-break set by
            # idle_first
            # both lines: keep the direction used last
            # second,
            # and use idle_first after an idle gap
            want = prev or idle_first
        else:
            want = "enter" if lines["enter"] else "exit"
```

```
        person = lines[want].popleft()
        result[person] = t
        prev, last_used = want, t
        t += 1
    return result


if __name__ == "__main__":
    both = [(0, "enter"), (0, "exit")]
    # idle tie: the exit goes first
    assert turnstile_times(both) == [1, 0]
    # a version where enter wins the idle tie
    assert turnstile_times(both, "enter") == [0, 1]
    # the direction in use keeps going
    keep = [(0, "enter"), (0, "enter"), (1, "exit")]
    assert turnstile_times(keep) == [0, 1, 2]
    # an idle second resets the state
    gap = [(0, "enter"), (10, "enter"), (10, "exit")]
    assert turnstile_times(gap) == [0, 11, 10]
    # huge timestamps are no problem: one loop per person
    far = [(10 ** 9, "enter"), (10 ** 9, "exit")]
    assert turnstile_times(far) == [10 ** 9 + 1, 10 ** 9]
    assert turnstile_times([]) == []
```

FUNCTION AND TEST CASES

Signature

```
def solution(time: list[int], direction: list[int]) -> lis
t[int]:
SAMPLE INPUT
[[0,0,1,5],[0,1,1,0]]
```

Test cases

Input	Expected
[[0, 0, 1, 5], [0, 1, 1, 0]]	[2, 0, 1, 5]
[[0, 0, 0], [0, 1, 0]]	[1, 0, 2]

STUDY HERE: topic 1, Heaps, Stacks and Queues

HARD | IN APP LC 1631, 778 | PracHub | Technical Screen | 28 of 33

Find minimum threshold enabling grid path

POSTED 2026-02-11 | 251 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 40 | 30 DAYS 33 | PRIORITY 53/228

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #1631 Path with Minimum Effort, LeetCode #778 Swim in Rising Water

LeetCode match: LC 1631 Path With Minimum Effort / LC 778 Swim in Rising Water (inferred from overview)

Posted on PracHub: 2026-02-11 (updated 2026-06-01)

Original: https://prachub.com/coding-questions/find-minimum-threshold-enabling-grid-path

QUESTION

Overview: This question evaluates a candidate's understanding of grid pathfinding, reachability under constraints, and algorithmic optimization, focusing on skills such as graph traversal and handling threshold-based feasibility.

APPROACH

Binary search the threshold and BFS/DFS for reachability, or run Dijkstra/union-find where path cost is the max cell value along the path.

HOW TO THINK ABOUT IT

The cost of a path is its largest cell, and a larger limit never makes things worse, so there are two equivalent plans: binary search the limit with a flood fill as the check, or run Dijkstra where the cost of a path is its maximum so far (extending a path never lowers it). Say the cost function out loud first: it is the maximum, not the sum.

Basis: LeetCode 778, Swim in Rising Water: the smallest threshold such that a path from the top-left to the bottom-right cell exists using only cells at or below it. (The related LeetCode 1631 minimises the largest difference between adjacent cells, see the follow-up.)

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(grid):
    best = INF
    for path in all_paths(grid, START, GOAL):
        peak = max(grid[r][c] for r, c in path)
        best = min(best, peak)
    return best
```

BRUTE FORCE

TIME	SPACE
exponential	O(path)

WHY IT IS SLOW

Exponential in the grid size. Dijkstra with 'cost = largest cell so far', or binary search on the limit with a flood fill, is O(m * n log(m * n)).

OPTIMAL SOLUTION · Dijkstra with cost = maximum cell on the path

TIME	SPACE
O(m * n log(m * n))	O(m * n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The cost is the largest difference between adjacent cells on the path (LeetCode 1631).

Same Dijkstra with max, but the step cost is abs(height difference) instead of the cell value. The binary-search version also works: the check allows an edge only if the difference is at most the limit.

FOLLOW-UP 2 Water rises over time (the cell is usable from time H[r][c]).

That is exactly the maximum-cell cost above; the answer is the time at which the corner becomes reachable.

FOLLOW-UP 3 Return the path, not only the value.

Remember each cell's parent at its last improvement and rebuild the path from the target; for the binary-search version run the flood fill once more at the answer limit with parent pointers.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Find minimum threshold enabling grid path
#
# This question evaluates a candidate's understanding of
# grid pathfinding, reachability under constraints, and
# algorithmic optimization, focusing on skills such as
# graph traversal and handling threshold-based
# feasibility.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The cost is the largest difference between adjacent
# cells on the path (LeetCode 1631).
# 2. Water rises over time (the cell is usable from time
# H[r][c]).
# 3. Return the path, not only the value.
```

```
#
# INTERVIEW NOTES
# The cost of a path is its largest cell, and a larger
# limit never makes things worse, so there are two
# equivalent plans: binary search the limit with a flood
# fill as the check, or run Dijkstra where the cost of a
# path is its maximum so far (extending a path never
# lowers it). Say the cost function out loud first: it is
# the maximum, not the sum.
#
# BRUTE FORCE (say it first): Try every simple path
# Depth-first search over all simple paths from the top-
# left to the bottom-right, tracking the largest cell,
# with pruning when it cannot beat the best. Exact and the
# oracle; exponential in the grid size. Time exponential.
# Space O(path).


import heapq


# BRUTE FORCE (say it first): Try every simple path | time
# exponential
def min_max_path_brute(H):
    m, n = len(H), len(H[0])
    best = [float("inf")]

    def go(r, c, seen, worst):
        worst = max(worst, H[r][c])
        if worst >= best[0]:
            return
        if (r, c) == (m - 1, n - 1):
            best[0] = worst
            return
        for nr, nc in (
            (r + 1, c),
            (r - 1, c),
            (r, c + 1),
            (r, c - 1),
        ):
            if (
                0 <= nr < m
                and 0 <= nc < n
                and (nr, nc) not in seen
            ):
                go(nr, nc, seen | {(nr, nc)}, worst)

    go(0, 0, {(0, 0)}, 0)
    return best[0]


# HELPERS for the base question
INF = float("inf")


def around(r, c, m, n):
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        a, b = r + dr, c + dc
        if 0 <= a < m and 0 <= b < n:
            yield a, b


# ALSO: the Dijkstra version
def best_path(H, step_cost, start_cost):
    # Dijkstra where a path costs the MAX of its steps;
    # the
    # first time a cell leaves the heap its cost is final
    m, n = len(H), len(H[0])
    best = {(0, 0): start_cost}
    parent = {}
    heap = [(start_cost, 0, 0)]
    while heap:
        cost, r, c = heapq.heappop(heap)
        if cost > best[(r, c)]:
            continue
        if (r, c) == (m - 1, n - 1):
            path = [(r, c)]
            while (r, c) in parent:
                r, c = parent[(r, c)]
                path.append((r, c))
            return cost, path[::-1]
        for a, b in around(r, c, m, n):
            nxt = max(cost, step_cost(H, r, c, a, b))
            if nxt < best.get((a, b), INF):
                best[(a, b)] = nxt
                parent[(a, b)] = (r, c)
                heapq.heappush(heap, (nxt, a, b))
    return None, []


# BASE QUESTION (optimal): Dijkstra with cost = maximum
# cell on the path
# Idea: Heap of (largest cell so far, cell). Pop the cell
# with the smallest value, extend to each neighbour with
# cost max(current, neighbour), and keep only
# improvements.
def min_max_path(H):
    # cost of a path = its largest cell value
    # (water rising over time is the same problem)
    def step(H, r, c, a, b):
        return H[a][b]
    return best_path(H, step, H[0][0])


# FOLLOW-UP 1: The cost is the largest difference between
# adjacent cells on the path (LeetCode 1631).
# Answer: Same Dijkstra with max, but the step cost is
# abs(height difference) instead of the cell value. The
# binary-search version also works: the check allows an
# edge only if the difference is at most the limit.
def min_effort_path(H):
    # LeetCode 1631: cost = the largest height difference
    # between two neighbouring cells on the path
    def step(H, r, c, a, b):
        return abs(H[a][b] - H[r][c])
    return best_path(H, step, 0)


if __name__ == "__main__":
    H = [[0, 1, 2], [4, 3, 5], [8, 7, 6]]
    cost, path = min_max_path(H)
    # 0 -> 1 -> 3 -> 7 -> 6 or via 2, 5, 6: the max is 6
    assert cost == 6
    assert path[0] == (0, 0) and path[-1] == (2, 2)
    assert max(H[r][c] for r, c in path) == cost
    assert min_max_path([[5]]) == (5, [(0, 0)])
    effort, path2 = min_effort_path([[1, 2, 2], [3, 8, 2],
                                     [5, 3, 5]])
    assert effort == 2
    assert min_effort_path([[7]]) == (0, [(0, 0)])
```

FUNCTION AND TEST CASES

Signature

```
def solution(grid: list[list[int]]) -> int:
```

Sample call

```
solution([[0, 2], [1, 3]])
```

Test cases

Input	Expected
`([[0, 2], [1, 3]],)`	`3`
`([[0, 100, 2], [1, 2, 2], [3, 4, 1]],)`	`2`

STUDY HERE: topic 1, Heaps, Stacks and Queues

HARD | PracHub | Technical Screen | 29 of 33

Compute minimax grid path and network delay

POSTED 2026-01-22 | 271 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 37.9 | 30 DAYS 0 | PRIORITY 99/228

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Related in your app (a neighbouring problem, not this exact question): LeetCode #1631 Path with Minimum Effort, LeetCode #743 Network Delay Time

LeetCode match: LC 1631 Path With Minimum Effort + LC 743 Network Delay Time (inferred from overview)

Posted on PracHub: 2026-01-22 (updated 2026-05-26)

Original: https://prachub.com/coding-questions/compute-minimax-grid-path-and-network-delay

QUESTION

Overview: This question evaluates algorithmic problem-solving in shortest-path and path-optimization contexts, specifically minimax pathfinding on a grid and single-source reachability/timing in a weighted directed graph, testing graph modeling, alternative path cost metrics, edge-case handling, and working with large numeric weights.

APPROACH

Part 1: Dijkstra (or binary search + BFS) where cost is the max edge/cell along the path. Part 2: Dijkstra from the source; answer is the max distance, or -1 if some node is unreachable.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Two parts. Part 1: ask what the path cost is (the maximum cell value along the path, including the start), the allowed moves (4-directional) and the endpoints (top-left to bottom-right). Part 2: ask whether the graph is directed, what to return if some node is unreachable (-1), and the size of weights. The scraped solution covers only part 1.

BRUTE FORCE TO SAY OUT LOUD

Part 1: DFS over all paths, O(4^(mn)). Part 2: Bellman-Ford, which is fine for small graphs.

OPTIMAL IDEA

Part 1: Dijkstra where the cost of a path is the maximum seen so far, with a min-heap keyed on that maximum: new_cost = max(cost, cell). O(mn log mn). Equivalent: binary search the cap and BFS, or sort cells and union-find until start and end connect. Part 2: Dijkstra from the source; the time for a signal to reach everyone is the largest shortest distance, or -1 if any node is unreachable. O((V + E) log V).

DRY-RUN THESE CASES

Part 1: [[1,10,6],[1,3,2],[7,2,2]] gives 3 (as in the example), a 1 x 1 grid, a grid where the best path takes a detour. Part 2: source with no outgoing edges, a disconnected node, parallel edges.

LIKELY FOLLOW-UPS

Part 1 with the cost as the sum minus something, or paths that may revisit cells. Part 2 with large weights (use 64-bit integers).

PITFALLS

Using the sum instead of the maximum when relaxing. Not counting the start cell's own value.

HOW TO THINK ABOUT IT

Both are 'shortest path' with a different notion of cost. For the grid the cost of a path is its maximum, which still works in Dijkstra: the best way to reach a cell is the one with the smallest maximum, and extending a path can never lower it. For the network, Dijkstra from the source gives every arrival time, and the answer is the largest of them (-1 if some node is never reached).

Basis: Two parts: LeetCode 1631-style minimax path (path cost = the largest cell value on the path, start cell included) and LeetCode 743, Network Delay Time (time for a signal to reach every node, or -1).

BRUTE FORCE · Part 1: DFS over every simple path

BRUTE FORCE · PSEUDO-CODE

```
def brute(grid):
    best = INF
    for path in all_paths(grid, START, GOAL):
        peak = max(grid[r][c] for r, c in path)
        best = min(best, peak)
    return best
```

TIME	SPACE
exponential	O(path)

WHY IT IS SLOW

The number of simple paths is exponential. Dijkstra where the cost of a path is its maximum cell (or a union-find over cells added in increasing order) is O(m * n log(m * n)).

BRUTE FORCE · Part 2: Bellman-Ford from the source

BRUTE FORCE · PSEUDO-CODE

```
def brute(n, edges, source):
    dist = [INF] * n
    dist[source] = 0
    for _ in range(n - 1):
        for u, v, w in edges:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
    worst = max(dist)
    return -1 if worst == INF else worst
```

TIME	SPACE
O(V * E)	O(V)

WHY IT IS SLOW

V - 1 rounds over all edges, O(V * E). Dijkstra with a heap is O((V + E) log V) for non-negative weights.

OPTIMAL SOLUTION · Part 1: Dijkstra where the cost of a path is its maximum cell

TIME	SPACE
O(m * n log(m * n))	O(m * n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Part 2: Dijkstra from the source, answer is the largest distance

TIME	SPACE
O((V + E) log V)	O(V + E)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Part 1 with a union-find (sort cells, add them one by one).

Add cells in increasing order of height, union neighbours already added, and stop when the start and the goal are connected: that height is the answer. O(mn log mn) for the sort.

FOLLOW-UP 2 Part 2: weights are huge.

Use 64-bit integers (Python is fine); keep sums in a wide type, and with weights up to 10^9 and 10^5 edges a path can reach 10^14. **Cost:** unchanged O((V + E) log V); sums need 64-bit (or Python) integers.

FOLLOW-UP 3 Part 2: the signal can be blocked at some nodes, or each node adds a processing delay.

A node delay is added to the edge cost leaving the node (or modelled by splitting each node into in and out with an edge of the delay). The same Dijkstra then applies.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Compute minimax grid path and network
# delay
#
# This question evaluates algorithmic problem-solving in
# shortest-path and path-optimization contexts,
# specifically minimax pathfinding on a grid and single-
# source reachability/timing in a weighted directed graph,
# testing graph modeling, alternative path cost metrics,
# edge-case handling, and working with large numeric
# weights.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Part 1 with a union-find (sort cells, add them one by
# one).
# 2. Part 2: weights are huge.
# 3. Part 2: the signal can be blocked at some nodes, or
# each node adds a processing delay.
#
# INTERVIEW NOTES
# Both are 'shortest path' with a different notion of
# cost. For the grid the cost of a path is its maximum,
# which still works in Dijkstra: the best way to reach a
# cell is the one with the smallest maximum, and extending
# a path can never lower it. For the network, Dijkstra
# from the source gives every arrival time, and the answer
# is the largest of them (-1 if some node is never
# reached).
#
# BRUTE FORCE (say it first): Part 1: DFS over every
# simple path
# Try all simple paths from the top-left to the bottom-
# right and take the smallest maximum cell value. Exact
# and the oracle; exponential in the grid size. Time
# exponential. Space O(path).


import heapq


# BRUTE FORCE (say it first): Part 1: DFS over every
# simple path | time exponential
def minimax_path_dfs(H):
    m, n = len(H), len(H[0])
    best = [float("inf")]
```

```
    def go(r, c, seen, worst):
        worst = max(worst, H[r][c])
        if worst >= best[0]:
            return
        if (r, c) == (m - 1, n - 1):
            best[0] = worst
            return
        for nr, nc in (
            (r + 1, c),
            (r - 1, c),
            (r, c + 1),
            (r, c - 1),
```

```
        ):
            if (
                0 <= nr < m
                and 0 <= nc < n
                and (nr, nc) not in seen
            ):
                go(nr, nc, seen | {(nr, nc)}, worst)

    go(0, 0, {(0, 0)}, 0)
    return best[0]


# HELPERS for the base question
INF = float("inf")


def neighbors(r, c, m, n):
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        a, b = r + dr, c + dc
        if 0 <= a < m and 0 <= b < n:
            yield a, b


# BASE QUESTION (optimal): Dijkstra where the cost of a
# path is its maximum cell
# Idea: Heap of (best maximum so far, cell). Pop the cell
# with the smallest bottleneck, and for each neighbour the
# new bottleneck is max(current, neighbour). Improve if
# smaller.
def minimax_path(H):
    # Part 1: the path whose highest cell is as low as
    # possible
    m, n = len(H), len(H[0])
```

```
    best = {(0, 0): H[0][0]}
    heap = [(H[0][0], 0, 0)]
    while heap:
        cost, r, c = heapq.heappop(heap)
        if cost > best[(r, c)]:
            continue
        if (r, c) == (m - 1, n - 1):
            return cost
        for a, b in neighbors(r, c, m, n):
            # the cost of a path is its maximum, never a
            # sum
            nxt = max(cost, H[a][b])
            if nxt < best.get((a, b), INF):
                best[(a, b)] = nxt
                heapq.heappush(heap, (nxt, a, b))
    return -1


# PART 2: network delay
def network_delay(n, edges, source, node_delay=None):
    # Part 2: when does the signal reach every node?
    # edges are (u, v, weight); -1 if some node is
    # unreachable
    graph = [[] for _ in range(n)]
    for u, v, w in edges:
        graph[u].append((v, w))
    dist = [INF] * n
    dist[source] = 0
    heap = [(0, source)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        # a processing delay is paid when leaving a node
        pause = node_delay[u] if node_delay else 0
        for v, w in graph[u]:
            if d + pause + w < dist[v]:
                dist[v] = d + pause + w
                heapq.heappush(heap, (dist[v], v))
```

```
    worst = max(dist)
    return -1 if worst == INF else worst


# FOLLOW-UP 1: Part 1 with a union-find (sort cells, add
# them one by one).
```

```
# Answer: Add cells in increasing order of height, union
# neighbours already added, and stop when the start and
# the goal are connected: that height is the answer. O(mn
# log mn) for the sort.
def minimax_path_union_find(H):
    # the same answer: add cells from lowest to highest
    # and
    # stop when the start and the goal are connected
    m, n = len(H), len(H[0])
    parent = {}

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    cells = sorted((H[r][c], r, c)
                   for r in range(m) for c in range(n))
    for h, r, c in cells:
        parent[(r, c)] = (r, c)
        for a, b in neighbors(r, c, m, n):
            if (a, b) in parent:
                parent[find((r, c))] = find((a, b))
        if (0, 0) in parent and (m - 1, n - 1) in parent:
            if find((0, 0)) == find((m - 1, n - 1)):
                return h
    return -1


if __name__ == "__main__":
    H = [[0, 2, 1], [1, 3, 1], [2, 2, 1]]
    assert minimax_path(H) == 2
    assert minimax_path(H) == minimax_path_union_find(H)
    assert minimax_path([[5]]) == 5
    edges = [(0, 1, 2), (1, 2, 3), (0, 2, 10)]
    assert network_delay(3, edges, 0) == 5
    # a node that cannot be reached
    assert network_delay(4, edges, 0) == -1
    # huge weights are fine with Python integers
    big = [(0, 1, 10 ** 9), (1, 2, 10 ** 9)]
    assert network_delay(3, big, 0) == 2 * 10 ** 9
    # each node adds a processing delay of 1
    slow = network_delay(3, big, 0, [1, 1, 1])
    assert slow == 2 * 10 ** 9 + 2
```

FUNCTION AND TEST CASES

Signature

```
def solution(H: list[list[int]]) -> int:
SAMPLE INPUT
[[1, 10, 6], [1, 3, 2], [7, 2, 2]]
```

Test cases

Input	Expected
`[[1, 10, 6], [1, 3, 2], [7, 2, 2]]`	`3`
`[[5, 1, 7], [4, 8, 2], [3, 6, 0]]`	`6`

STUDY HERE: topic 1, Heaps, Stacks and Queues

HARD | PracHub | Technical Screen | 30 of 33

Return top-k subset sums in descending order

POSTED 2026-01-07 | 286 DAYS AGO | OLDER

STUDY: TALK | ALL TIME 5 | 30 DAYS 0 | PRIORITY 170/228

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues

Optimal answer from: the interview site's own solution

LeetCode match: LC 2386 Find the K-Sum of an Array

Posted on PracHub: 2026-01-07 (updated 2026-05-21)

Original: https://prachub.com/coding-questions/return-top-k-subset-sums-in-descending-order

QUESTION

Overview: This question evaluates proficiency in combinatorial reasoning, algorithmic optimization for large search spaces, and efficient extraction of top-k values within the coding & algorithms domain.

APPROACH

Sort values, start from the all-positives sum, and use a max/min-heap of (sum, index) states that either drop the next element or swap it for the following one, popping k times. O(n log n + k log k).

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the empty subset counts, whether values can be negative or repeated, how large k is compared with 2^n, and whether equal sums count separately.

BRUTE FORCE TO SAY OUT LOUD

Enumerate all 2^n subsets, sort, take k. Fine for n <= 20; say that out loud.

OPTIMAL IDEA

Reduce negatives: let base be the sum of the positive values, and work with absolute values. Every subset sum of the original equals base minus a subset sum of the absolute values, so the k largest sums are base minus the k smallest subset sums of the sorted absolute values. Generate the k smallest sums with a min-heap of (sum, last index): from a state you either add the next element or replace the last element with the next one. Each pop yields the next smallest sum and creates at most two states, so O(n log n + k log k).

DRY-RUN THESE CASES

All positive values. All negative values (best is the empty subset). A zero in the array. k = 1 and k = 2^n.

LIKELY FOLLOW-UPS

INTERVIEW-DAY PLAN

Need k sums without storing all of them, subsets themselves (store parent pointers), or a limit on subset size.

PITFALLS

Forgetting to sort first, which breaks the add-next/replace-last move set. Pushing duplicate states.

HOW TO THINK ABOUT IT

k is small compared with 2^n, so never enumerate all subsets. Turn the problem into 'k smallest sums of non-negative numbers': subtract everything from the sum of the positive values. Then grow answers in increasing order with a heap where each state has exactly two successors (extend with the next element, or swap the last element for the next one).

Basis: Written; the classic 'k smallest subset sums' heap technique (also behind LeetCode 2386, K-Sum of an Array).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(nums, k):
    sums = []
    for size in range(len(nums) + 1):
        for subset in combinations(nums, size):
            sums.append(sum(subset))
    sums.sort(reverse=True)
    return sums[:k]
```

TIME	SPACE
O(2^n * n)	O(2^n)

WHY IT IS SLOW

2^n subsets and a sort of them: O(2^n * n) time and O(2^n) space, hopeless past about n = 20. A heap that grows the k best sums step by step is O(n log n + k log k).

OPTIMAL SOLUTION · Heap of the k smallest sums of absolute values

TIME	SPACE
O(n log n + k log k)	O(n + k)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the subsets themselves, not only the sums.

Store each heap state as a node with (last index, parent node) instead of copying lists, and rebuild a subset by walking parents only for the k answers. That keeps pushes O(1).

FOLLOW-UP 2 k can be as large as 2^n.

Then the output itself is that large; the heap approach is still O(k log k) and stops when the heap empties. Check k against 2^n first and fall back to enumeration for tiny n.

FOLLOW-UP 3 Limit subsets to at most m elements.

Add the element count to each heap state and refuse to push successors that exceed m. The ordering argument is unchanged because both successors never make the sum smaller.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Return top-k subset sums in descending
# order
#
# This question evaluates proficiency in combinatorial
# reasoning, algorithmic optimization for large search
# spaces, and efficient extraction of top-k values within
# the coding & algorithms domain.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Return the subsets themselves, not only the sums.
# 2. k can be as large as 2^n.
# 3. Limit subsets to at most m elements.
#
# INTERVIEW NOTES
# k is small compared with 2^n, so never enumerate all
# subsets. Turn the problem into 'k smallest sums of non-
# negative numbers': subtract everything from the sum of
# the positive values. Then grow answers in increasing
# order with a heap where each state has exactly two
# successors (extend with the next element, or swap the
# last element for the next one).
#
# BRUTE FORCE (say it first): Enumerate all 2^n subsets
# and sort
# Compute every subset sum, sort in descending order and
# take k. Fine for n up to about 20, hopeless beyond that.
# Time O(2^n * n). Space O(2^n).


import heapq
from itertools import combinations


def brute(nums, k):
    # every subset: the check used by the demo
    sums = []
    for r in range(len(nums) + 1):
        for c in combinations(nums, r):
            sums.append(sum(c))
    return sorted(sums, reverse=True)[:k]


# BRUTE FORCE (say it first): Enumerate all 2^n subsets
# and sort | time O(2^n * n)
def top_k_sums_brute(nums, k):
```

```
    sums = []
    for mask in range(1 << len(nums)):
        sums.append(
            sum(
                nums[i]
                for i in range(len(nums))
                if mask >> i & 1
            )
        )
    return sorted(sums, reverse=True)[:k]


# BASE QUESTION (optimal): Heap of the k smallest sums of
# absolute values
# Idea: Every subset sum equals (sum of positives) minus a
# subset sum of the absolute values, so the k largest sums
# are the sum of positives minus the k smallest subset
# sums of the sorted absolute values.
def top_k_sums(nums, k):
    # Idea: start from the best subset (all positives).
    # Every
    # other subset is that one with some elements
    # "flipped",
    # which costs |x|. So the k best subsets are the k
    # cheapest sets of flips: a heap over sorted |x|.
    base = sum(x for x in nums if x > 0)
    cost = sorted(abs(x) for x in nums)
    out = [0]
    heap = [(cost[0], 0)] if cost else []
    while heap and len(out) < k:
        s, i = heapq.heappop(heap)
        out.append(s)
        if i + 1 < len(cost):
            # two successors, neither is ever cheaper:
            # add the next element ...
            heapq.heappush(heap, (s + cost[i + 1], i + 1))
            # ... or move the last flipped one up by one
            moved = s - cost[i] + cost[i + 1]
            heapq.heappush(heap, (moved, i + 1))
    return [base - c for c in out[:k]]


# FOLLOW-UP 1: Return the subsets themselves, not only the
# sums.
# Answer: Store each heap state as a node with (last
```

```
# index, parent node) instead of copying lists, and
# rebuild a subset by walking parents only for the k
# answers. That keeps pushes O(1).
def top_k_subsets(nums, k):
    # also return the subsets: states keep a link to their
    # parent instead of copying lists
    order = sorted(range(len(nums)),
                   key=lambda i: abs(nums[i]))
    cost = [abs(nums[i]) for i in order]
    best = {i for i in range(len(nums)) if nums[i] > 0}

    def subset(chain):
        flipped = set()
        while chain is not None:
            flipped.add(order[chain[0]])
            chain = chain[1]
        return sorted(nums[i] for i in best ^ flipped)

    base = sum(nums[i] for i in best)
    out = [(base, sorted(nums[i] for i in best))]
    heap = [(cost[0], 0, (0, None))] if cost else []
    while heap and len(out) < k:
        s, i, chain = heapq.heappop(heap)
        out.append((base - s, subset(chain)))
        if i + 1 < len(cost):
            heapq.heappush(heap, (s + cost[i + 1], i + 1,
                                  (i + 1, chain)))
            # replace the last flip i by i + 1
            moved = s - cost[i] + cost[i + 1]
            heapq.heappush(heap, (moved, i + 1,
```

```
                                (i + 1, chain[1])))
    return out[:k]


if __name__ == "__main__":
    nums = [3, -1, 2, -4]
    assert top_k_sums(nums, 5) == brute(nums, 5)
    assert top_k_sums(nums, 4) == [5, 4, 3, 2]
    # k bigger than the number of subsets: all 2^n sums
    assert top_k_sums([1, 2], 10) == brute([1, 2], 10)
    assert top_k_sums([], 3) == [0]
    cases = ([5, -2, 7, 1], [-1, -2, -3], [0, 4, -4])
    for sample in cases:
        for k in range(1, 2 ** len(sample) + 1):
            got = top_k_sums(sample, k)
```

```
            assert got == brute(sample, k)
    # the subsets add up to the sums
    for total, sub in top_k_subsets(nums, 4):
        assert sum(sub) == total
    totals = [t for t, _ in top_k_subsets(nums, 4)]
    assert totals == [5, 4, 3, 2]
```

FUNCTION AND TEST CASES

Signature

```
def solution(w: list[int], k: int) -> list[int]:
```

Sample call

```
solution([3, 2, 1], 4)
```

Test cases

Input	Expected
([3, 2, 1], 4)	[6, 5, 4, 3]
([5], 2)	[5, 0]

STUDY HERE: topic 1, Heaps, Stacks and Queues

HARD | PracHub | Technical Screen | 31 of 33

Minimize travel time with optional meeting point

POSTED 2026-01-06 | 287 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 37.9 | 30 DAYS 33 | PRIORITY 59/228

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #743 Network Delay Time, LeetCode #2203 Minimum Weighted Subgraph With the Required Paths

LeetCode match: LC 743 Network Delay Time (multi-source Dijkstra); related LC 2203 (inferred from overview)

Posted on PracHub: 2026-01-06 (updated 2026-05-22)

Original: https://prachub.com/coding-questions/minimize-travel-time-with-optional-meeting-point

QUESTION

Overview: This question evaluates a candidate's understanding of graph shortest-path computation and meeting-point optimization, testing skills in handling weighted edges, aggregating path costs, and reasoning about simultaneous movements on a network.

APPROACH

Run Dijkstra from each traveler's start, then pick the node that minimizes the total (or max, per the statement) travel time; compare against the no-meeting option.

INTERVIEW-DAY PLAN

CLARIFY FIRST

From the scraped solution the setup is two travelers A and B and a destination. Confirm: do they have to travel together after meeting, do they wait for each other (the meeting time is the later arrival), is the graph directed or undirected, and what to return (the minimum time and the meeting node, with ties going to the smallest node, or -1 and -1 if impossible). My copy has only the overview, so verify these.

BRUTE FORCE TO SAY OUT LOUD

Try every node as a meeting point with a fresh shortest path computation each time. O(n * (E log V)).

OPTIMAL IDEA

Run Dijkstra from A, Dijkstra from B, and Dijkstra from the destination on the reversed graph (so you get every node's distance to the destination in one pass). For each node m the time is max(distA[m], distB[m]) + distToDest[m]. Take the minimum, with ties to the smallest node. Compare with the no-meeting option if each can also go directly. O((V + E) log V).

INTERVIEW-DAY PLAN

DRY-RUN THESE CASES

A and B start at the same node, a node that only one of them can reach, a directed graph where the way back is longer, two nodes with equal time.

LIKELY FOLLOW-UPS

More travelers (Dijkstra from each, take the max), minimize total distance instead of arrival, or an optional meeting point.

PITFALLS

Forgetting to reverse the graph for the destination distances in a directed graph. Using sum instead of max for waiting.

HOW TO THINK ABOUT IT

Break it into shortest paths from three places: from A, from B, and into the destination (a Dijkstra on the reversed graph if directed). For each node m the cost is max(distA[m], distB[m]) + dist(m to destination); take the minimum. Three single-source searches replace trying every meeting node with its own search.

Basis: Written from the scraped solution (confirm it): two travellers A and B, a destination; they may meet at a node (the meeting happens when the later one arrives) and then travel together to the destination; return [best time, best node] with ties to the smallest node, or [-1, -1].

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(n, edges, A, B, dest):
    d = floyd_warshall(n, edges)  # all pairs
    best, best_m = INF, None
    for m in range(n):
        cost = max(d[A][m], d[B][m]) + d[m][dest]
        if cost < best:
            best, best_m = cost, m
    return best
```

TIME	SPACE
O(V^3)	O(V^2)

WHY IT IS SLOW

All-pairs distances cost O(V^3) when only three single-source distances are needed. Dijkstra from A, from B and into the destination is O((V + E) log V).

OPTIMAL SOLUTION · Three Dijkstra runs (from A, from B, and into the destination)

TIME	SPACE
O((V + E) log V)	O(V + E)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 More than two travellers.

Run one Dijkstra per traveller and one from the destination, then for each node take the maximum arrival time (if they wait for each other) or the sum of distances (if the cost is total distance) plus the shared leg. K + 1 searches.

FOLLOW-UP 2 Meeting is optional.

Also compute the cost with no meeting (each goes straight to the destination: max(distA[dest], distB[dest])) and take the smaller. Meeting at the destination itself already covers it when the cost is the later arrival.

FOLLOW-UP 3 Edge weights change over time (traffic).

Dijkstra still works if the graph is FIFO (leaving later never arrives earlier); each edge's cost becomes a function of the departure time, evaluated when you relax it.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Minimize travel time with optional
# meeting point
#
# This question evaluates a candidate's understanding of
# graph shortest-path computation and meeting-point
# optimization, testing skills in handling weighted edges,
# aggregating path costs, and reasoning about simultaneous
# movements on a network.
#
# FOLLOW-UPS (answered below, in this order):
# 1. More than two travellers.
# 2. Meeting is optional.
# 3. Edge weights change over time (traffic).
#
# INTERVIEW NOTES
# Break it into shortest paths from three places: from A,
# from B, and into the destination (a Dijkstra on the
# reversed graph if directed). For each node m the cost is
# max(distA[m], distB[m]) + dist(m to destination); take
# the minimum. Three single-source searches replace trying
# every meeting node with its own search.
#
# BRUTE FORCE (say it first): Floyd-Warshall all-pairs,
# then try every meeting node
# Compute the shortest distance between every pair of
# nodes, then for each node m take max(d[A][m], d[B][m]) +
# d[m][dest]. Simple and the oracle; cubic in the number
# of nodes. Time O(V^3). Space O(V^2).


import heapq


# HELPERS for the brute force
INF = float("inf")


# BRUTE FORCE (say it first): Floyd-Warshall all-pairs,
# then try every meeting node | time O(V^3)
def meeting_point_floyd(n, edges, a, b, dest, directed):
    INF = float("inf")
    d = [
        [0 if i == j else INF for j in range(n)]
        for i in range(n)
    ]
    for u, v, w in edges:
        d[u][v] = min(d[u][v], w)
        if not directed:
            d[v][u] = min(d[v][u], w)
    for k in range(n):
        for i in range(n):
            for j in range(n):
                if d[i][k] + d[k][j] < d[i][j]:
                    d[i][j] = d[i][k] + d[k][j]
    best = (INF, -1)
    for m in range(n):
        t = max(d[a][m], d[b][m]) + d[m][dest]
        if t < best[0]:
            best = (t, m)
    return (
        [-1, -1]
        if best[0] == INF
        else [best[0], best[1]]
    )


# HELPERS for the base question
def dijkstra(graph, source):
    dist = [INF] * len(graph)
    dist[source] = 0
    heap = [(0, source)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        for v, w in graph[u]:
            if d + w < dist[v]:
                dist[v] = d + w
                heapq.heappush(heap, (d + w, v))
    return dist


def build(n, edges, directed):
    # forward graph, and the reversed graph for "to dest"
    fwd = [[] for _ in range(n)]
    rev = [[] for _ in range(n)]
    for u, v, w in edges:
        fwd[u].append((v, w))
        rev[v].append((u, w))
        if not directed:
            fwd[v].append((u, w))
            rev[u].append((v, w))
    return fwd, rev


# BASE QUESTION (optimal): Three Dijkstra runs (from A,
# from B, and into the destination)
# Idea: distA and distB come from forward searches. The
# distance from every node to the destination is a search
# from the destination on the reversed graph (the same
# graph if undirected).
def meeting_point(n, edges, a, b, dest, directed=False):
    # both start at the same time, meet at node m, wait
    # for
    # each other, then travel together to dest. Meeting at
    # dest itself covers the "no meeting" plan.
    fwd, rev = build(n, edges, directed)
    da, db = dijkstra(fwd, a), dijkstra(fwd, b)
    # distance from every node TO dest = search the
    # reverse graph
    to_dest = dijkstra(rev, dest)
```

```
    best = (INF, -1)
    for m in range(n):
        total = max(da[m], db[m]) + to_dest[m]
        if total < best[0]:
            best = (total, m)
    return [-1, -1] if best[0] == INF else list(best)


# FOLLOW-UP 1: More than two travellers.
# Answer: Run one Dijkstra per traveller and one from the
# destination, then for each node take the maximum arrival
# time (if they wait for each other) or the sum of
# distances (if the cost is total distance) plus the
# shared leg.
def meeting_point_many(n, edges, starts, dest,
                       directed=False):
    # k travellers: one search each, plus one from dest
    fwd, rev = build(n, edges, directed)
    dists = [dijkstra(fwd, s) for s in starts]
    to_dest = dijkstra(rev, dest)
    best = (INF, -1)
    for m in range(n):
        wait = max(d[m] for d in dists)
        if wait + to_dest[m] < best[0]:
```

```
            best = (wait + to_dest[m], m)
    return [-1, -1] if best[0] == INF else list(best)


if __name__ == "__main__":
    # a star: both travellers reach the hub 2, then go to
    # 3
    edges = [(0, 2, 1), (1, 2, 2), (2, 3, 4), (0, 3, 9),
             (1, 3, 9)]
    # meet at 2: max(1, 2) + 4 = 6; going alone costs 9
    assert meeting_point(4, edges, 0, 1, 3) == [6, 2]
    # nobody can reach the destination
    short = [(0, 1, 1)]
    assert meeting_point(3, short, 0, 1, 2) == [-1, -1]
    # same start: they simply walk together
    assert meeting_point(4, edges, 0, 0, 3)[0] == 5
    # three travellers
    got = meeting_point_many(4, edges, [0, 1, 1], 3)
    assert got == [6, 2]
    # directed roads: reverse edges are not usable
    one_way = [(0, 1, 1), (1, 2, 1)]
    walk = meeting_point(3, one_way, 0, 1, 2, True)
    assert walk == [2, 1]
```

FUNCTION AND TEST CASES

Signature

```
def solution(n: int, edges: list[list[int]], startA: int,
startB: int, dest: int, directed: bool = False) -> list[in
t]:
```

Sample call

```
solution(4, [(0, 1, 2), (1, 2, 1), (1, 3, 2), (2, 3, 2)],
0, 2, 3, False)
```

Test cases

Input	Expected
(4, [(0, 1, 2), (1, 2, 1), (1, 3, 2), (2, 3, 2)], 0, 2, 3, False)	[4, 1]
(4, [(0, 1, 2), (2, 1, 3), (1, 3, 2), (0, 3, 10), (2, 3, 5)], 0, 2, 3, True)	[5, 1]

STUDY HERE: topic 1, Heaps, Stacks and Queues

HARD | IN APP LC 778, 1631 | PracHub | Technical Screen | 32 of 33

Minimize maximum height along a grid path

POSTED 2026-01-04 | 289 DAYS AGO | OLDER

STUDY: TYPE | ALL TIME 40 | 30 DAYS 33 | PRIORITY 54/228

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #778 Swim in Rising Water, LeetCode #1631 Path with Minimum Effort

LeetCode match: LC 778 Swim in Rising Water / LC 1631

Posted on PracHub: 2026-01-04 (updated 2026-06-26)

Original: https://prachub.com/coding-questions/minimize-maximum-height-along-a-grid-path

QUESTION

Overview: This question evaluates algorithmic pathfinding and graph-modeling skills, specifically reasoning about grid-based route selection where the objective is defined by an aggregate maximum over visited cell heights.

APPROACH

Min-heap Dijkstra keyed by the max height seen so far (or binary search the cap + BFS, or union-find by sorted cell heights). O(mn log mn).

HOW TO THINK ABOUT IT

The cost of a path is its largest cell, and a larger limit never makes things worse, so there are two equivalent plans: binary search the limit with a flood fill as the check, or run Dijkstra where the cost of a path is its maximum so far (extending a path never lowers it). Say the cost function out loud first: it is the maximum, not the sum.

Basis: LeetCode 778, Swim in Rising Water: the smallest maximum height such that a path from the top-left to the bottom-right cell exists using only cells at or below it. (The related LeetCode 1631 minimises the largest difference between adjacent cells, see the follow-up.)

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(grid):
    best = INF
    for path in all_paths(grid, START, GOAL):
        peak = max(grid[r][c] for r, c in path)
        best = min(best, peak)
    return best
```

BRUTE FORCE

TIME	SPACE
exponential	O(path)

WHY IT IS SLOW

Exponential in the grid size. Dijkstra with 'cost = largest cell so far', or binary search on the limit with a flood fill, is O(m * n log(m * n)).

OPTIMAL SOLUTION · Dijkstra with cost = maximum cell on the path

TIME	SPACE
O(m * n log(m * n))	O(m * n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The cost is the largest difference between adjacent cells on the path (LeetCode 1631).

Same Dijkstra with max, but the step cost is abs(height difference) instead of the cell value. The binary-search version also works: the check allows an edge only if the difference is at most the limit.

FOLLOW-UP 2 Water rises over time (the cell is usable from time H[r][c]).

That is exactly the maximum-cell cost above; the answer is the time at which the corner becomes reachable.

FOLLOW-UP 3 Return the path, not only the value.

Remember each cell's parent at its last improvement and rebuild the path from the target; for the binary-search version run the flood fill once more at the answer limit with parent pointers.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Minimize maximum height along a grid path
#
# This question evaluates algorithmic pathfinding and
# graph-modeling skills, specifically reasoning about
# grid-based route selection where the objective is
# defined by an aggregate maximum over visited cell
# heights.
#
# FOLLOW-UPS (answered below, in this order):
# 1. The cost is the largest difference between adjacent
# cells on the path (LeetCode 1631).
# 2. Water rises over time (the cell is usable from time
# H[r][c]).
# 3. Return the path, not only the value.
```

```
#
# INTERVIEW NOTES
# The cost of a path is its largest cell, and a larger
# limit never makes things worse, so there are two
# equivalent plans: binary search the limit with a flood
# fill as the check, or run Dijkstra where the cost of a
# path is its maximum so far (extending a path never
# lowers it). Say the cost function out loud first: it is
# the maximum, not the sum.
#
# BRUTE FORCE (say it first): Try every simple path
# Depth-first search over all simple paths from the top-
# left to the bottom-right, tracking the largest cell,
# with pruning when it cannot beat the best. Exact and the
# oracle; exponential in the grid size. Time exponential.
# Space O(path).


import heapq


# BRUTE FORCE (say it first): Try every simple path | time
# exponential
def min_max_path_brute(H):
    m, n = len(H), len(H[0])
    best = [float("inf")]

    def go(r, c, seen, worst):
        worst = max(worst, H[r][c])
        if worst >= best[0]:
            return
```

```
        if (r, c) == (m - 1, n - 1):
            best[0] = worst
            return
        for nr, nc in (
            (r + 1, c),
            (r - 1, c),
            (r, c + 1),
            (r, c - 1),
        ):
            if (
                0 <= nr < m
                and 0 <= nc < n
                and (nr, nc) not in seen
            ):
                go(nr, nc, seen | {(nr, nc)}, worst)

    go(0, 0, {(0, 0)}, 0)
    return best[0]


# HELPERS for the base question
INF = float("inf")


def around(r, c, m, n):
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        a, b = r + dr, c + dc
        if 0 <= a < m and 0 <= b < n:
            yield a, b


def reachable(H, limit):
    # flood fill using only cells with height <= limit
    m, n = len(H), len(H[0])
    if H[0][0] > limit:
        return False
    seen, stack = {(0, 0)}, [(0, 0)]
    while stack:
        r, c = stack.pop()
        if (r, c) == (m - 1, n - 1):
            return True
```

```
        for a, b in around(r, c, m, n):
            if (a, b) not in seen and H[a][b] <= limit:
                seen.add((a, b))
                stack.append((a, b))
    return False


# BASE QUESTION (optimal): Dijkstra with cost = maximum
# cell on the path
# Idea: Heap of (largest cell so far, cell). Pop the cell
# with the smallest value, extend to each neighbour with
# cost max(current, neighbour), and keep only
# improvements.
def min_max_path(H, with_path=False):
    # Dijkstra where a path costs the MAX of its cells
    m, n = len(H), len(H[0])
    best, parent = {(0, 0): H[0][0]}, {}
    heap = [(H[0][0], 0, 0)]
    while heap:
        cost, r, c = heapq.heappop(heap)
        if cost > best[(r, c)]:
            continue
        if (r, c) == (m - 1, n - 1):
            if not with_path:
                return cost
            path = [(r, c)]
            while (r, c) in parent:
                r, c = parent[(r, c)]
                path.append((r, c))
            return cost, path[::-1]
        for a, b in around(r, c, m, n):
            nxt = max(cost, H[a][b])
            if nxt < best.get((a, b), INF):
                best[(a, b)] = nxt
                parent[(a, b)] = (r, c)
                heapq.heappush(heap, (nxt, a, b))


# ALTERNATIVE: binary search the answer
def min_max_path_bsearch(H):
    # binary search the answer over the sorted heights
    heights = sorted({h for row in H for h in row})
    lo, hi = 0, len(heights) - 1
    while lo < hi:
        mid = (lo + hi) // 2
        if reachable(H, heights[mid]):
            hi = mid
        else:
            lo = mid + 1
```

```
    return heights[lo]


# FOLLOW-UP 1: The cost is the largest difference between
# adjacent cells on the path (LeetCode 1631).
# Answer: Same Dijkstra with max, but the step cost is
# abs(height difference) instead of the cell value. The
# binary-search version also works: the check allows an
# edge only if the difference is at most the limit.
def min_effort_path(H):
    # LeetCode 1631: cost = the largest height DIFFERENCE
    m, n = len(H), len(H[0])
    best = {(0, 0): 0}
    heap = [(0, 0, 0)]
    while heap:
        cost, r, c = heapq.heappop(heap)
        if cost > best[(r, c)]:
            continue
        if (r, c) == (m - 1, n - 1):
            return cost
        for a, b in around(r, c, m, n):
            nxt = max(cost, abs(H[a][b] - H[r][c]))
```

```
            if nxt < best.get((a, b), INF):
                best[(a, b)] = nxt
                heapq.heappush(heap, (nxt, a, b))


if __name__ == "__main__":
    H = [[0, 2, 1], [1, 3, 1], [2, 2, 1]]
    assert min_max_path(H) == min_max_path_bsearch(H)
    cost, path = min_max_path(H, True)
    assert path[0] == (0, 0) and path[-1] == (2, 2)
    assert max(H[r][c] for r, c in path) == cost
    assert min_max_path([[7]]) == 7
    grid = [[0, 1, 2], [4, 3, 5], [8, 7, 6]]
    assert min_max_path(grid) == 6
    assert min_max_path_bsearch(grid) == 6
    assert min_effort_path([[1, 2, 2], [3, 8, 2],
                            [5, 3, 5]]) == 2
```

FUNCTION AND TEST CASES

Sample call

```
solution([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
```

Test cases

Input	Expected
`([[1, 2, 3], [4, 5, 6], [7, 8, 9]],)`	`9`
`([[8, 4, 7], [6, 5, 9], [3, 2, 1]],)`	`8`

ALSO REPORTED · 1 more

Path Minimizing the Maximum Cell (Swim in Rising Water) | 1Point3Acres | onsite-coding | asked 2026-07-08 | Low

https://www.1point3acres.com/interview/problems/company/google/path-min-max-grid-dijkstra

The clean solution is a Dijkstra variant: priority queue keyed on the running maximum, relax with new = max(dist[cur], height[nxt]). The same skeleton handles the safety variant with edge values in place of cell heights, plus the admissibility filter. Also mentionable: binary-search the answer + BFS/DFS reachability check — but Dijkstra is the cleaner one-pass solution.

The grid prompt minimizes a maximum rather than a sum. Be ready to justify why Dijkstra still works: the path cost is monotone as a path grows, so extending a settled vertex never lowers its committed cost.

The safety version preserves the bottleneck-path shape but changes cells to weighted edges and adds an explicit admissibility threshold.

For the plain shortest-path follow-up, do standard Dijkstra with a min-heap; skip stale priority-queue entries (if curDist != dist[u]: continue) and account for unreachable nodes.

Preparation: Drill the grid and weighted-graph variants back to back, stating the path-cost invariant before implementation. Practice proving correctness for a monotone max-along-path objective and test source-equals-target, disconnected graphs, and values exactly at the threshold.

STUDY HERE: topic 1, Heaps, Stacks and Queues

HARD | IN APP LC 218 | PracHub | Technical Screen | 33 of 33

Compute city skyline outline

POSTED 2025-12-08 | 316 DAYS AGO | OLDER

STUDY: TALK ALL TIME 29.8 30 DAYS 0 PRIORITY 125/228

ABOUT THIS QUESTION

Topic: Heaps, Stacks and Queues

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #218 The Skyline Problem

LeetCode match: LC 218 The Skyline Problem

Posted on PracHub: 2025-12-08 (updated 2026-06-26)

Original:
https://prachub.com/coding-questions/compute-city-skyline-outline

QUESTION

You are given a list of rectangular buildings in a 2D city skyline. Each building is represented by three integers `[L, R, H]`:

- `L`: the x-coordinate of the left edge of the building,
- `R`: the x-coordinate of the right edge of the building (`L < R`),
- `H`: the height of the building. All buildings stand on a flat horizontal line (y = 0) and extend vertically up to height `H`. Buildings may overlap in the x-dimension. Your task is to compute the *skyline* formed by these buildings, as seen from a distance. The skyline should be represented as a list of **key points `[x, height]`** that describe the outer contour of the union of all buildings. A **key point** is a point where the height of the skyline changes. The output list must satisfy all of the following:

1. The list is sorted by increasing `x`.
2. No two consecutive points have the same height (i.e., the height must change at each key point).
3. The first point has a height greater than 0.
4. The last point has height 0, marking the end of the skyline.

Input

- An integer **n** — the number of buildings.
- A list of **n** buildings, where each building is given as a triple `[L, R, H]` of integers. You may assume:
- `1 <= n <= 10^4`
- `0 <= L < R <= 10^9`
- `1 <= H <= 10^9`

Output

Return a list of key points representing the skyline. Each key point is a pair `[x, height]` (or a 2-element array/tuple) of integers.

Example

Input:

- `n = 3` - buildings = `[[2, 9, 10], [3, 7, 15], [5, 12, 12]]` One valid output:

QUESTION

- `[[2, 10], [3, 15], [7, 12], [12, 0]]`
  Explanation (informal):
- From x = 2 to 3, the highest building has height 10.
- From x = 3 to 7, the highest building has height 15.
- From x = 7 to 12, the remaining highest building has height 12.
- After x = 12, there are no buildings, so the height drops to 0. Design an algorithm to compute the skyline key points for the given buildings.

APPROACH

Sweep line over building edges with a max-heap of active heights (lazy deletion); emit a point whenever the max height changes. O(n log n). Divide and conquer also works.

HOW TO THINK ABOUT IT

The skyline height at x is the tallest building covering x, and it can only change at a building's left or right edge. So sweep over those x positions in order, keep the active buildings in a max-heap by height, and report a key point whenever the maximum changes. The details that matter are the event order at the same x and removing finished buildings lazily.

Basis: LeetCode 218, The Skyline Problem.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(buildings):
    xs = set()
    for L, R, H in buildings:
        xs |= {L, R}
    out, prev = [], 0
    for x in sorted(xs):
        h = 0
        for L, R, H in buildings:
            if L <= x < R:
                h = max(h, H)
        if h != prev:
            out.append([x, h])
            prev = h
    return out
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Each edge scans all buildings, O(n^2). A sweep with a max-heap of active buildings and lazy deletion is O(n log n).

OPTIMAL SOLUTION · Sweep line with a max-heap and lazy deletion

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Divide and conquer instead of a heap.

Build the skyline of each half and merge the two by sweeping both lists with the current height of each; the merged height is the max of the two. T(n) = 2T(n/2) + O(n) = O(n log n), same as the heap.

FOLLOW-UP 2 Coordinates up to 10^9 or buildings of a very large count.

Only the edge coordinates matter, so nothing depends on the coordinate range. The sweep is O(n log n) in the number of buildings; use a balanced tree or a heap with lazy deletion for the active set.

FOLLOW-UP 3 Output the area under the skyline.

Sum height * (next x - x) over consecutive key points: one extra pass over the output.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Compute city skyline outline
#
# You are given a list of rectangular buildings in a 2D
# city skyline. Each building is represented by three
# integers [L, R, H]: - L: the x-coordinate of the left
# edge of the building, - R: the x-coordinate of the right
# edge of the building (L < R), - H: the height of the
# building. All buildings stand on a flat horizontal line
# (y = 0) and extend vertically up to height H. Buildings
# may overlap in the x-dimension. Your task is to compute
# the *skyline* formed by these buildings, as seen from a
# distance. The skyline should be represented as a list of
# **key points** [x, height] that describe the outer
# contour of the union of all buildings. A **key point**
# is a point where the height of the skyline changes. The
# output list must satisfy all of the following: 1. The
# list is sorted by increasing x. 2. No two consecutive
# points have the same height (i.e., the height must
# change at each key point). 3.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Divide and conquer instead of a heap.
# 2. Coordinates up to 10^9 or buildings of a very large
# count.
# 3. Output the area under the skyline.
#
# INTERVIEW NOTES
# The skyline height at x is the tallest building covering
# x, and it can only change at a building's left or right
# edge. So sweep over those x positions in order, keep the
# active buildings in a max-heap by height, and report a
# key point whenever the maximum changes. The details that
# matter are the event order at the same x and removing
# finished buildings lazily.
#
# BRUTE FORCE (say it first): Compute the height at every
# edge coordinate
# Collect all edge x-coordinates in order. At each one the
# height is the maximum height among buildings with L <= x
# < R. Output a point where the height differs from the
# previous one. Correct and the oracle; each coordinate
# scans all buildings. Time O(n^2). Space O(n).


import heapq
```

```
# BRUTE FORCE (say it first): Compute the height at every
# edge coordinate | time O(n^2)
def skyline_brute(buildings):
    xs = sorted(
        {x for l, r, h in buildings for x in (l, r)}
    )
    out, prev = [], 0
    for x in xs:
        h = max(
            (
                hh
                for l, r, hh in buildings
                if l <= x < r
            ),
            default=0,
        )
        if h != prev:
            out.append([x, h])
            prev = h
    return out


# HELPERS for follow-up 1
def merge(a, b):
    # divide and conquer: merge two skylines with the
    # current
    # height of each; the result height is their maximum
    out, i, j, ha, hb = [], 0, 0, 0, 0
    while i < len(a) or j < len(b):
        take_a = i < len(a) and (j == len(b)
                                 or a[i][0] <= b[j][0])
        if take_a:
            x, ha = a[i]
            i += 1
        else:
            x, hb = b[j]
            j += 1
        # when both change at the same x, wait for the
        # second
        if i < len(a) and a[i][0] == x:
            continue
        if j < len(b) and b[j][0] == x:
            continue
```

```
        top = max(ha, hb)
        if not out or out[-1][1] != top:
            out.append([x, top])
    return out


# BASE QUESTION (optimal): Sweep line with a max-heap and
# lazy deletion
# Idea: Make events (x, -height) for left edges and (x, 0,
# right) for right edges, sorted so that at the same x
```

```
# taller starts come first and ends come after starts.
# Keep a heap of (-height, right edge).
def skyline(buildings):
    # buildings are [left, right, height]
    # start events carry -height so taller starts sort
    # first;
    # end events carry 0
    events = sorted([(l, -h, r) for l, r, h in buildings]
                    + [(b[1], 0, 0) for b in buildings])
    out = []
    # the active buildings: (-height, right edge), ground
    # first
    heap = [(0, float("inf"))]
    for x, neg_h, right in events:
        # buildings that ended at or before x are gone
        while heap[0][1] <= x:
            heapq.heappop(heap)
        if neg_h:
            heapq.heappush(heap, (neg_h, right))
        top = -heap[0][0]
        # a key point only where the height changes
        if not out or out[-1][1] != top:
            out.append([x, top])
    return out


# FOLLOW-UP 1: Divide and conquer instead of a heap.
# Answer: Build the skyline of each half and merge the two
# by sweeping both lists with the current height of each;
# the merged height is the max of the two. T(n) = 2T(n/2)
# + O(n) = O(n log n), same as the heap.
def skyline_dc(buildings):
    if len(buildings) == 1:
        l, r, h = buildings[0]
        return [[l, h], [r, 0]]
```

```
    if not buildings:
        return []
    mid = len(buildings) // 2
    return merge(skyline_dc(buildings[:mid]),
                 skyline_dc(buildings[mid:]))


# FOLLOW-UP 3: Output the area under the skyline.
# Answer: Sum height * (next x - x) over consecutive key
# points: one extra pass over the output.
def area_under(points):
    # sum of height * width between consecutive key points
    return sum(h * (points[i + 1][0] - x)
               for i, (x, h) in enumerate(points[:-1]))


if __name__ == "__main__":
    b = [[2, 9, 10], [3, 7, 15], [5, 12, 12],
         [15, 20, 10], [19, 24, 8]]
    want = [[2, 10], [3, 15], [7, 12], [12, 0], [15, 10],
            [20, 8], [24, 0]]
    assert skyline(b) == want
    assert skyline_dc(b) == want
    flat = skyline([[0, 2, 3], [2, 5, 3]])
    assert flat == [[0, 3], [5, 0]]
    assert skyline([]) == []
    # 10*1 + 15*4 + 12*5 + 10*5 + 8*4 = 212
    assert area_under(want) == 212
    assert skyline([[1, 2, 1], [1, 2, 2], [1, 2, 3]]) == [
        [1, 3], [2, 0]]
```

FUNCTION AND TEST CASES

Sample call

```
getSkyline(3, [[2, 9, 10], [3, 7, 15], [5, 12, 12]])
```

Test cases

Input	Expected
(3, [[2, 9, 10], [3, 7, 15], [5, 12, 12]])	[[2, 10], [3, 15], [7, 12], [12, 0]]
(1, [[0, 2, 3]])	[[0, 3], [2, 0]]